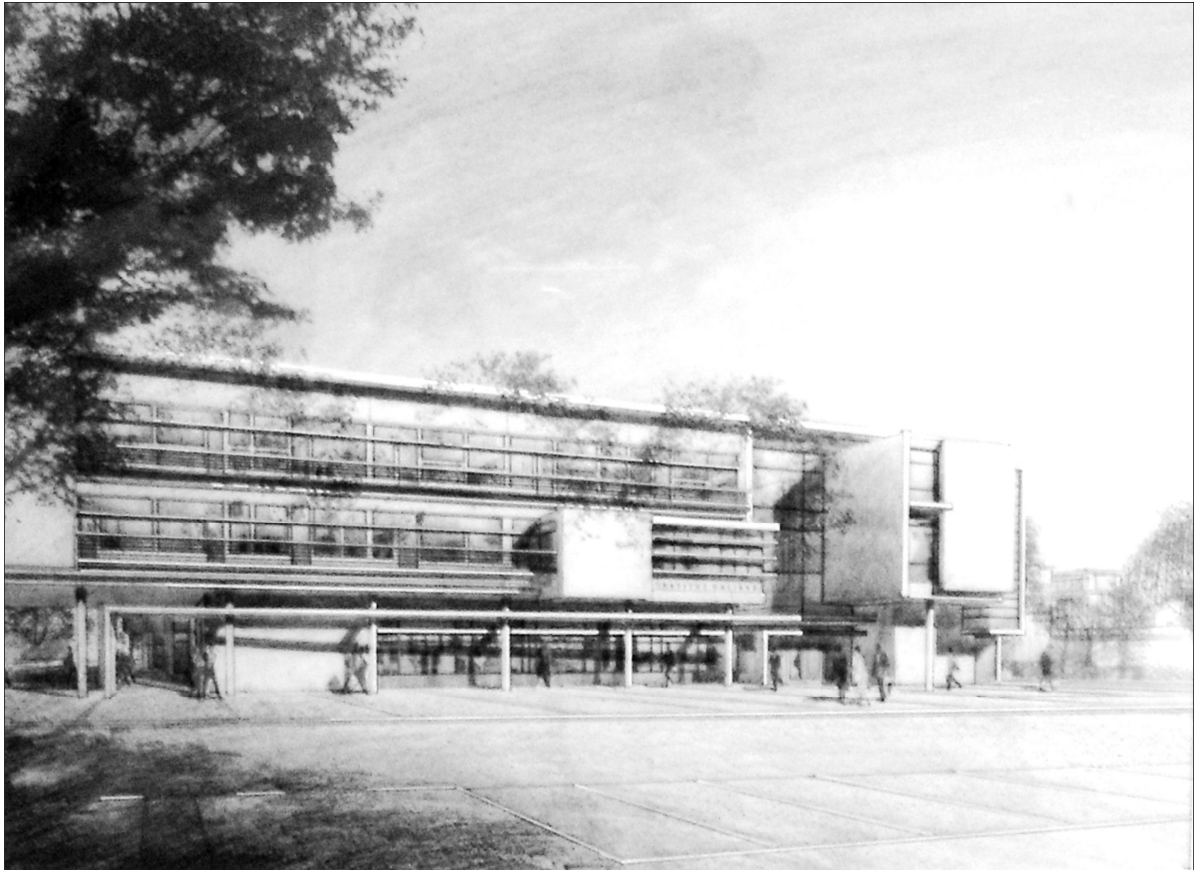




ASPECTS MATERIELS & LOGICIELS DU DSP 21061 DE ANALOG DEVICES

RESUME DU COURS

THIEBAUT Bernard

« PROCESSEURS & CO »

Pour simplifier, il existe **4 grandes familles** :

- ① les microprocesseurs,
- ② les microcontrôleurs,
- ③ les processeurs spécialisés,
- ④ les DSP (ou Digital Signal Processor).

(même si les différences entre ces familles tendent à diminuer !)

QUESTION INITIALE :

**COMMENT S'Y RETROUVER
DANS LA « JUNGLE »
DE CES DIFFERENTES FAMILLES
DE PROCESSEURS ?**

CELA DEPEND EN FAIT DE L'APPLICATION !!!

PROCESSEURS & CO (suite)

① MICROPROCESSEURS

⇒ pour des environnements avec beaucoup de mémoire vive
gros système d'exploitation: mono/multitâche, temps réel ou pas
travaux de natures différentes (pas de spécialisation à priori)
relié à différents types d'interfaces :

clavier, écran, souris, USB, disques, graveurs, webcam...
historiquement avec une architecture de type **Von Neuman**

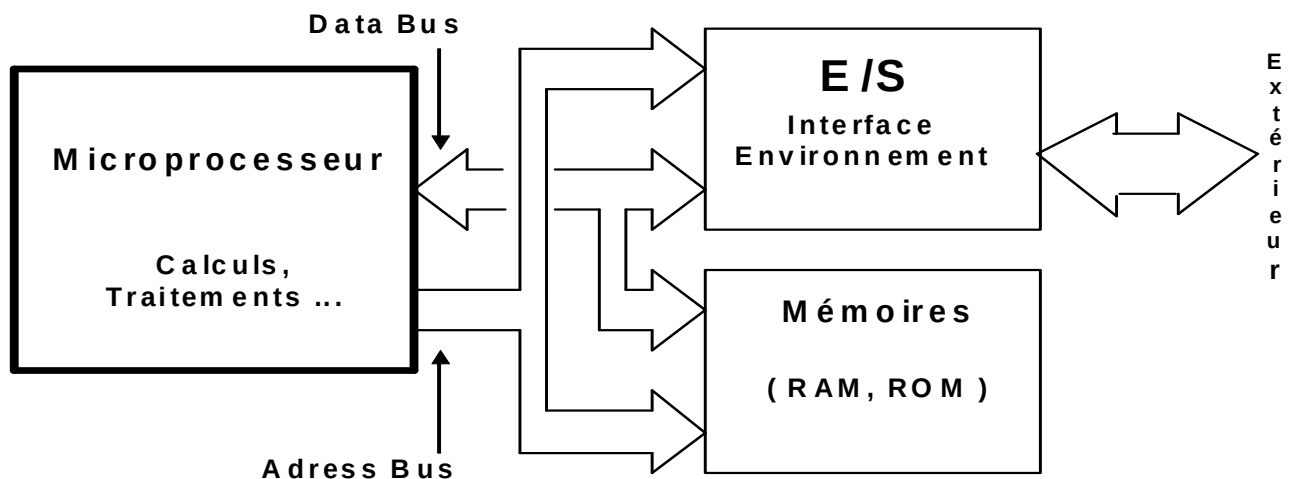


Figure I.1. (page 3)

Au final, le domaine par excellence de la micro-informatique.

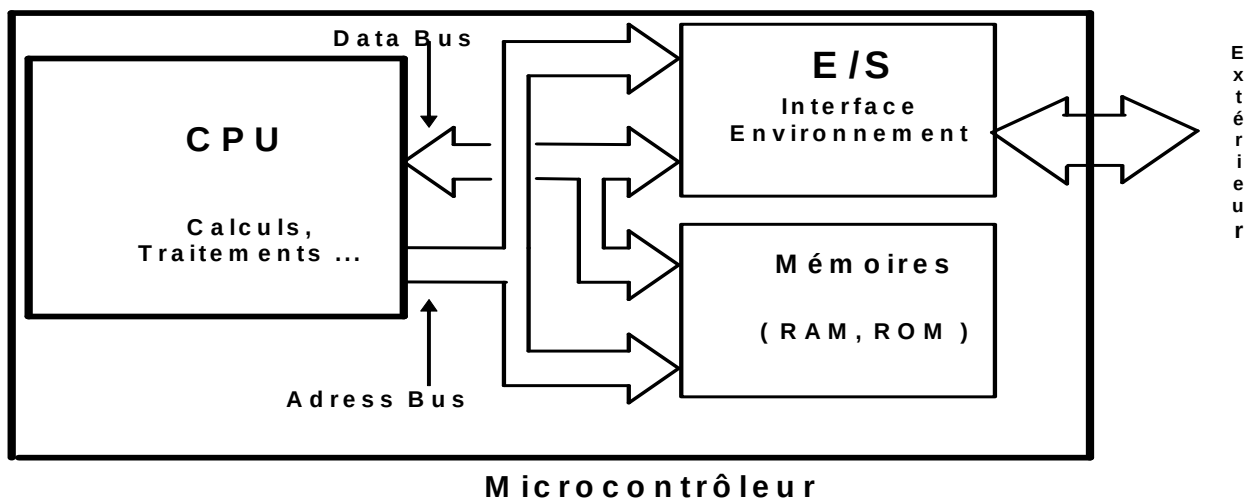
APPLICATIONS TYPE :

- ⇒ CŒUR DES MICROORDINATEURS,
- ⇒ DANS LES SERVEURS,
- ⇒ DANS LES STATIONS DE TRAVAIL ...

PROCESSEURS & CO (suite)

② MICROCONTRÔLEURS

- ⇒ moins de mémoire vive
moins de « gros » calculs
mais plutôt beaucoup d'entrées/sorties : capteurs et actionneurs
souvent (mais pas toujours) architecture de type **Von Neuman**
intégration sur une même « puce » de :
 la CPU
 des mémoires
 des périphériques...
d'où des systèmes dits « single chip » à hardware simplifié.



CPU :
Central Process Unit :là où sont faits les divers traitements
C'est le « cœur » des microcontrôleurs

Figure I.2. (page 3)

Au final, pour avoir des systèmes autonomes et à bas coût.

APPLICATIONS TYPE :

- ⇒ SYSTEMES EMBARQUES,
- ⇒ ELECTRONIQUE PORTABLE,
- ⇒ AUTOMATISMES INDUSTRIELS DELOCALISES ...

PROCESSEURS & CO (suite)

③ PROCESSEURS SPECIALISES

⇒ pour « soulager » le microprocesseur de certaines tâches
cf. bloc E/S Interface Environnement figure I.1. (page3)

Au final, pour accélérer le fonctionnement du système complet.

APPLICATIONS TYPE :

- ⇒ CONTRÔLEURS DE CLAVIER, DE DISQUES...
- ⇒ PROCESSEURS VIDEO,
- ⇒ PROCESSEURS DE CALCUL,
- ⇒ GESTIONNAIRE RESEAU, USB,
- ⇒ CONTRÔLEURS DE DMA ...

PROCESSEURS & CO (suite)

④ « DIGITAL SIGNAL PROCESSOR »

- ⇒ processeur dédié au traitement numérique du signal,
contrainte inhérente : le calcul « temps réel »,
mise en place de solutions hardware pour être plus rapide :
architecture de type **Harvard** (comme dans les PIC)
utilisation d'un « pipe » instructions
développement du parallélisme
unités de calcul internes indépendantes
optimisation des boucles...

**Au final, pour pouvoir implanter des algorithmes temps réel
de traitement numérique du signal de plus en plus sophistiqués.**

APPLICATIONS TYPE :

- ⇒ TRAITEMENT DU SON, DE L'IMAGE,
- ⇒ TECHNIQUES DE COMPRESSION (VOIX, PHOTOS...),
- ⇒ TELEPHONIE NUMERIQUE,
- ⇒ MODEM HAUTS DEBITS,
- ⇒ IMAGERIE MEDICALE,
- ⇒ SYSTEMES BIOMETRIQUES,
- ⇒ CRYPTAGE DE DONNEES...

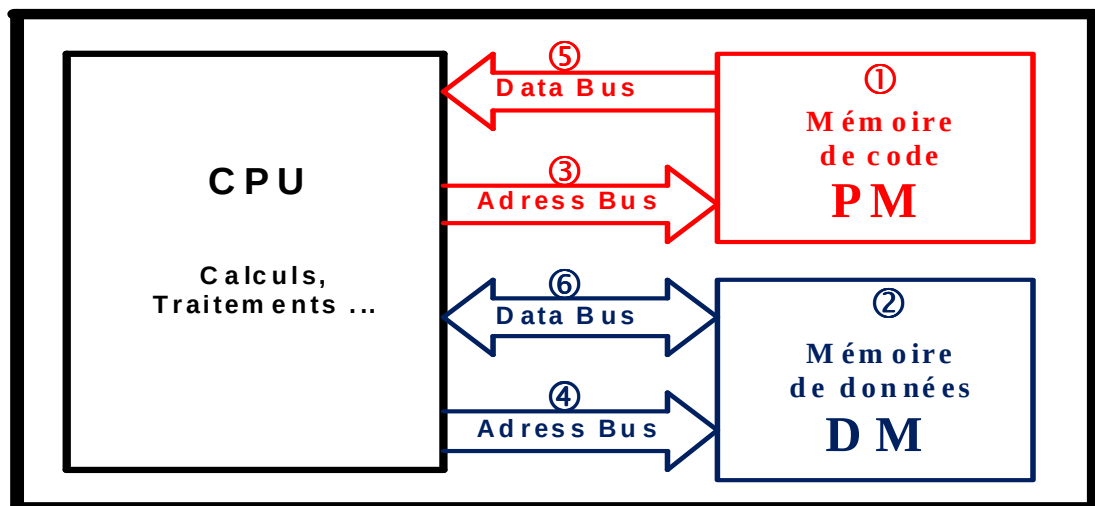
DIGITAL SIGNAL PROCESSOR (suite)

ARCHITECTURE HARVARD = vitesse $\times 2$!

Deux blocs de mémoire **différents** et **indépendants** :

① un premier bloc pour contenir la **liste des instructions** du programme :
⇒ c'est la **mémoire de programme** (appelée souvent **PM**),

② un second bloc pour contenir les **opérandes des instructions** :
⇒ c'est la **mémoire de données** (appelée souvent **DM**).



Architecture Harvard dans un DSP

Figure I.3. (page 5)

D'où l'existence de **4 bus** :

- ③ un **bus d'adresses** pour accéder aux cases de la mémoire de code,
- ④ un **bus d'adresses** pour accéder aux cases de la mémoire de données,
- ⑤ un **bus de données** pour lire le contenu des cases de la mémoire de code,
- ⑥ un **bus de données** pour lire / écrire dans la mémoire de données.

Ces bus fonctionnent de manière **indépendante**, permettant dans un même cycle de bus :

- ⇒ d'accéder à une instruction dans la mémoire de code **PM**,
- ⇒ et de lire ou d'écrire dans la mémoire de données **DM** pour accéder à une variable ou pour la modifier.

DIGITAL SIGNAL PROCESSOR (suite)

UN « PIPE » INSTRUCTIONS = vitesse $\times 3$!

A chaque tick d'horloge q (entier naturel représentant les périodes du quartz système), les 3 actions suivantes sont faites en même temps :

exécution de l'instruction (q) :

↳ par **l'unité d'exécution**,

décodage de l'instruction suivante ($q+1$) :

↳ par **l'unité de décodage**,

recherche de l'instruction encore suivante ($q+2$) :

↳ par **l'unité de recherche en mémoire de code PM**

Tout en laissant possible un accès simultané pour les données dans la mémoire de données grâce à l'architecture Harvard.

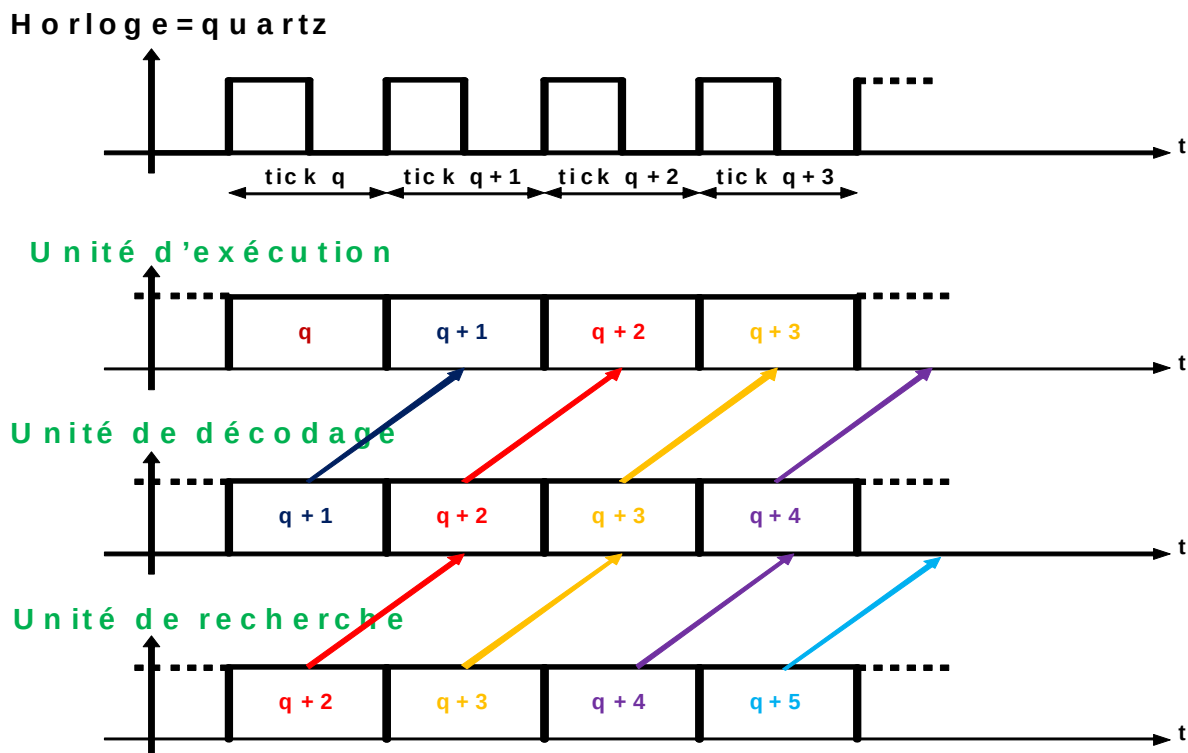


Figure I.4. (page 6)

C'est le principe du fonctionnement dit en « **pipe-line** ».

TRAITEMENT NUMERIQUE DU SIGNAL

Prenons un peu de recul : c'est quoi au fait le TNS ?

Appliquer des algorithmes sur les échantillons prélevés, à chaque instant d'échantillonnage, sur un signal analogique.

Et donc au final travailler avec des nombres !!!

Le **DSP** doit être capable d'effectuer les 4 opérations arithmétiques de base sur ces nombres, et ceci **le plus rapidement possible**.

Exemple (qui nous servira souvent dans ce cours !!)

Equation d'un filtre numérique à Réponse Impulsionnelle Finie :

$$y(n) = \sum_{j=0}^{N-1} a(j).x(n-j)$$

pour calculer la valeur numérique du signal de sortie **y(n)** de ce filtre à l'instant **t_n = n.T_e** en fonction :

① des valeurs numériques du signal d'entrée :

- **x(n)** à l'instant **t_n**,
- **x(n-1)** à l'instant **t_{n-1}**,
- **x(n-2)** à l'instant **t_{n-2}**

② en utilisant les coefficients **a(j)** (pour **j=0** à **N-1**)

⚡ ⚡ ... **Mais attention ne pas confondre :**

l'entier **n** = multiples de la **période d'échantillonnage** **T_e**
(par exemple 125μs si on échantillonne à **F_e = 8 kHz**)

l'entier **q** = multiples de la **période du quartz** du DSP (cf. figure I.4.)
(de l'ordre de quelques ns)

TRAITEMENT NUMERIQUE DU SIGNAL

Prenons un peu de recul : c'est quoi au fait le TNS ? (suite)

Et au final, il faut pouvoir :

⇒ **multiplier** deux opérandes :
par exemple $a(k)$ et $x(n-k)$ pour $j = k$ en un **seul coup d'horloge**,

⇒ **accumuler** le résultat obtenu par cette multiplication :
pour construire au fur et à mesure le « $\sum_{j=0}^k$ » à partir du « $\sum_{j=0}^{k-1}$ »
dans ce **même coup d'horloge**,

⇒ **tout en préparant**, toujours dans ce **même coup d'horloge**,
la recherche des opérandes (coefficient et échantillon) suivants :
 $a(k+1)$ et $x(n-k-1)$ pour $j = k+1 \dots$

Grâce à l'architecture Harvard, le découpage de la mémoire en 2 blocs indépendants permettra de répondre à cette problématique et de façon assez efficace.

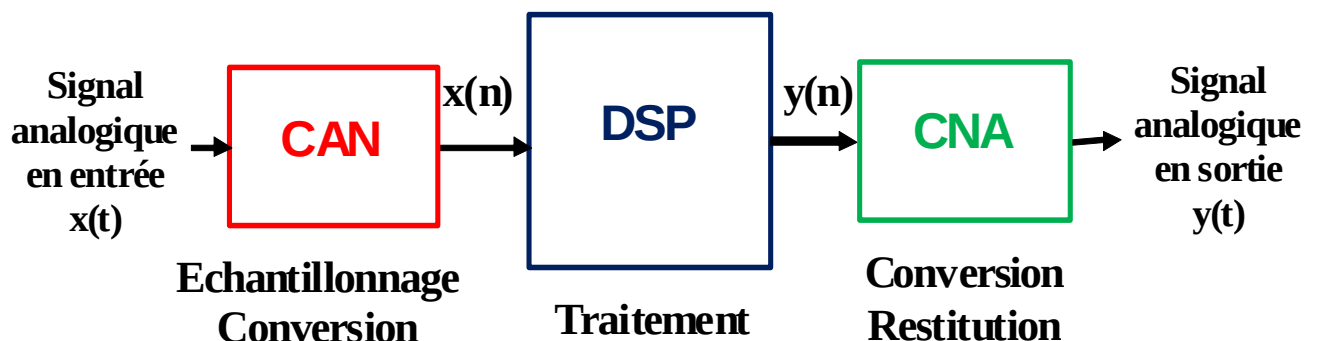
TRAITEMENT NUMERIQUE DU SIGNAL

ASPECTS TEMPS REEL

Le **DSP** et sa carte d'évaluation nous permettront de faire du traitement numérique du signal en **temps réel**, avec de « vrais » signaux analogiques fournis par exemple par un GBF.

Ces signaux seront :

- **échantillonnés** et **convertis** en **numérique** par un **Convertisseur Analogique Numérique (CAN)**,
- pour être **traités** par un programme s'exécutant sur le **DSP**,
- puis **reconvertis** en **analogique** via un **Convertisseur Numérique Analogique (CNA)** et enfin **restitués** en sortie.



Le calcul de $y(n)$ par le DSP doit être fait avant de prélever l'échantillon suivant $x(n+1)$.

Le temps de calcul de $y(n)$ doit donc être inférieur à T_e .

Contrairement à l'approche avec **Matlab** où l'on définit en machine des signaux « virtuels » sur lesquels on vient tester des traitements particuliers, bref où l'on effectue des **simulations** en étant donc **un peu plus éloigné des aspects temps réel**.

TRAITEMENT NUMERIQUE DU SIGNAL

CES TP SONT AU « CARREFOUR » DE PLUSIEURS CHEMINS

- le **cours de TNS** pour les **aspects plus théoriques**,
- les **TP de Matlab** comme outil de **calcul** et de **simulation**,
- **l'informatique** et la **programmation en langage C**.

OBJECTIFS PRINCIPAUX DE CE COURS

Chapitre II. à V. : aspects hardware

Architecture interne,
Principes de fonctionnement,
Utilisation des interruptions,
Codec.

Bref: comment le hardware est conçu pour augmenter la vitesse.

Chapitre VI. et VII. : aspects software

Environnement de développement,
Mise en œuvre en langage C.

Bref: comment utiliser et mettre en œuvre un DSP.

Ce cours est une espèce de résumé (**efficace, condensé et concentré**) de la documentation complète du **DSP** (environ 4000 pages en anglais) pour **comprendre** les outils utilisés en TP et pour pouvoir développer quelques petites applications en **relative autonomie**.

LES DSP CHEZ ANALOG DEVICES

Deux grandes familles de DSP chez Analog Devices :

- la famille **ADSP 21xx** :

↳ sur **16 bits** avec calcul en **virgule fixe**,

- la famille **ADSP 21xxx** :

↳ sur **32/40 bits** avec calcul en **virgule flottante**.

**Résumé des principales caractéristiques
pour quelques éléments de ces 2 familles.**
(tableau bien sûr non exhaustif !!!)

	Réf.	RAM	ROM	Instruc. Time(ns)	S. Port	Timer	Ext. IT	CAN CNA	Volt	Nb. Pin
2 1 x x	ADSP2101	3K	-	60	2	1	3	0	5	80
	ADSP2103	3K	-	100	2	1	3	0	3,3	80
	ADSP2105	1,5K	-	100	1	1	3	0	5	68
	ADSP2106	0,5K	1K	100	1	1	3	0	5	68
	ADSP2111	3K	-	60	2	1	3	1	5	100
	ADSP21msp50/51	3K	2K	77	2	1	3	1	5	144
2 1 x x x	ADSP21010	32K	-	80	2	1	4	0	5	304
	ADSP21020	64K	-	40	2	1	4	0	5	223

Figure I.5. (page 9)

Pour des raisons :

- **historiques** :

↳ des plus anciens aux plus récents,

- de **simplicité** et de **pédagogie** :

↳ des plus simples aux plus complexes,

présentation d'abord des **21xx** puis des **21xxx** dans un second temps

En TP, développements pratiques faits sur un **ADSP 21061**.

LA FAMILLE 21xx

ARCHITECTURE INTERNE

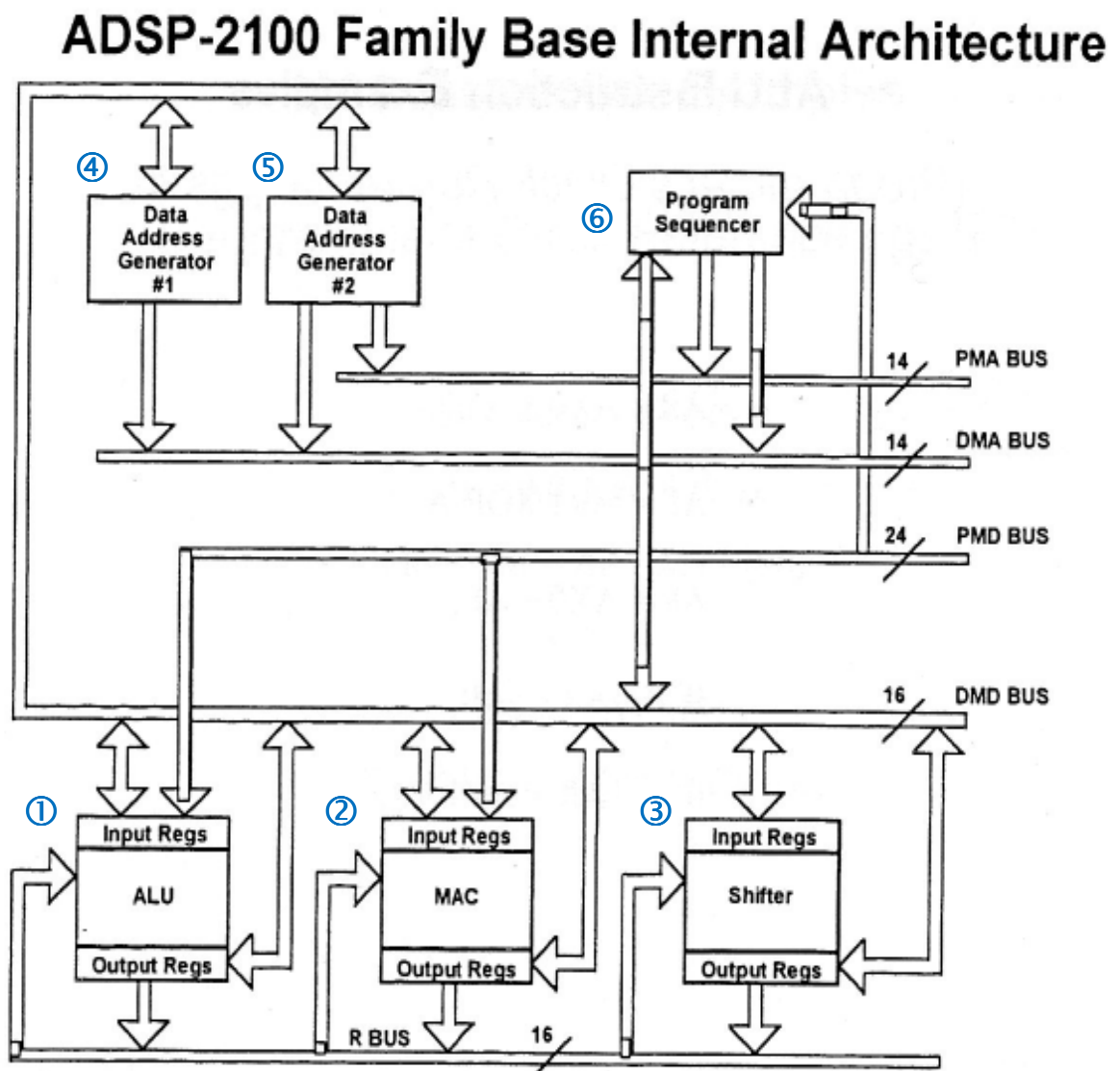


Figure II.1. (page 10)

LA FAMILLE 21xx

ARCHITECTURE INTERNE (suite)

⇒ **six** grands blocs (ou unités) fonctionnels, repérés de ① à ⑥ :

- ① **ALU** : unité de calculs arithmétiques et logiques,
- ② **MAC** : unité de multiplication et d'accumulation,
- ③ **SHIFTER** : unité de décalage,
- ④ **DATA ADDRESS GENERATOR 1** : unité de gestion des adresses pour la mémoire de données,
- ⑤ **DATA ADDRESS GENERATOR 2** : unité de gestion des adresses pour la mémoire de code,
- ⑥ **PROGRAM SEQUENCER** : unité assurant le déroulement du programme.

⇒ **deux** blocs mémoire **indépendants** :
(et n'apparaissant pas sur la figure précédente)

- **PM** (**P**rogram **M**emory) : pour le code (éventuellement données)
- **DM** (**D**ata **M**emory) : pour les données.

⇒ **quatre** bus internes d'interconnexion : (architecture Harvard)

- le **PMA** : **P**rogram **M**emory **A**dressbus,
- le **PMD** : **P**rogram **M**emory **D**atabus,
- le **DMA** : **D**ata **M**emory **A**dressbus,
- le **DMD** : **D**ata **M**emory **D**atabus.

En un seul cycle horloge, le **DSP** peut accéder à la mémoire de code et modifier ou lire des données de la mémoire de données.

Notons aussi le **RBUS** pour les échanges entre les blocs ①, ② et ③.

LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES DONNEES

Elles **fonctionnent en parallèle** pour optimiser les temps de calcul.

ALU : Arithmetic and Logic Unit

Existe dans tous les micro/processeurs/contrôleurs.

Pour réaliser :

- ↪ toutes les opérations **arithmétiques** de base :
addition, soustraction...
 - ↪ et toutes les opérations **logiques** :
fonctions **OR, AND, XOR...**, **complément, comparaison...**
- sur les opérandes, c'est-à-dire les variables du programme :
fournies par les bus **DMD** ou **PMD**.

ALU Block Diagram

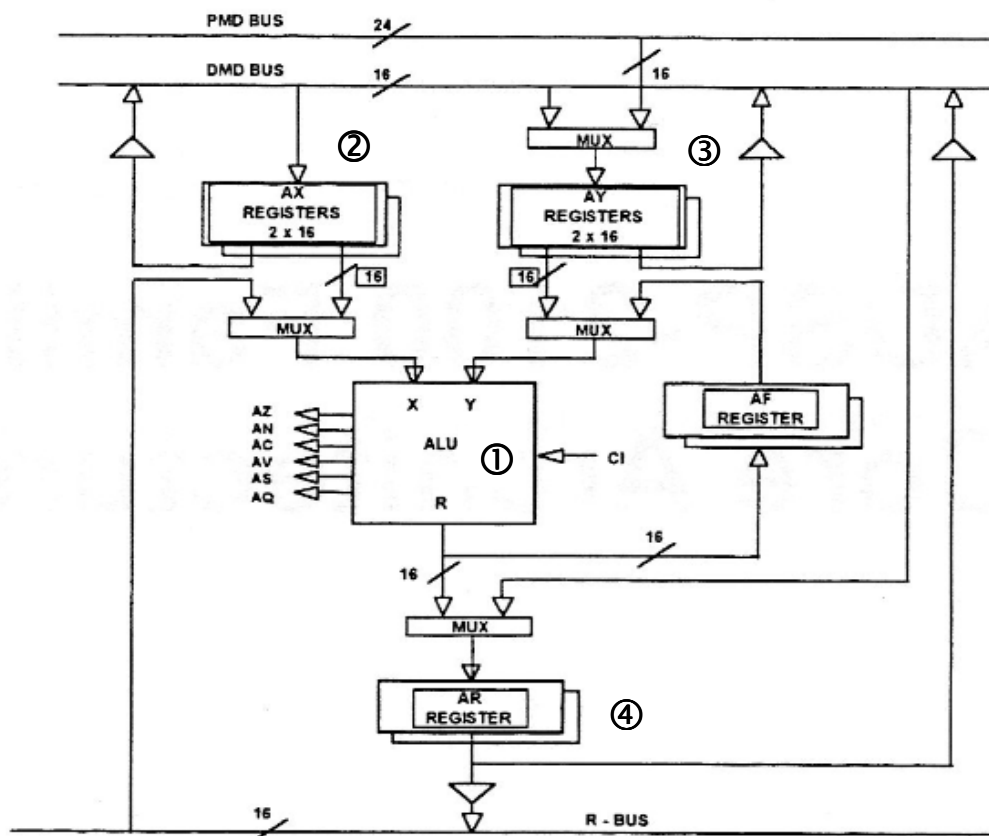


Figure II.2. (page 11)

LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES DONNEES

MAC : Multiplier / ACcumulator Unit

Ce bloc est plus intéressant car il est plus **spécifique** à un DSP.

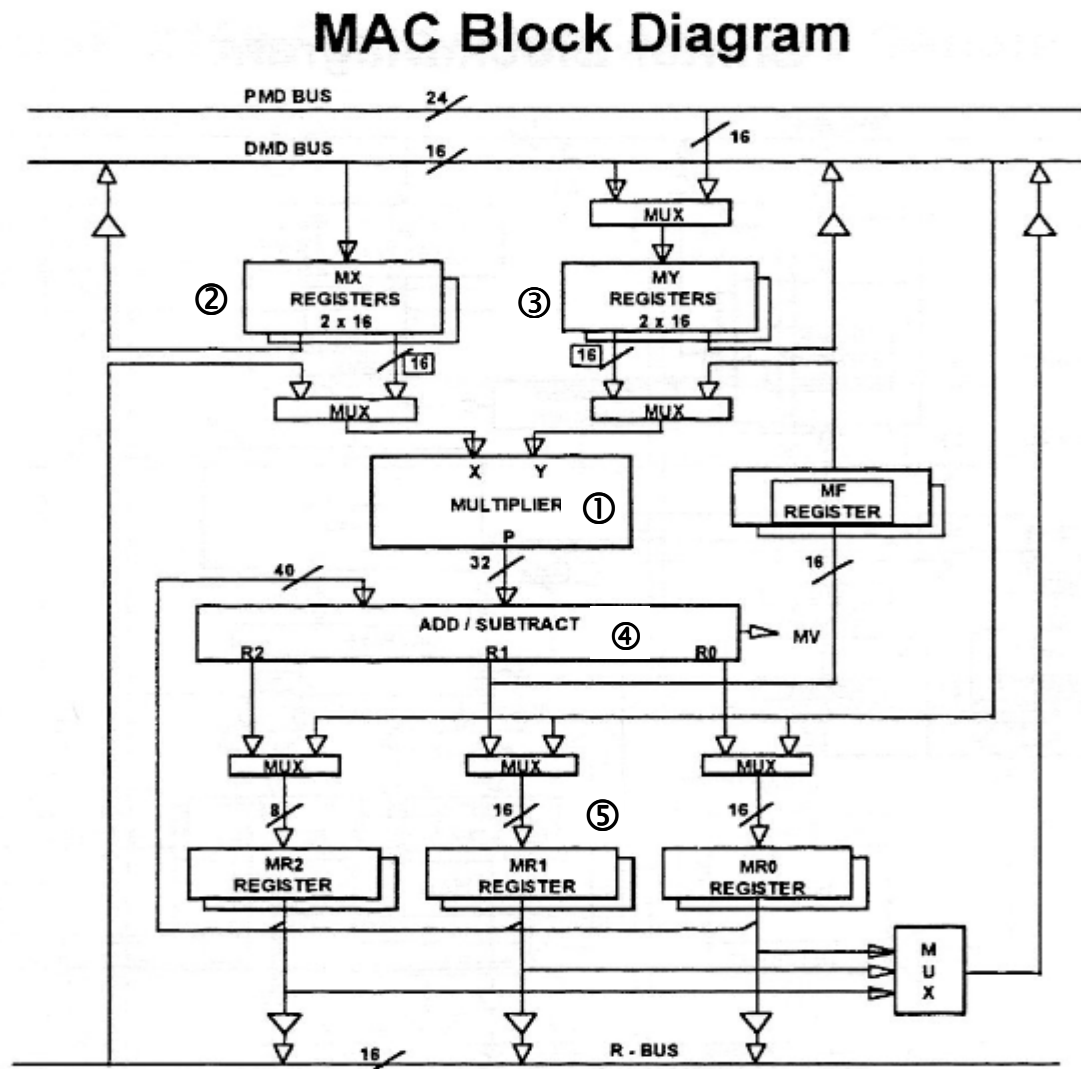


Figure II.3. (page 12)

LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES DONNEES

MAC : Multiplier / ACcumulator Unit (suite)

Par exemple, examinons une instruction assembleur DSP du type :

$$\mathbf{MR} = \mathbf{MR} + \mathbf{MX0*MY0}$$

Elle s'exécute en **un seul cycle** d'horloge et met en œuvre le **MAC**.

Pour implanter facilement un calcul comme :

$$y(n) = \sum_{j=0}^{N-1} a(j).x(n-j)$$

dans une boucle répétée N fois (de j = 0 à N-1).

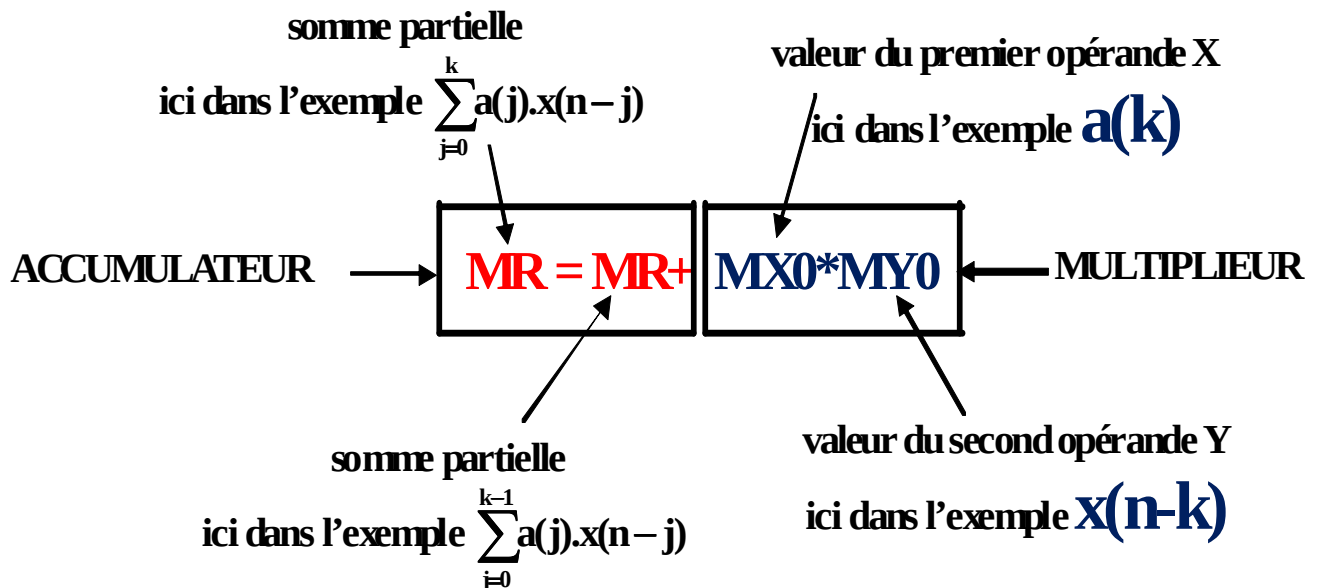


Figure II.4. (page 13)

LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES DONNEES

SHIFTER : SHIFTER Unit

Pour réaliser des décalages de n bits **en un seul tick d'horloge**

↳ soit sur la **gauche** $\equiv$ **multiplication** par 2^n ,

↳ soit sur la **droite** $\equiv$ **division** par 2^n .

Utile par exemple pour des algorithmes de calcul de **FFT**.

Shifter Block Diagram

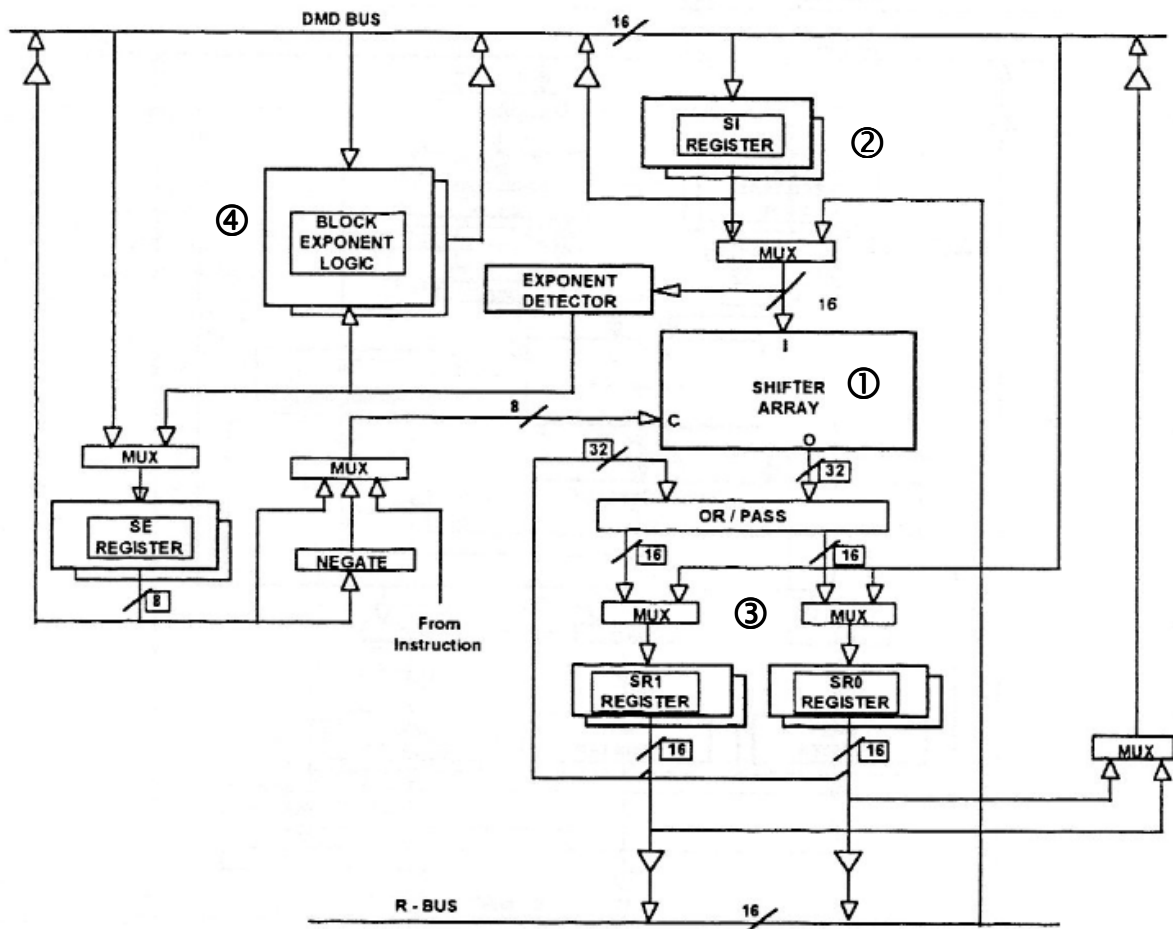


Figure II.5. (page 14)

LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES ADRESSES

Les DAG : Data Address Generator

Il y a deux **DAG** (cf. figure II.1. page 10) :

- **DAG1** pour accès aux données dans la mémoire de données **DM**,
- **DAG2** pour accès aux données dans la mémoire de code **PM**.

Pour faciliter la gestion des **pointeurs** pour les accès des données situées en mémoire de données (**DM**) et en mémoire de code (**PM**) :

↳ **accélération** du calcul d'adresses par **incrémentation / décrémentation** de pointeurs,

↳ **implémentation facilitée** et **accélérée** pour des structures de données plus complexes comme les **buffers circulaires** par exemple.

(qui sont souvent utilisés pour stocker les coefficient **a(j)** et/ou les échantillons numériques **x(n-j)**) comme on va le voir...)

Data Address Generator Block Diagram

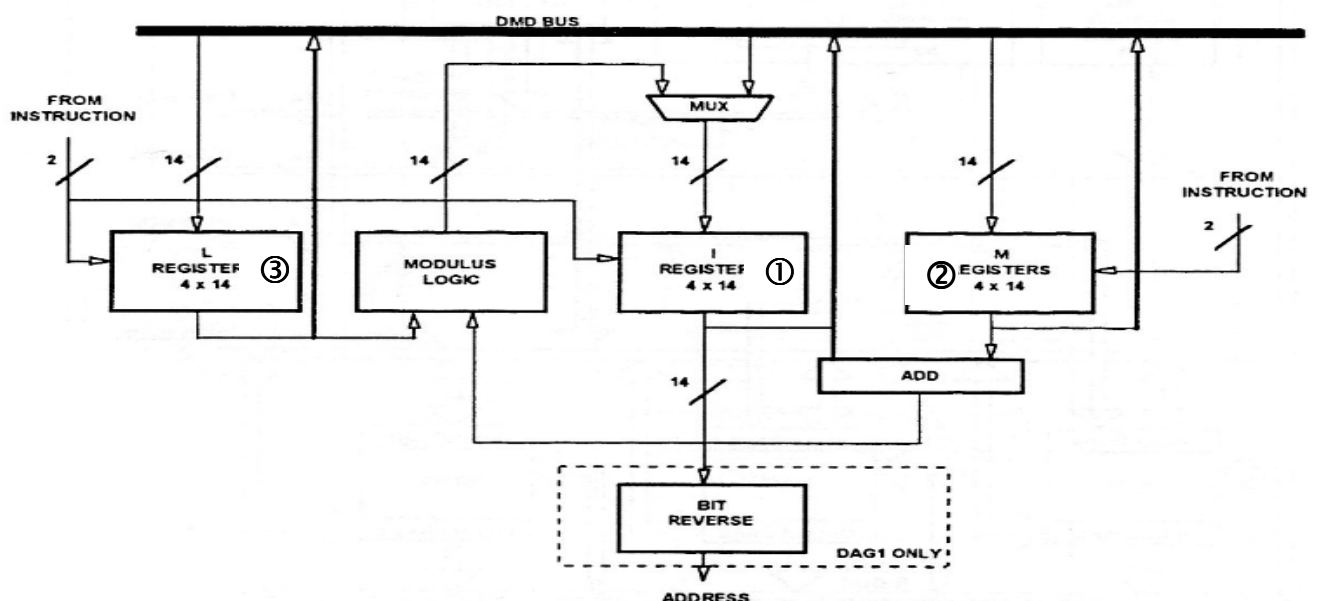


Figure II.6. (page 16)

LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES ADRESSES

Les DAG : Data Address Generator (suite)

PRINCIPE DE FONCTIONNEMENT

L'adresse, en sortie d'un **DAG** est calculée à partir de 3 types de registres :

① 4 registres de type **I** comme **Index**: **I0** à **I3**

↳ contiennent un **pointeur**,

② 4 registres de type **M** comme **Modifier** : **M0** à **M3**

↳ contiennent un **offset** qui, rajouté à un des registres **I** (**I+M** en signé), permet de pointer par exemple sur l'élément suivant ou précédent d'un buffer pour accéder à cet élément. Mais de façon plus générale, pour accéder à n'importe quel élément d'un buffer (faire le lien à l'indice pour un tableau...)

③ 4 registres de type **L** comme **Length** : **L0** à **L3**

↳ pour la gestion des buffers circulaires, par une arithmétique en **modulo Li** : ils contiennent alors la **Longueur** de ce buffer.

Pas clair ?

Alors 3 exemples, du plus simple au plus compliqué !!!

Résumé du cours DSP : chapitre II.

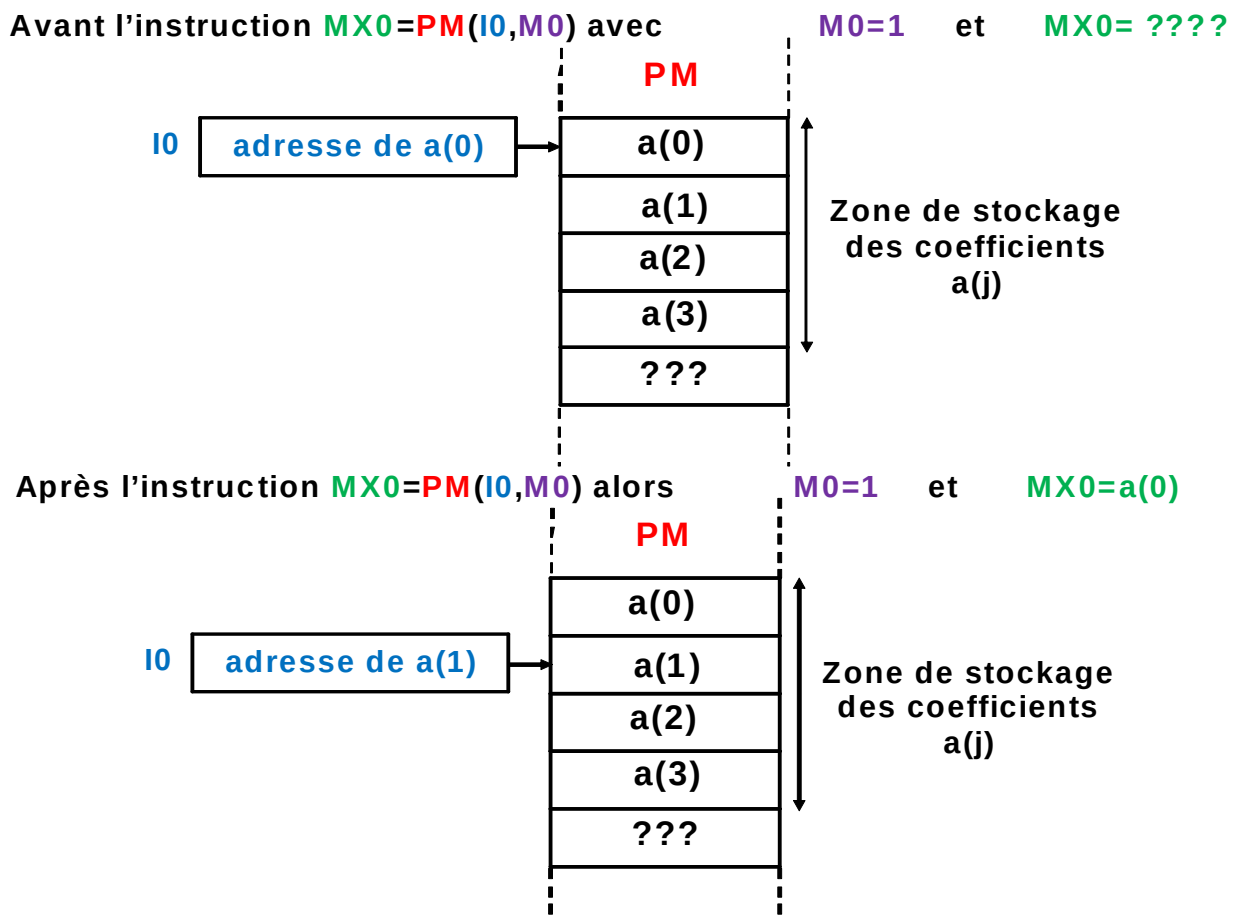
Exemple 1 de base sur un **buffer « linéaire »** : avec **L0 = 0** Instruction **PM(I0,M0)** dans une boucle

Supposons que les coefficients **a(j)** du filtre se trouvent rangée dans un tableau de taille **4** dans la mémoire de code **PM** pointée par **I0**.

L'instruction : **MX0 = PM(I0,M0)** fait 2 choses :

① elle fait tout d'abord un accès à la mémoire **de code** pour charger le registre **MX0** (une des entrée d'opérande du multiplieur) avec la donnée qui se trouve à l'adresse fournie par la valeur courante de **I0** (**pointeur**) : par exemple le coefficient **a(0)**,

② puis elle met ensuite (par post-modification) **automatiquement** la valeur **I0+M0** dans **I0** (pour mettre à jour le pointeur et pouvoir au passage suivant dans la boucle pointer sur le coefficient suivant **a(1)** qui se trouve juste après dans le tableau des coefficients du filtre.



Résumé du cours DSP : chapitre II.

A la fin (au bout de 4 « tours » dans la boucle) **I0** pointera donc sur la case mémoire qui est juste après celle qui contient le dernier coefficient **a(3)**.

Ainsi, pour réutiliser ce buffer de coefficients pour le calcul suivant de $y(n+1)$ qui est du même type, il faudra penser à rajouter une ligne supplémentaire pour initialiser le registre **I0** pour qu'il pointe à nouveau sur le premier coefficient **a(0)** (d'où une perte de temps).

Ainsi il est plus intéressant d'utiliser un buffer circulaire pour stocker les coefficients **a(j)** du filtre et c'est là qu'interviennent les registres **L**.

Exemple 2 : cas d'un **buffer circulaire** (ici de taille 4) Et donc avec **L0 = 4**

L'instruction **MX0 = PM (I0,M0)** fonctionnera de la même manière pour accéder successivement aux différents coefficients, à la seule différence que lorsque le contenu de **I0** a été incrémenté de 3 (c'est-à-dire quand il pointera sur le dernier coefficient **a(3)** après l'avoir chargé dans **MX0**), le bloc modulo du hardware du **DAG** entre en ligne de compte et au lieu de l'incrémenter une fois de plus comme dans le cas précédent, il le remettra à sa valeur initiale pour qu'il pointe à nouveau sur le premier élément du buffer circulaire à savoir **a(0)**.

Et ceci sans écrire de ligne de code supplémentaire !

C'est donc le hardware, plus rapide, qui a été conçu à la base pour remplacer une instruction de programme, et ainsi accélérer le calcul de $y(n)$.

LA FAMILLE 21xx

Exemple 3 : MAC et DAG en même temps

Pour prolonger celui déjà détaillé précédemment sur la figure II.4.

Soit la ligne des trois instructions assembleur suivante :
(syntaxe compacte, caractéristique de la programmation des DSP)

MR=MR+MX0*MY0, MX0=PM(I0,M0), MY0=DM(I1,M1)

Il faut bien comprendre que ces trois instructions se dérouleront en même temps (et non les unes après les autres) et donc en parallèle **pendant le même tick q d'horloge**, car elles font travailler des unités différentes.

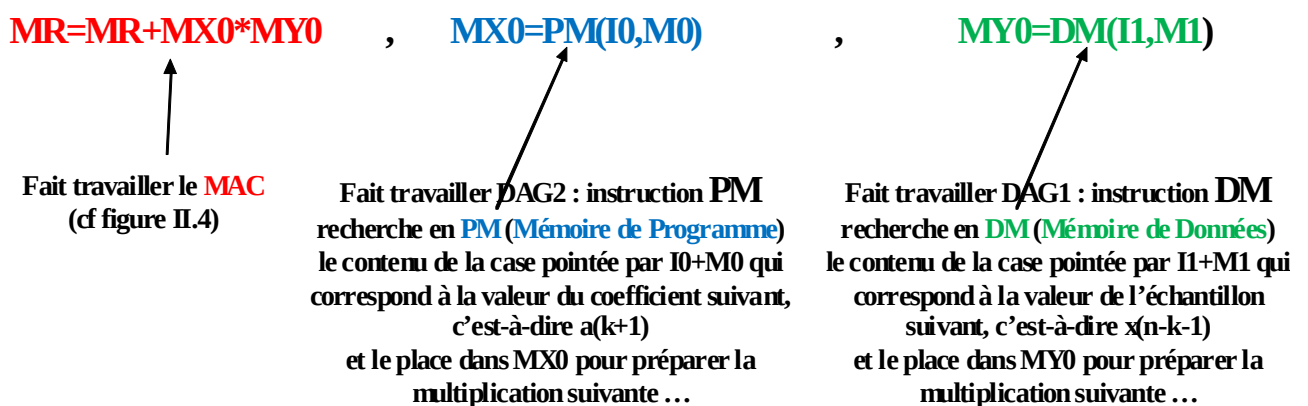


Figure II.7. (page 17)

L'architecture hardware des 21xx (où les différentes unités de calcul sur les données ou sur les adresses travaillent en parallèle) permet une implémentation optimale, compacte et très rapide en temps d'exécution pour les algorithmes de traitement numérique du signal.

Mais cette ligne assembleur pose un problème : lequel ???

LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES ADRESSES

PROGRAM SEQUENCER

Pour optimiser le déroulement d'un programme en fournissant, de façon optimale et à chaque tick **q** d'horloge, l'adresse de la case mémoire PM où se trouve **l'instruction suivante** à exécuter.

C'est en quelque sorte un « super » **PROGRAM COUNTER (PC)**, architecturé autour du registre **PROGRAM COUNTER**.

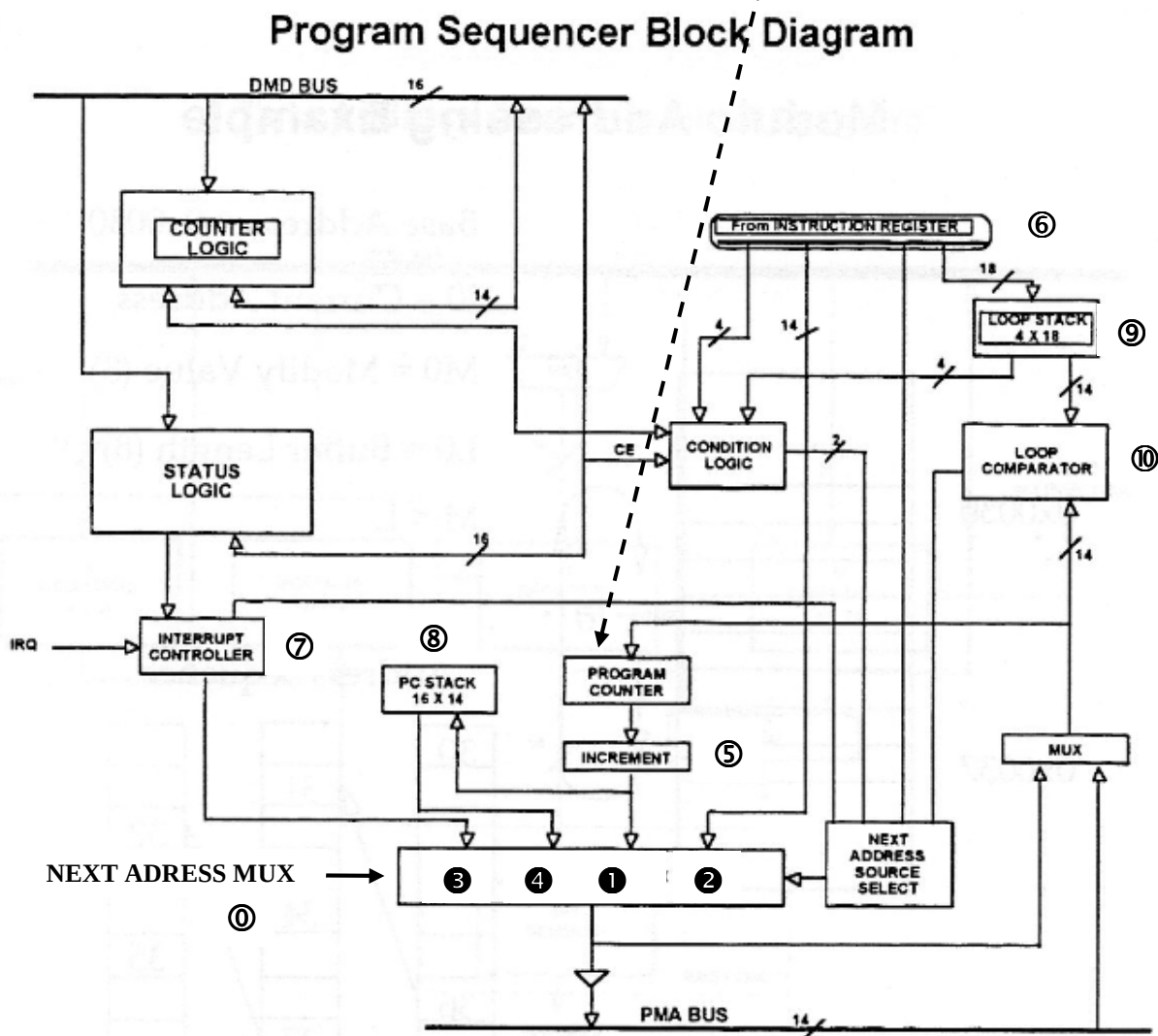


Figure II.8. (page 18)

LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES ADRESSES

PROGRAM SEQUENCER (suite)

De façon générale, il existe **4 possibilités** pour déterminer l'adresse de la prochaine instruction à exécuter :

(cf. bloc ① **NEXT ADDRESS MUX** pour déterminer cette adresse)

Cas ① ⇒ déroulement séquentiel du programme

La prochaine instruction se trouve dans la case mémoire contigüe **suivante** : il suffit d'incrémenter la valeur courante du **PROGRAM COUNTER** (PC), d'où le bloc **INCREMENT** ⑤. Remarquons que c'est ce qui se passe 99% du temps d'exécution...

Cas ② ⇒ rupture de séquence sous contrôle logiciel

L'adresse de la prochaine instruction est issue de l'instruction en cours, elle est fournie par l'**INSTRUCTION REGISTER** ⑥.

Par exemple en cas de branchement (conditionnel ou inconditionnel) à une autre adresse en mémoire ou en cas d'appel de sous-programme...

Cas ③ ⇒ rupture de séquence sous contrôle matériel

L'adresse de la prochaine instruction correspond à celle d'un vecteur d'interruption fourni par un contrôleur d'interruption (**INTERRUPT CONTROLLER** ⑦).

Ce vecteur d'interruption correspond à l'adresse de la première instruction de la routine de traitement d'une interruption matérielle par exemple.

LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES ADRESSES

PROGRAM SEQUENCER (suite)

Cas ④ ⇒ gestion optimale des boucles

Met en œuvre 3 éléments :

- bloc ⑧ : une **PC STACK** (pile de PC),
- bloc ⑨ : une **LOOP STACK** (pile de boucle),
- bloc ⑩ : un **LOOP COMPARATOR** (comparateur de boucle).

Pour gérer efficacement les boucles, fréquentes en TNS, du type :

REPETER LA BOUCLE JUSQU'A CE QUE
LA CONDITION DE SORTIE SOIT VRAIE

ou (idem)

REPETER LA BOUCLE TANT QUE
LA CONDITION DE SORTIE EST FAUSSE

Mais ce test de sortie de boucle entraîne un ralentissement dans le temps d'exécution de la boucle car il faut rajouter des instructions pour vérifier si cette condition de sortie est soit vraie soit fausse.

L'objectif ici, en « sous-traitant » le maximum de choses au hardware, est de ne perdre aucun cycle :

- ni dans les phases d'exécution,
- ni dans les phases de décodage et de recherche d'instruction.

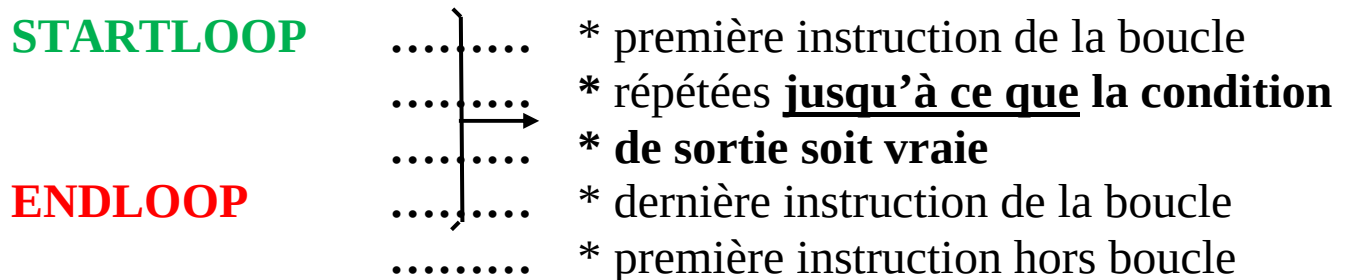
LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES ADRESSES

PROGRAM SEQUENCER (suite)

Exemple :

DO ENDLOOP UNTIL CONDITION



Soit l'équation au récurrence d'un filtre avec 4 coefficients $a(j)$:

$$y(n) = \sum_{j=0}^3 a(j).x(n-j) \quad (1)$$

ADRESSES	INSTRUCTIONS	COMMENTAIRES
	CNTR =4	Initialisation du compteur de boucle. Boucle répétée 4 fois car 4 coefficients dans l'équation.
	DO ENDLOOP UNTIL CE	A chaque boucle CNTR est automatiquement décrémentée. Quand CNTR=0 le flag CE (Conter Expired $\cong$ compteur à 0) passe à 1. Mais tant que CE reste à 0, on refait la boucle une fois de plus.
STARTLOOP	MR=MR+MX0*MY0, MX0=PM(I0,M0), MY0=DM(I1,M1)	Première instruction de la boucle. Corps de boucle pour le calcul de $y(n)$ et la recherche du coefficient $a(j)$ et de l'échantillon $x(n-j)$ suivant
ENDLOOP	Yn=MR	Dernière instruction de la boucle. Mise à jour de Yn (sortie du filtre) qui se calcule au fur et à mesure des 4 passages dans la boucle.
	Sortie de Yn vers le CNA	Première instruction en hors boucle. Envoi de la valeur Yn vers le CNA
	Instruction suivante	...

LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES ADRESSES

PROGRAM SEQUENCER (suite)

Automatiquement, par le hardware, à l'exécution de l'instruction :

« DO ENDLOOP UNTIL CE » :

↳ l'adresse **ENDLOOP** est empilée dans la **LOOP STACK** ⑨,

↳ l'adresse **STARTLOOP** est empilée dans la **PC STACK** ⑧.

Quand la dernière instruction de la boucle est exécutée (plus précisément 2 instructions avant grâce au pipe) et grâce au **LOOP COMPARATOR** câblé ⑩, le **PROGRAM SEQUENCER** vérifie si la condition de sortie est vraie ou pas (ici par test du flag **CE**) :

- si **CE = 0** : dépilement de la **PC STACK** et récupération de l'adresse **STARTLOOP** placée sur le bus **PMA** (cas ④) et **exécution (ré-itération)** de la boucle **une fois de plus**,

Sens de déroulement des cycles machine :

Exécution	f-2	f-1	f	d
Décodage	f-1	f	d	d+1
Recherche	f	d	d+1	d+2

condition de sortie de boucle **fausse (CE=0)** (testée deux instructions avant la fin de boucle) recherche dans la **PC STACK** de l'adresse **STARTLOOP (d)** de début de boucle on exécute une fois de plus la boucle

CAS D'UNE RE-ITERATION DE LA BOUCLE (CE=0)

f = adresse de l'instruction de **fin** de boucle, c'est-à-dire **ENDLOOP**,

d = adresse de l'instruction de **début** de boucle, c'est-à-dire **STARTLOOP**.

Figure II.9.

LA FAMILLE 21xx

LES UNITES DE CALCUL SUR LES ADRESSES

PROGRAM SEQUENCER (suite)

- si **CE = 1** : passage à la **première instruction hors boucle**.

Sens de déroulement des cycles machine :

	→			
Exécution	f-2	f-1	f	f+1
Décodage	f-1	f	f+1	f+2
Recherche	f	f+1	f+2	f+3
	condition de sortie de boucle vraie (CE=1) (testée deux instructions avant la fin de boucle)	recherche de la première instruction en hors boucle (f+1)		on exécute l'instruction qui suit la boucle

CAS D'UNE TERMINAISON DE LA BOUCLE (CE=1)

Figure II.9.

Conclusion : aucune perte de cycle pour la gestion de la boucle !!!

Au final :

↪ utilisation de deux buffers circulaires (via les registres modulo **Li** des **DAG**) pour stocker les échantillons et les coefficients du filtre,

↪ calcul de la sortie **y(n)** du filtre grâce à une boucle de ce type,

↪ tout ceci entre les instants $t_n = n.T_e$ et $t_{n+1} = (n+1).T_e$ correspondant à deux prises d'échantillons successives (souvent sous interruption) de la valeur d'entrée **x(n)** du filtre.

Mais **contrainte à respecter** $\Rightarrow$ le temps de calcul de **y(n)**
(=durée d'exécution boucle fois nombre de passage dans la boucle)

doit être inférieur à $t_{n+1} - t_n = T_e = \frac{1}{F_e}$

Dans un DSP, l'architecture hardware a été conçue à la base pour minimiser ce temps de calcul, pour pouvoir diminuer T_e et donc travailler à F_e la **plus grande possible** (cf. théorème de **Shanon**).

LA FAMILLE 21xx

EN RESUMÉ :

Voici l'architecture interne de base **commune**
à tous les **DSP** de la famille 21xx.

ADSP-2100 Family Base Architecture

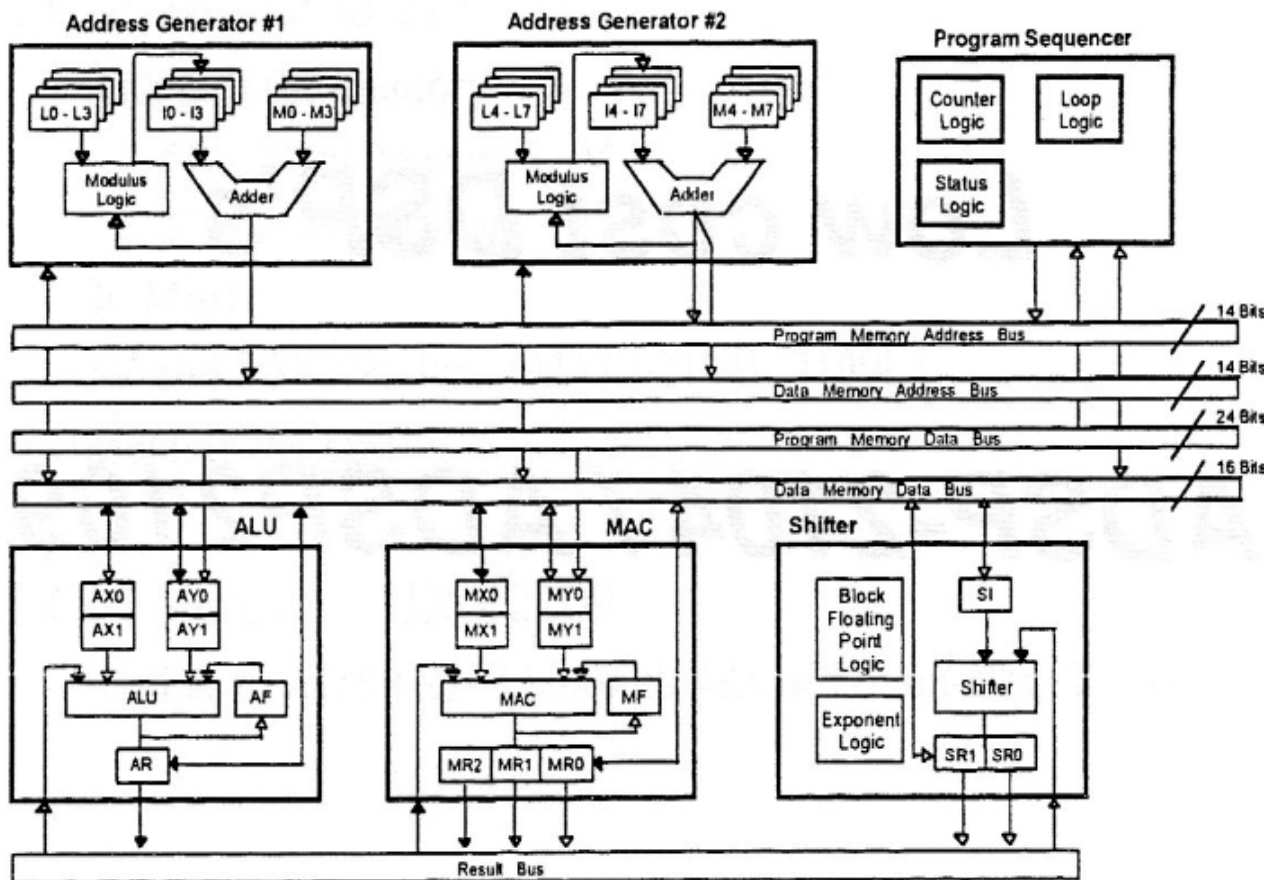


Figure II.10. (page 23)

LA FAMILLE 21xxx

Architecture du cœur du processeur **comparable** avec les 21xx.

- les 3 unités de calculs internes : **ALU**, **MAC** et **SHIFTER**,
- les 2 **DAG** : **DAG1** et **DAG2**,
- le **PROGRAM SEQUENCER**.

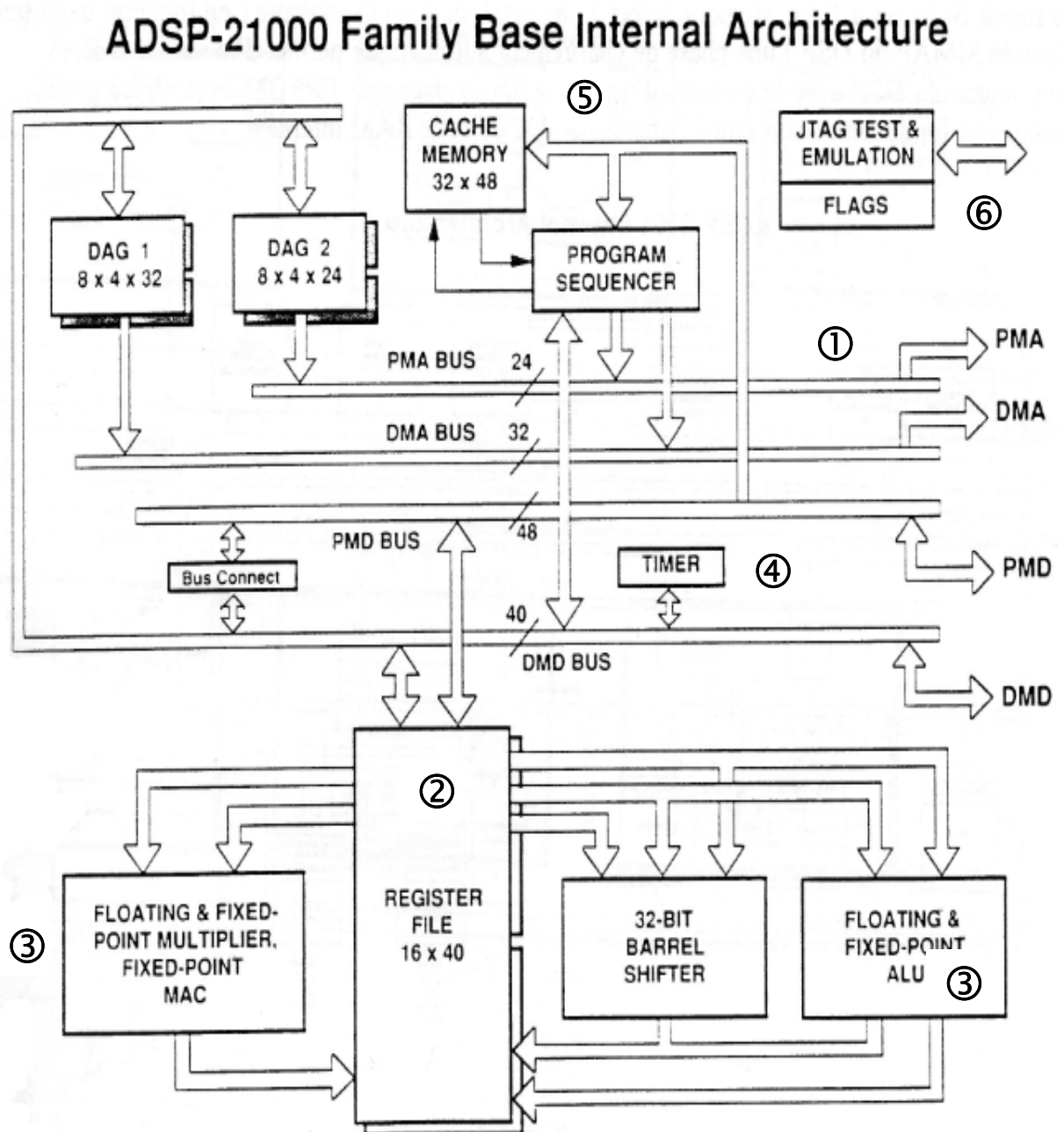


Figure III.1. (page 26)

LA FAMILLE 21xxx (suite)

Avec en plus les différences suivantes de ① à ⑥ pour améliorer les performances des 21xxx.

① **augmentation** de la taille des bus internes :

↳ pour **améliorer** la **vitesse de traitement** et les **tailles** des mémoires de programme **PM** et de données **DM**.

② registres de travail des différentes unités de calcul internes **banalisés** en un ensemble de 16 registres (**Register file 16X40 Bit**) :

↳ pour « **fluidifier** » les échanges de données entre elles et **augmenter** les possibilités de **stockage** de résultats temporaires ou intermédiaires.

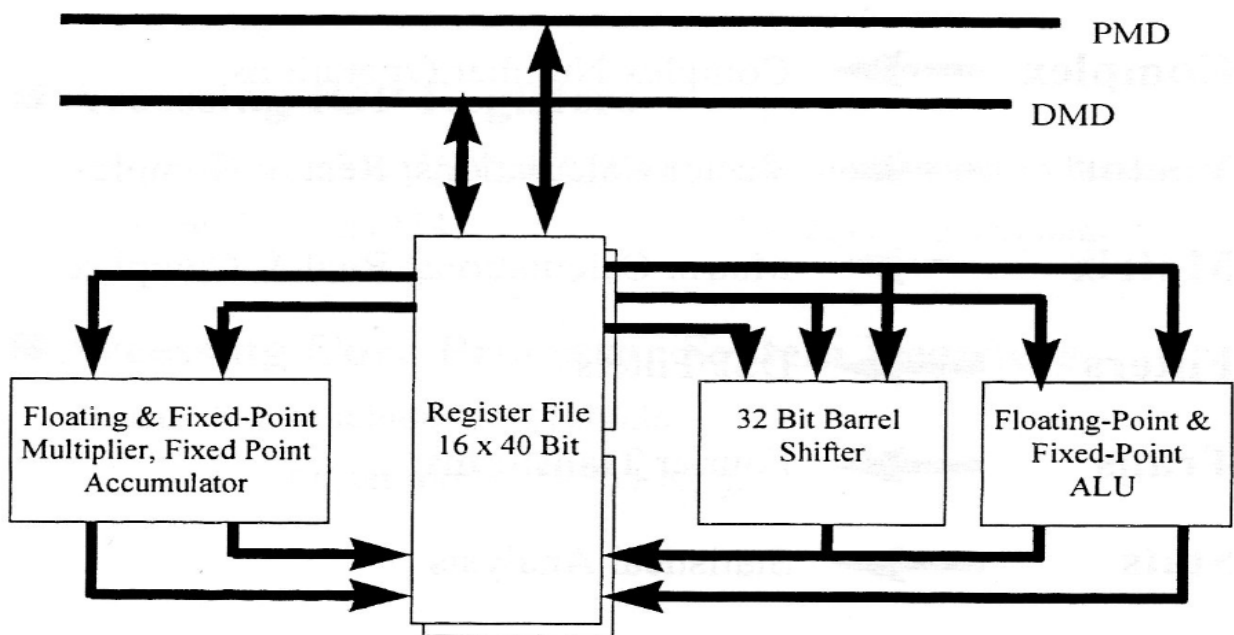


Figure III.2. (page 27)

③ la nouvelle famille **21xxx** fonctionne en **virgule flottante** :

↳ sur **32 bits (Single Precision : standard IEEE 754/854)**

↳ ou sur **40 bits (Extended Precision)**.

Remarque

Voir figures III.3. à III.9. pages 28 et 29 du polycopié complet de cours pour plus de précisions sur les formats supportés :

- **entiers : non signés** ou **signés**,
- et **fractionnaires : virgule fixe** et/ou **flottante**.

LA FAMILLE 21xxx (suite)

④ existence d'un **timer sur 32 bits** permettant de générer des interruptions périodiques de fréquence programmable :

↳ pour définir par exemple la période d'échantillonnage T_e et donc la **fréquence d'échantillonnage** F_e respectant le critère du **théorème de Shannon**.

⑤ présence d'un **cache instructions** (**INSTRUCTION CACHE**) pouvant contenir 32 instructions :

↳ pour **éviter** dans certains cas des **conflits d'accès** à la mémoire **PM** et augmenter la **rapidité de traitement**.

Contrairement à un cache **associatif** où seules sont mises en cache (et de façon transparente pour le programmeur), à un instant donné, les instructions qui sont **le plus fréquemment utilisées**, ici on a affaire à un cache **sélectif** : seules certaines instructions sont mises en cache (et toujours de façon transparente pour le programmeur).

Mais lesquelles ?

Pour y répondre, revenons encore à l'exemple vu sur la figure II.7. et représentant le « cœur » de la plupart des algorithmes en traitement numérique du signal à savoir effectuer un calcul du type :

$$y(n) = \sum_{j=0}^{N-1} a(j).x(n-j) \text{ (multiplication et accumulation par addition)}$$

Cet exemple correspondait, on l'a vu, à l'exécution simultanée de :

$$MR=MR+MX0*MY0, \quad MX0=PM(I0,M0), \quad MY0=DM(I1,M1)$$

LA FAMILLE 21xxx (suite)

Or, il faut toujours avoir en tête que pendant que le DSP exécute l'instruction (**q**), il décode aussi l'instruction (**q+1**) et il va rechercher en mémoire **PM** l'instruction (**q+2**), et tout ceci pendant le tick (**q**) d'horloge du quartz (principe du pipe).

Or l'exécution de l'instruction précédente demande aussi un accès (ici en lecture) pour des données situées dans la mémoire de code (pour charger **MX0** par **PM(I0,M0)**) et donc dans la même mémoire que celle demandée par la phase de recherche de l'instruction (**q+2**).

on dit qu'il y a **conflit** sur la mémoire **PM**
(car **double accès simultané** à la **même** mémoire **PM**).

Sans le cache, on perdrait un cycle !!!

Et comme, en général, ce type d'instruction est, comme on l'a vu, située dans une boucle du type **DO ENDLOOP UNTIL** **CONDITION**, ce conflit se produira autant de fois que cette boucle sera exécutée, ce qui n'est pas acceptable.

Le cache sert à réduire (en moyenne) le délai de traitement de ce type de conflit :

la première fois que ce conflit se produit (à la première exécution de la boucle) le DSP attend un cycle pour rechercher l'instruction (**q+2**) car la priorité est donnée à l'exécution de cette instruction (**q**).

Et il place ensuite l'instruction recherchée (**q+2**) dans le cache pour éviter que ce conflit ne se reproduise à nouveau, à la prochaine itération de la boucle. C'est en ce sens que le cache est **sélectif**.

On voit finalement que tout se passe comme si on rajoutait un troisième bloc de mémoire (en plus de **PM** et de **DM**) auxiliaire à la mémoire de code.

Le DSP 21061 utilisé en TP (suite)

① Le cœur du processeur (bloc **CORE PROCESSOR**) : conforme à celui, décrit plus haut, de la famille 21xxx.

② La mémoire interne (bloc **DUAL PORTED SRAM**) : organisée en **deux blocs indépendants** nommés

BLOCK0 et BLOCK1

dont les tailles dépendent du type de DSP.

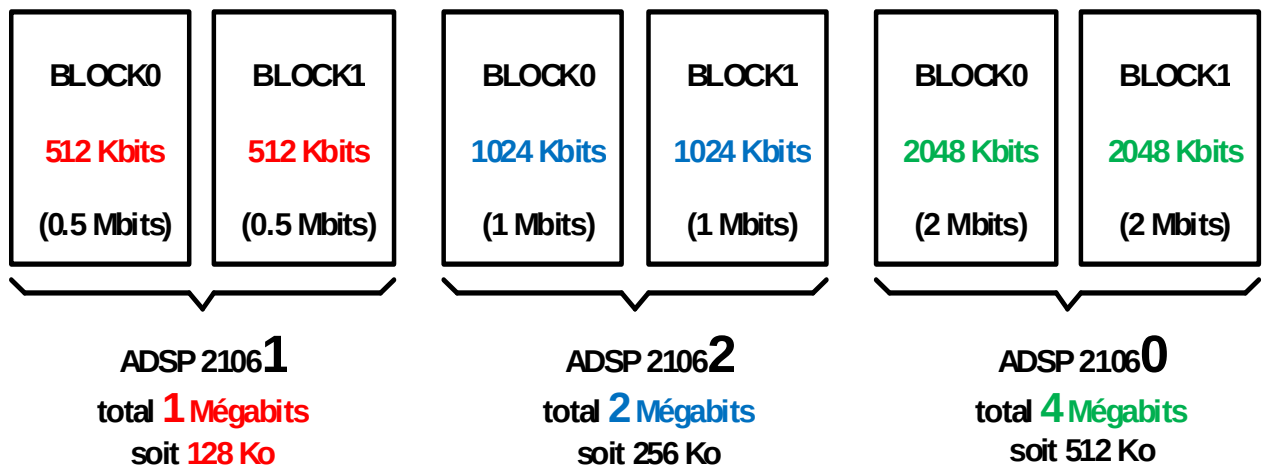


Figure IV.2. (page 33)

Pour optimiser la gestion de ces deux blocs et permettre l'exécution d'instructions complexes utilisant les trois unités de calcul et la recherche de données en mémoire :

↳ le **BLOCK0** est piloté par le **DAG2** via le **PMA Bus** pour servir de **PM** (pour les données de la mémoire de programme).

Rappelons que c'est le séquenceur de programme qui permet d'accéder aux instructions situées aussi dans la mémoire de programme.

↳ le **BLOCK1** est piloté par le **DAG1** via le **DMA Bus** pour servir de **DM** (pour les données de la mémoire de données).

Le DSP 21061 utilisé en TP (suite)

Cartographie mémoire

Voici le **mapping mémoire complet** pour les ADSP 2106x.

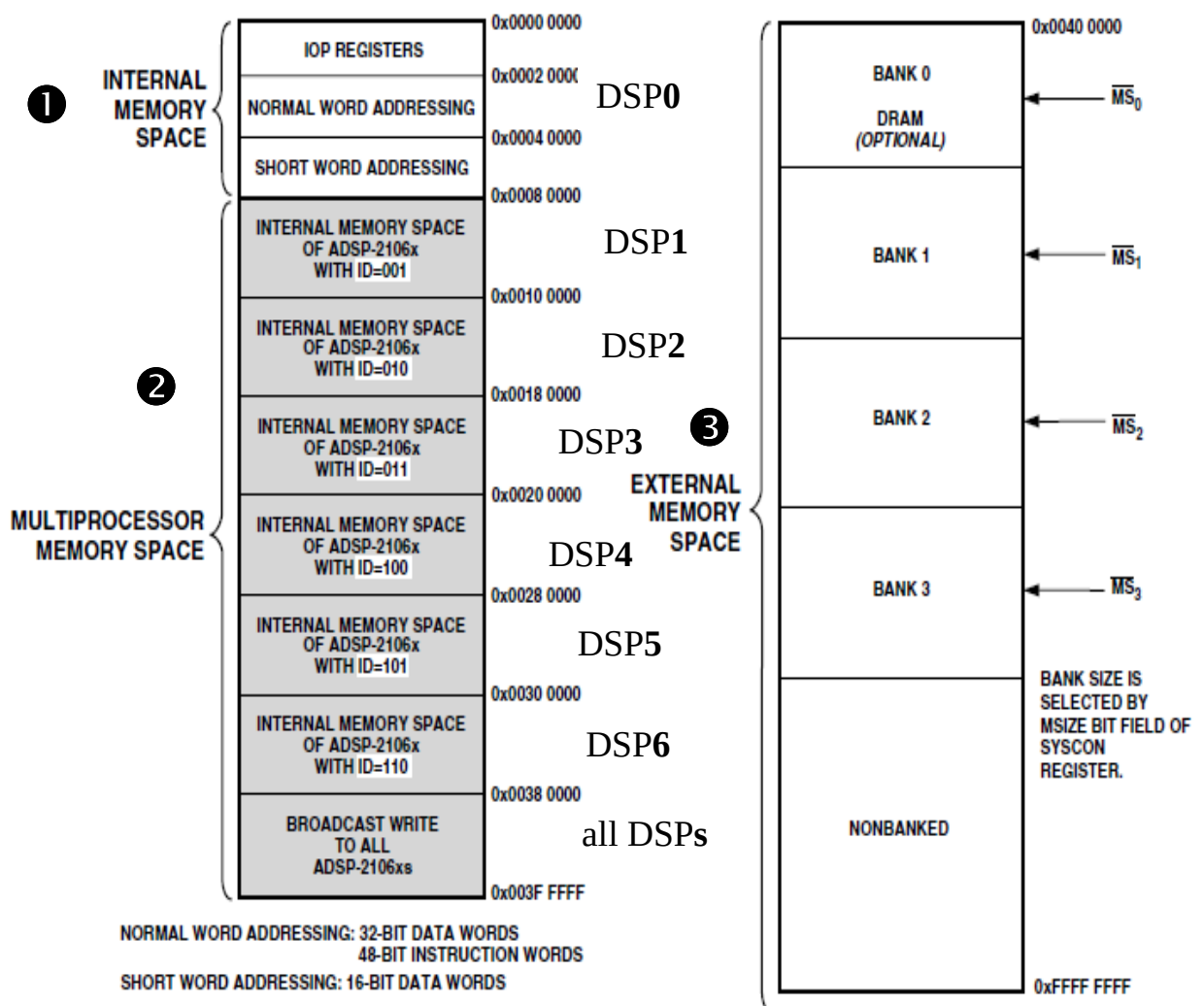


Figure IV.3. (page 34)

Le DSP 21061 utilisé en TP (suite)

Cartographie mémoire : trois zones

Découpage du mapping en **trois zones principales** :

❶ la mémoire **interne** du DSP0 :

↪ entre les adresses **0x0000 0000** et **0x0007 FFFF**

❷ les mémoires **internes** des autres DSP, dans le cas des systèmes **multiprocesseurs** :

↪ entre les adresses **0x0008 0000** et **0x003F FFFF**

Pour des applications plus complexes, on peut associer dans le même système jusqu'à 7 DSP (DSP0 à DSP6) qui peuvent travailler en parallèle dans une architecture de type maître/esclave.

A tout instant, un des DSP est le **maître** du système, les autres étant ses **esclaves**.

Cet espace, constitué des mémoires et ressources internes de tous les DSP du système appartient au mapping mémoire de chaque DSP et est donc accessible par chaque DSP.

❸ la mémoire **externe** que l'on peut rajouter, pour augmenter la capacité mémoire totale du système :

↪ entre les adresses **0x0040 0000** et **0xFFFF FFFF**

Ce qui porte au total à **4 Giga octets** ($= 2^{32}$) la capacité d'adressage **maximale** d'une telle architecture.

Le DSP 21061 utilisé en TP (suite)

Cartographie mémoire : principe (suite)

Quelques remarques

- la zone comprise entre les adresses **0x0038 0000** et **0x003F FFFF** (notée **all DSPs** sur la figure IV.3.) est une zone de « **broadcast** » : en écrivant dans cette zone, le DSP **maître** peut communiquer des informations à tous les autres DSP **esclaves** en même temps au lieu d'avoir à écrire ces informations dans chacun des DSP à tour de rôle, ce qui permet de gagner du temps pour des informations communes concernant **tous** les esclaves.

- les **registres internes d'Entrée/Sortie (IO REGISTER)** : ils servent pour la programmation de certains périphériques du DSP comme le timer, les liaisons séries, les interruptions... et se trouvent dans le mapping entre les adresses : **0x0000 0000** et **0x0000 00FF**. (ils jouent un peu le même rôle que les registres IO du PIC qui permettaient aussi de piloter les interfaces de ce microcontrôleur)

D'autres remarques à caractère « plus électronique » existent dans mon polycopié de cours complet mais risquent de dépasser les aspects purement « traitement du signal »....

Et au final, tous ces aspects purement électroniques sont pris en compte par le **LINKER** du compilateur C grâce au fichier **d'architecture matérielle** permettant de détailler la cartographie mémoire complète du système.

Le DSP 21061 utilisé en TP (suite)

③ Une interface avec l'extérieur (bloc **EXTERNAL PORT**) constituée par les trois éléments suivants :

- les **bus externes** constitués :
 - d'un bus d'**adresses sur 32 bits** (**ADDR BUS MUX**),
 - d'un bus de **données sur 48 bits** (**DATA BUS MUX**),
 - ↳ pour piloter la mémoire externe.
- une **interface multi-processeurs** (bloc **MULTIPROCESSOR INTERFACE**).
 - ↳ pour assurer le contrôle des échanges entre les différents DSP.
- une **interface avec un processeur hôte** (bloc **HOST PORT**)
 - ↳ pour les phases de développement : compilation, linkage, téléchargement, mise au point...
En TP, nous utiliserons une autre technique via une liaison série.

Le DSP 21061 utilisé en TP (suite)

④ Un **processeur d'entrées / sorties** (bloc **I/O PROCESSOR**), indépendant du Core Processor, et qui comprend :

- **les registres de contrôle pour les IO.**

↳ mappés entre les adresses **0x0000 0000** et **0x0000 00FF** pour piloter les périphériques du DSP.

- **deux ports série SPORT0 et SPORT1** (bloc **serial PORTS**)

Il s'agit de 2 liaisons **séries synchrones** et **indépendantes**.

Ces liaisons ont la partie réception et la partie transmission indépendantes, autorisant le fonctionnement en **vrai full duplex**.

Elles peuvent aussi fonctionner sous interruption (voir plus loin comment).

Une de ces liaisons (**SPORT0**) permet d'assurer la liaison avec un **Codec** (ici un **Sound Port Stéréo Codec** de référence **AD 1847** orienté vers le **traitement du son**) que nous détaillerons plus loin.

Le protocole de liaison entre le **DSP** et ce **Codec** est le **TDM** (Time Division Multiplexed) dont nous parlerons également dans la suite de ce polycopié.

- **un contrôleur de DMA** (bloc **DMA CONTROLLER**)

La notion de **DMA** (**D**irect **M**emory **A**ccess) est liée à celle de **transfert de données** dans un système informatique.

Comme elle sera abondamment utilisée en TP et qu'elle ne vous est pas forcément très familière, elle mérite qu'on s'y arrête un peu.

Le DSP 21061 utilisé en TP (suite)

Vers la technique de DMA

Le déplacement de données entre les mémoires internes et l'extérieur ou même entre les mémoires internes elles-mêmes, est une tâche très fréquente dans le cas du traitement du signal :

↪ applications audio :

transfert de fichiers de sons à traiter ou traités :
voix, musique...

↪ applications vidéo :

transfert de fichiers d'images à traiter ou traitées :
dessins, photos, séquences animées...

L'idée de base est de **diminuer** le temps pris par ces transferts (temps « perdu » par le DSP sur le temps nécessaire au traitement numérique proprement dit).

Pour réaliser ces mouvements de données, il existe différentes techniques, **plus ou moins faciles** à implanter mais aussi **plus ou moins efficaces** en temps.

Nous allons en présenter trois :

- de la **plus simple** (mais aussi **la moins efficace** en temps),
 - à la **plus compliquée** (mais aussi **la plus performante**),
- ce qui nous permettra d'aboutir, au final, au principe de la **DMA**.

❶ technique par **scrutation** (ou **pooling**),

❷ technique par **interruption** (par **IT**)

❸ technique par **DMA**,

sur la base **d'un transfert de données** d'une zone de **mémoire interne** via la liaison série **SPORT0** en **transmission** vers un CNA. (données qui ont été traitées par le **DSP** et qui sont envoyées en sortie vers le **CNA**, comme on le fera d'ailleurs en TP).

Le DSP 21061 utilisé en TP (suite)

Vers la technique de DMA (suite)

Le principe général d'une transmission par liaison série consiste à écrire les données à transmettre (les unes après les autres) dans le **registre de transmission TX0** de la liaison **SPORT0**, ce registre se charge ensuite de les **sérialiser** pour les envoyer sur la liaison série **bit à bit** au rythme de la **vitesse choisie** (le registre de transmission est donc un registre à décalage à **entrée parallèle** et à **sortie série**).

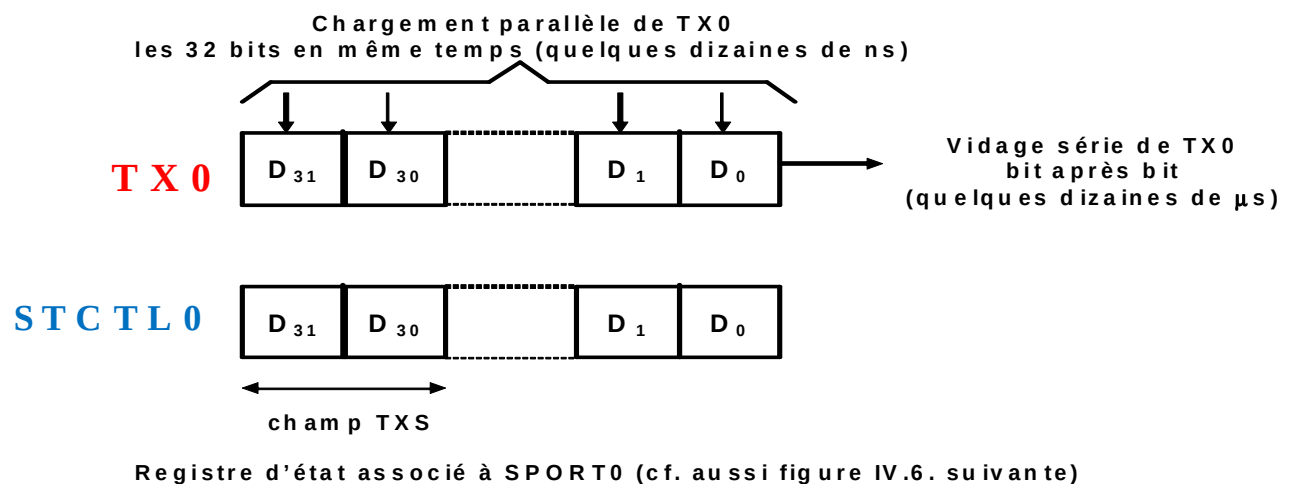


Figure IV.5. (page 40)

Mais attention, comme la vitesse de la liaison série est très faible (de l'ordre de la dizaine de μs par élément de 32 bits à émettre) par rapport au temps de chargement (32 bits en même temps) du registre de transmission (de l'ordre de la dizaine de ns) soit un facteur 1000 !!!, il faut toujours attendre que l'élément en cours de transfert soit entièrement transmis avant de placer l'élément suivant dans le registre de transmission, car sinon on écraserait les éléments les uns sur les autres et le message émis serait entièrement corrompu.

Il est important d'avoir bien compris cela et de l'avoir toujours en tête avant de regarder les trois exemples suivants !!!

Le DSP 21061 utilisé en TP (suite)

Vers la technique de DMA (suite)

Question qui se pose alors...

Comment savoir lorsque ce registre de transmission **TX0** est :

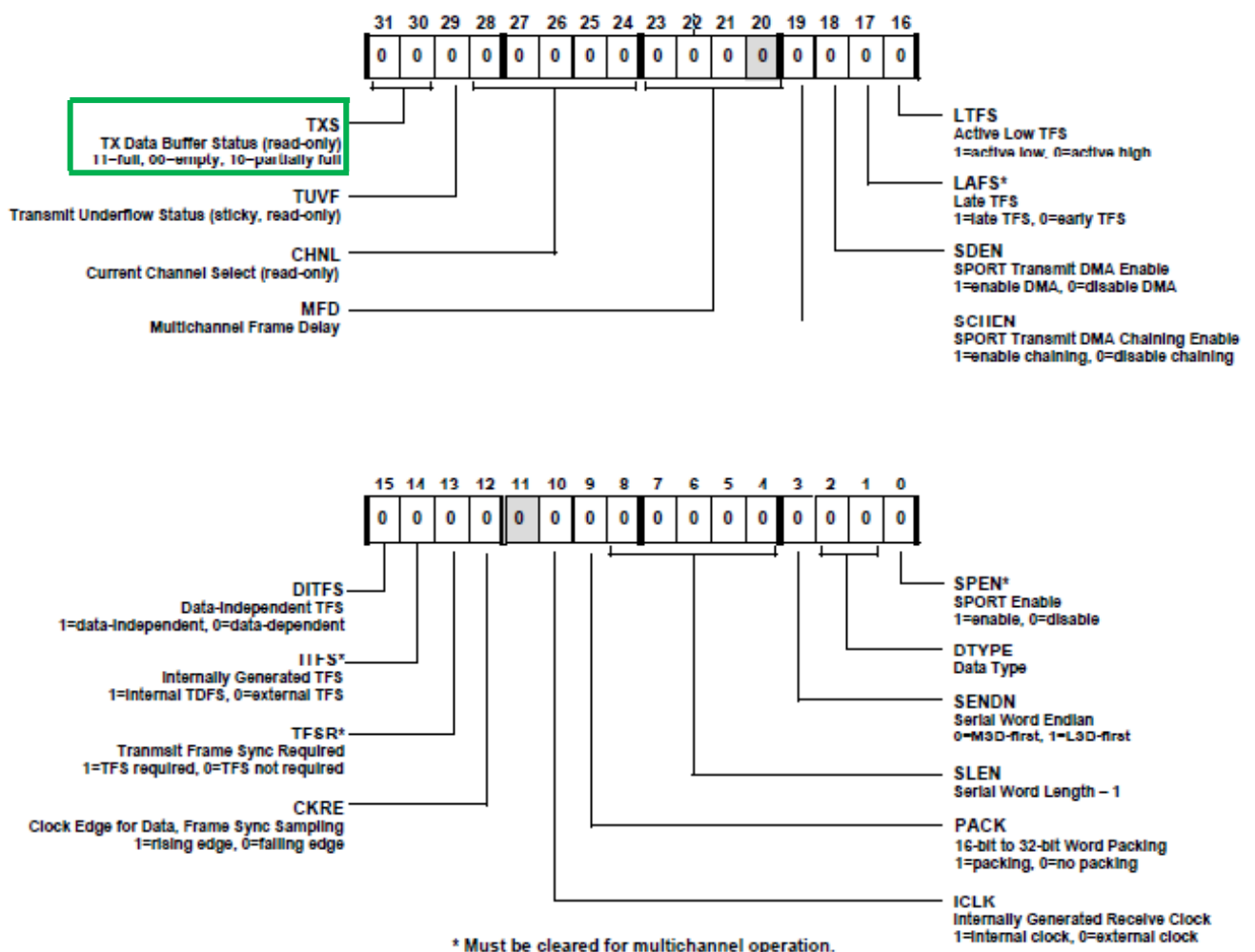
↳ **vide**, donc **prêt à recevoir** une nouvelle donnée à transmettre,

↳ ou **plein**, donc **encore en train d'émettre** la donnée précédente ?

Pour cela, il existe un registre **STCTL0**, associé à **SPORT0**.

Ce registre fournit des informations sur **l'état** de la liaison série.

(validation, format, vitesse, mode de fonctionnement...)



STCTL0, STCTL1 Transmit Control Registers

Figure IV.6. (page 41)

Le DSP 21061 utilisé en TP (suite)

Vers la technique de DMA (suite)

Et en particulier, les bits **31** et **30** (champ **TXS**) de ce registre **STCTL0** fournissent à tout instant, l'état du registre de transmission **TX0**, suivant la logique :

- si le champ **TXS** de (**STCTL0**) = **1 1** :
 - ⇒ le registre de transmission **TX0** est plein,
- si le champ **TXS** (de **STCTL0**) = **0 0** :
 - ⇒ le registre de transmission **TX0** est vide.

Remarque

Les trois méthodes que l'on va présenter maintenant pour établir une transmission de données du DSP vers l'extérieur vont mettre en œuvre des principes qui seraient tout à fait comparables dans le cas d'une réception de données série venant de l'extérieur vers le DSP.

Le DSP 21061 utilisé en TP (suite)

La technique de transmission par pooling (ou scrutation)

Le principe de base est le suivant :

**Tant que le registre de transmission TX0 est plein,
le Processor Core se met en attente qu'il soit vide
(rappelez-vous du facteur 1000 !!!)
avant de le remplir avec le prochain élément à transmettre,
et ceci jusqu'au dernier :
la conséquence est qu'il ne peut rien faire en parallèle,
d'où les limites de cette méthode...**

En pratique, c'est la technique :

- ↳ la plus **simple** à comprendre,
- ↳ la plus **facile** à programmer,
- ↳ la plus **rapide** à mettre au point.

Mais à contrario, c'est aussi celle qui est :

- ↳ la **moins performante**,

car c'est elle qui consomme le plus de temps pour le transfert et donc qui laisse le moins de temps pour des éventuels traitements numériques.

En gros, tout le temps est pris par la transmission car le DSP ne peut rien faire en parallèle :

il passe son temps à attendre que le transfert complet soit terminé.

Résumé du cours DSP : chapitre IV.

Exemple de structure de programme pour un transfert de données sur SPORT0 par pooling (scrutation)

```
/* fichier de définition des registres internes et de certains de leurs champs de bits */
#include <def21060.h>
/* les registres et champs de bits des SPORT0 et SPORT1 */
#include <sport.h>
/* taille du fichier à transmettre : ici on a pris 4 mais le fichier pourrait être plus long */
#define    N    4

/* le tableau ou buffer des données (entiers sur 32 bits) à transmettre */
/* ce tableau source sera rangé en mémoire de données par dm */
long dm    source[N]={ 0x11111111, 0x22222222, 0x33333333, 0x44444444};
int      k = 0;

/* Le programme de transfert suivi d'un éventuel traitement */
void main ( void )
{
/* initialisation de la liaison série SPORT0 (validation, vitesse, format...)*
    TDIV0 = 0x00270007 ; /* initialisation du registre TDIV0 de SPORT0 */
                        /* pour fixer entre autre la vitesse de la liaison série */
                        /* se reporter à son contenu exact pour plus d'infos */

    STCTL0 = 0x000064F1 ; /* initialisation du registre STCTL0 de SPORT0 */
                        /* STCTL0 contient des informations de contrôle*/
                        /* comme le format des données, la validation de */
                        /* SPORT0, l'utilisation ou pas de la DMA ... */
                        /* se reporter à son contenu exact pour plus d'infos */

/* début du transfert */
    for ( k=0 ; k<N ; k++ )
    {
        TX0 = source[k];      /*écriture de l'élément courant du buffer dans le */
                              /* registre de transmission TX0 */

/* utilisation du masque TXS (bits 31 et 30) qui vaut 0b11000....000 = 0xC000 0000 */
        while ( STCTL0 & TXS ); /* tant que TX0 est plein, on attend */
                                /* ici TX0 est vide, ce qui signifie que */
                                /* l'élément courant a été totalement transmis*/
                                /* alors on peut envoyer l'élément suivant */

    }

/* fin du transfert */
/* à partir de là toutes les données ont été transmises sur SPORT0 */

    while ( 1 )              / boucle sans fin de traitement */
        traitement();        /* on ne peut faire du traitement que quand toutes */
                              /* les données ont été transmises */
}
```

Le DSP 21061 utilisé en TP (suite)

La technique de transmission sous interruption

Le principe ici est **totale**ment différent.

Au lieu d'attendre que TX0 soit vide avant d'envoyer la donnée suivante, c'est la liaison SPORT0 qui va générer une interruption (IT) de transmission, signalant que TX0 est vide (et donc que la donnée précédente a été entièrement transmise). Cette IT entraîne automatiquement l'exécution d'une routine d'IT venant charger dans TX0 la donnée suivante à émettre.

Ainsi, par rapport à la méthode précédente, le **Processor Core** du **DSP** peut effectuer autre chose **en parallèle**, puisqu'il ne passe pas son temps à attendre que la donnée courante soit transmise : il n'est dérangé que quand la donnée courante a été **entièrement** transmise.

Même structure du programme mais avec un bloc supplémentaire.

Ce bloc n'existait pas dans la technique par scrutation :
il correspond à la routine de traitement de l'interruption.

Ce type de programme sous IT est **plus difficile** à concevoir et à écrire qu'avec la technique du pooling, car il faut penser à toutes les choses suivantes, sans en oublier aucune :

- **initialiser** le vecteur d'interruption (fonction **interrupt** du C),
- **valider** le fonctionnement sous IT pour qu'elle soit générée par **SPORT0** quand TX0 est vide (donnée entièrement transmise),
- **démasquer** toutes les interruptions du DSP,
- **écrire la routine d'IT** : c'est quand même le minimum !!!
- **acquitter** l'interruption dans cette routine d'IT pour éviter des appels intempestifs liés à la même cause (ici l'écriture dans le registre **TX0** permet de faire **automatiquement** cet acquittement)

Ce type de programme sera aussi **plus délicat** à mettre au point.

Résumé du cours DSP : chapitre IV.

Exemple de structure de programme pour un transfert de données sur SPORT0 sous interruption

```
/* zone des #include et des #define : idem cas précédent !!! */

/* zone de déclaration des variables globales du programme : idem cas précédent !!! */

/* Le programme de transfert avec un éventuel traitement en parallèle */
void main ( void )
{
/* initialisation de la liaison série SPORT0 (validation, vitesse, format...)*
    TDIV0 = 0x00270007 ;      /* idem cas précédent par scrutation*/
    STCTL0 = 0x000064F1 ;    /* idem cas précédent par scrutation */
/* initialisation pour le fonctionnement sous interruption TX0 vide*/
    interrupt (SIG_SPT0I, it_transmit); /* installation de l'IT en émission */
                                     /* it_transmit est le nom de la routine */
                                     /* d'IT qui sera exécutée chaque fois */
                                     /* que le registre TX0 sera vide */

    SPT0I = SPT0I | IMASK;    /* mise à 1 du bit IMASK de SPT0I pour */
                             /* valider les IT sur registre de TX0 vide */

    IRPTEN=IRPTEN | MODE1; /* démasquage de l'interruption */

/* début du transfert*/
    TX0 = source[k];         /*écriture du premier élément du buffer dans TX0 */

/* appel de la fonction de traitement du signal */
    while ( 1 )              /* boucle sans fin de traitement : on peut faire du */
        traitement();        /* traitement en même temps que le transfert */
}

/* routine d'IT appelée chaque fois que le registre de transmission TX0 est vide */
/* c'est-à-dire entre l'envoi de chaque élément du buffer source */
void it_transmit ( int sig_num )
{
    if ( k==N-1 )            /* dernier élément : buffer entièrement transmis ?*/
    {
        SPT0I=SPT0I & ~IMASK; /* mise à 0 du bit IMASK de SPT0I */
                             /* pour invalider les IT sur TX0 vide*/

        IRPTEN=IRPTEN & ~MODE1; /* masquage de l'interruption */
    }                         /* et fin du transfert */

    k++ ;                    /* passage à l'élément suivant */

    TX0 = source[k];         /* et l'émettre */
}
```

Le DSP 21061 utilisé en TP (suite)

La technique de transmission par DMA

Avec la technique sous IT, on a vu que le DSP pouvait effectuer des traitements **en même temps** que le transfert des données puisqu'il n'était dérangé par une IT qu'entre chaque donnée à transférer.

Dans notre exemple, il y aura 4 interruptions, donc c'est acceptable. Mais imaginons, et c'est souvent le cas en pratique, des fichiers source plus volumineux à transférer (plusieurs milliers d'éléments à transférer, voire plus), le temps perdu par ces milliers d'interruptions peut commencer à devenir prohibitif.

L'idée, pour améliorer encore la technique sous IT, consiste à dire :

L'idéal serait qu'il n'y ait qu'une seule interruption à la fin du transfert, et cela quelque soit la taille du fichier à transférer !!!

Cela revient, pour le DSP, à complètement « sous-traiter » la tâche du transfert.

C'est là qu'intervient le contrôleur de **DMA :DMA CONTROLLER**

En fait, c'est un bloc spécialisé capable de prendre en compte **complètement de « A à Z »** et **en totale autonomie** une opération de transfert qu'on appelle alors une **DMA (Direct Memory Access)**.

Bien sûr, cette prise en charge du transfert nécessitera avant tout une **phase d'initialisation** (ou **phase de paramétrage**) pour fournir au **DMA CONTROLLER** tous les éléments nécessaires pour que ce transfert ait lieu en autonomie et sous son seul contrôle, sans « déranger » le DSP (sauf une seule fois, pour signaler la fin du transfert).

Ainsi une **DMA** s'avère être une méthode **très rapide** et **quasiment transparente** pour le programmeur pour transférer des blocs de données importants entre la mémoire interne et l'extérieur : mémoire externe, liaisons série, périphériques, processeur hôte ...

Le DSP 21061 utilisé en TP (suite)

La technique de transmission par DMA (suite)

Nouvelle question qui se pose alors...

De quels types d'informations le contrôleur de DMA a-t-il besoin pour prendre en charge entièrement un tel transfert ?

Il y en a **au moins cinq** que l'on va détailler maintenant.

① Choix d'un canal de DMA :

↪ en quelque sorte, le chemin par où les données seront transférées.

Sur le **DSP 21061**, il existe 6 canaux de DMA **indépendants** :

Canal 0	Liaison série SPORT0 en réception
Canal 1	Liaison série SPORT1 en réception
Canal 2	Liaison série SPORT0 en transmission
Canal 3	Liaison série SPORT1 en transmission
Canal 4	DMA externe : mémoire externe, processeur hôte, multiprocessing
Canal 5	DMA externe : mémoire externe, processeur hôte, multiprocessing

Dans l'exemple, on utilise le **canal 2** (transmission sur SPORT0).

② Une adresse de début (de base) du buffer source :

↪ c'est-à-dire où se trouve en mémoire le buffer à transférer.

Cette adresse sera initialisée pour chaque canal *i* dans un registre nommé :

II*i* (Internal Index **i**, pour *i*=0 à 5) du **contrôleur de DMA**.

Le DSP 21061 utilisé en TP (suite)

La technique de transmission par DMA (suite)

③ Une valeur d'incrément ou de décrétement :

↪ pour calculer à partir de l'adresse précédente du début du buffer, l'adresse de la donnée suivante à transférer sous DMA.
(c'est en fait à la taille des éléments du bloc à transférer).

Cette valeur d'incrément ou de décrétement sera mise pour chaque canal i dans un registre nommé :

IMi (Internal **M**odifier **i**, pour $i=0$ à 5) du **contrôleur de DMA**.

④ La taille du bloc à transférer :

↪ c'est en fait un compteur qui se décrémente automatiquement de un pour chaque donnée transférée et qui indiquera la fin du transfert DMA quand il atteindra une valeur nulle.

Cette taille sera mise pour chaque canal i dans un registre nommé :

Ci (**C**ount **i**, pour $i=0$ à 5) du **contrôleur de DMA**.

⑤ Éventuellement la direction du transfert :

↪ pour le sens des échanges (vers ou venant de l'extérieur du DSP).
(implicite pour les canaux 0 à 3, nécessaire pour les canaux 4 et 5).

La donnée de ces 5 éléments permettra de paramétrer correctement le transfert sous DMA.

Au final, cette méthode de transfert sera en pratique **très largement utilisée** car **très rapide** et **très efficace**.

Résumé du cours DSP : chapitre IV.

Exemple de structure de programme pour un transfert de données sur SPORT0 sous DMA

```
/* zone des #include et des #define : idem cas précédent !!! */

/* zone de déclaration des variables globales du programme : idem cas précédent !!! */

/* Le programme de transfert avec un éventuel traitement en parallèle */
void main ( void )
{
/* initialisation de la liaison série SPORT0 (validation, vitesse, format...)*
    TDIV0 = 0x00270007 ;      /* idem cas précédent par scrutation*/
    STCTL0 = 0x000465F1 ;    /* ici on autorise la DMA sur transmission */
                             /* la routine d'IT devient IT sous DMA */
                             /* se reporter à son contenu exact pour infos*/
/* initialisation pour le fonctionnement sous interruption DMA */
    interrupt ( SIG_SPT0I,it_dma ); /* installation de l'IT en émission */
                                    /* it_dma est le nom de la routine d'IT*/
                                    /* cette IT sera exécutée une seule*/
                                    /* fois, à la fin du transfert du buffer */

    SPT0I = SPT0I | IMASK;    /* mise à 1 du bit IMASK de SPT0I pour */
                             /* valider les IT sur registre TX0 vide */

    IRPTEN=IRPTEN | MODE1; /* démasquage de l'interruption */
/* début du paramétrage de la DMA canal 2 */
    II2 = source;            /* initialisation de II2 avec l'adresse du buffer source */
    IM2 = 1;                 /* valeur de l'incrément dans IM2 : 1 */
    C2 = N;                  /* initialisation de C2 avec longueur du buffer source */
/* fin du paramétrage de la DMA canal 2 */

/* à partir de là, le transfert se fait automatiquement sous contrôle total du contrôleur
de DMA : le DSP ne sera dérangé par l'IT it_dma qu'une fois le transfert terminé */

    while ( 1 )              /* boucle sans fin de traitement */
        traitement();        /* on peut faire du traitement pendant le transfert */
}

/* routine d'IT appelée lorsque le buffer source a été entièrement transmis */
void it_dma ( int sig_num )
{
    . /* on peut ici par exemple initialiser une autre DMA : il faut pour cela : */
    . /* - effacer le bit d'autorisation de la DMA dans STCTL0 */
    . /* - paramétrer une nouvelle DMA ( registres II2, IM2 et C2 */
    . /* - remettre à 1 le bit d'autorisation de la DMA dans STCTL0 */
}
```

Le DSP 21061 utilisé en TP (suite)

La technique de transmission par DMA : **améliorations**

Mais la vitesse et l'efficacité peuvent encore être poussés plus loin.

Comment ?

Si on y regarde bien, le procédé de transfert de buffer sous DMA que l'on vient de présenter, **nécessite tout de même une interruption entre chaque transfert complet de ce buffer.**

On peut imaginer des applications où le buffer représente par exemple une image et où l'on souhaite transférer des séquences vidéo (suite d'images). Pour éviter d'avoir une IT entre chaque image, il y a un moyen encore plus rapide : **le chaînage de DMA.**

Qu'est-ce que c'est ?

Cette technique permet « **d'enchâîner** » **automatiquement** (sous contrôle du DMA CONTROLLER) plusieurs transferts sous DMA les uns à la suite des autres.

Pour cela, on définit en mémoire une « liste chaînée » dont chaque « maillon » est une structure un peu particulière appelée aussi un **TCB** (comme **T**ransfert **C**ontrol **B**lock).

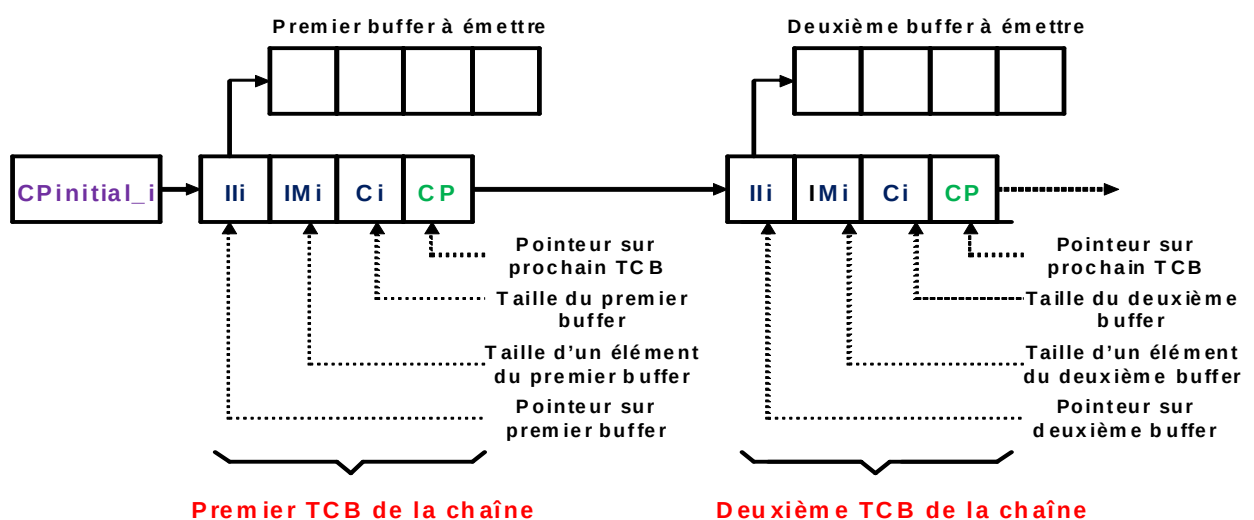


Figure IV.7. (page 48)

Le DSP 21061 utilisé en TP (suite)

La technique de transmission par DMA : chaîne de TCB

Chaque **TCB** de cette chaîne contient en fait les paramètres d'initialisation des différents transferts DMA (**IIi**, **IMi** et **Ci**) auxquels a été rajouté un **CP** (Chain Pointer) qui « pointe » sur le **TCB** suivant dans la chaîne.

Grâce à cette chaîne, les différents transferts DMA vont s'enchaîner **automatiquement** les uns à la suite des autres. Inutile pour le DSP de perdre du temps pour paramétrer une nouvelle DMA à la suite de la précédente : le contrôleur de DMA, grâce à cette chaîne de **TCB**, aura toutes les informations pour initialiser la DMA suivante.

Un registre spécial pour chaque canal i de DMA (en plus des registres **IIi**, **IMi** et **Ci** déjà présentés) appelé le registre **CPinitial_i** pointe sur le premier **TCB** de la chaîne pour **initialiser** le processus.

En résumé :

En autorisant le chaînage de DMA et **en initialisant CPinitial_i** :

- le contrôleur de DMA vient automatiquement paramétrer la première DMA en chargeant les registres du canal i avec le contenu du premier **TCB**, et le premier transfert peut commencer,
- lorsque ce premier transfert est fini (quand **Ci = 0**), au lieu de générer une IT, le canal « s'auto-initialise » en pointant sur le **TCB** suivant de la chaîne et en chargeant cette fois-ci les registres du canal i avec le contenu de ce deuxième **TCB**, et le deuxième transfert peut s'enchaîner...
- ... jusqu'au dernier **TCB** de la chaîne (que l'on peut tester par la dernière valeur du registre CP par exemple).

On évite ainsi les interruptions entre chaque transfert DMA...
ce qui était bien l'objectif recherché !!!

Le DSP 21061 utilisé en TP (suite)

La technique de transmission par DMA : chaîne de TCB

Quelle méthode sera utilisée pendant les manipulations en TP ?

Nous ferons de l'acquisition de signaux (grâce à un CAN), pour obtenir des échantillons que le DSP traitera au fur et à mesure de leur prélèvement avant de les restituer sur le CNA en sortie.

Dans ce cas particulier, les échantillons, une fois qu'ils ont été traités et restitués, n'ont pas à être conservés dans un buffer différent propre à chaque échantillon.

On pourra donc utiliser le **même buffer** à chaque transfert DMA.

Cette situation particulière permet de faire **du chaînage de TCB** « **auto-référencé** », c'est-à-dire que chaque nouvel échange se fera avec les mêmes paramètres que l'échange précédent.

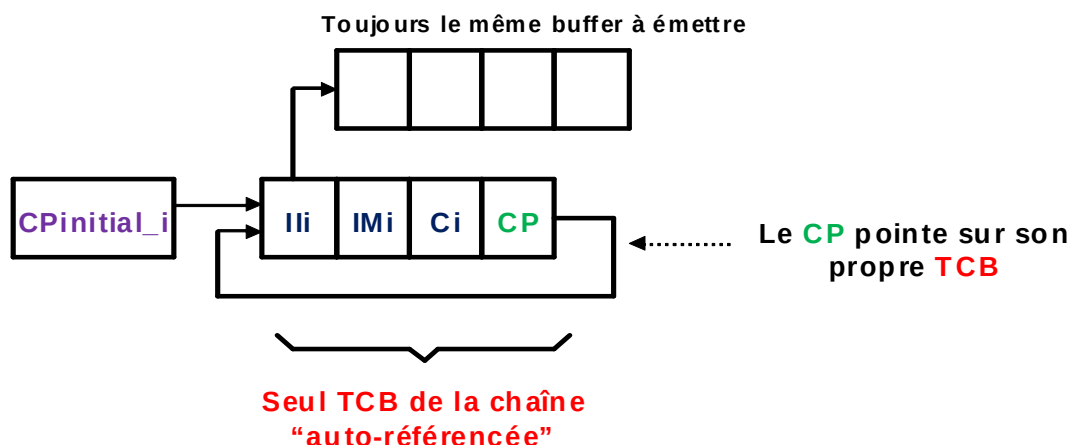


Figure IV.8. (page 49)

Le DSP 21061 utilisé en TP (suite)

La technique de transmission par DMA : en langage C

Un exemple de programme en C utilisant cette technique et fourni plus loin dans ce cours permet de gérer deux buffers de données :

↪ un premier buffer en **réception** (**rx_buf**) pour les échantillons fournis par le **CAN**, et déclaré comme un tableau de 3 entiers :

```
int rx_buf[3] ; /* buffer échantillons en entrée */
```

Ce buffer contient **$x(n)$** correspondant à la valeur **numérique** fournie par le **CAN** de l'échantillon prélevé **en temps réel** (à chaque instant d'échantillonnage nT_e) sur le **signal analogique d'entrée $x(t)$** .

↪ un deuxième buffer en **transmission** (**tx_buf**) pour les données traitées numériquement par le DSP et qui seront envoyées vers le **CNA** et déclaré également comme un tableau de 3 entiers initialisés :

```
int tx_buf[3] = {0xCC40 ,0,0} ; /* buffer échantillons en sortie */
```

Ce buffer contiendra la valeur **numérique $y(n)$** (à chaque instant d'échantillonnage nT_e) qui sera envoyé en **temps réel** vers le **CNA** pour fabriquer le **signal analogique de sortie $y(t)$** .

N'hésitez pas à jeter un œil sur le programme complet pour vous en inspirer et pour vérifier la cohérence de ces différentes déclarations avec les principes théoriques que l'on vient de présenter.

Le DSP 21061 utilisé en TP (suite)

La technique de transmission par DMA : en langage C

Voici par exemple la partie permettant de définir une structure de type **TCB** :

```
/* définition d'une structure de type TCB pour le chaînage des DMA de la */  
/* liaison DSP-Codec. Un TCB complet comprend en fait 9 éléments. */  
/* Et non 4 comme dans la présentation simplifiée vue précédemment */  
  
typedef struct  
{  
    unsigned    ei; /* non utilisé ici : multiprocessing */  
    unsigned    em; /* non utilisé ici : multiprocessing */  
    unsigned    ec; /* non utilisé ici : multiprocessing */  
    unsigned    gp1; /* registre d'usage général n°1 */  
    unsigned    gp2; /* registre d'usage général n°2 */  
    unsigned**   cp; /* pointe sur le TCB suivant dans la chaîne */  
                /* pointeur sur un pointeur !!! */  
    unsigned    c; /* taille du bloc à transférer à chaque DMA */  
    int         im; /* incrément pour le calcul de l'adresse de */  
                /* l'élément suivant dans l'échange DMA */  
    unsigned*    ii; /* pointeur buffer : adresse de base du buffer */  
    }            tcb; /* structure tcb */
```

Déclaration de deux TCB dont les noms sont :

- **rx_tcb** : associé au buffer **rx_buf**,
- **tx_tcb** : associé au buffer **tx_buf**.

```
tcb    rx_tcb,tx_tcb ;    /* déclaration de deux tcb */
```

Le DSP 21061 utilisé en TP (suite)

La technique de transmission par DMA : en langage C

Initialisation des éléments de ces deux TCB :

```
rx_tcb.c = 3; /* taille du buffer de réception rx_buf */
rx_tcb.im = 1; /* bloc à transférer du CAN vers le DSP */
               /* incrément pour pointer sur */
               /* l'élément suivant de rx_buf */
rx_tcb.ii = rx_buf; /* pointe sur buffer de réception rx_buf */
rx_tcb.cp = &rx_tcb.ii; /* pointe sur lui-même (autoréférencé) */
                       /* donc paramétrage identique pour */
                       /* toutes les DMA qui vont s'enchaîner */
                       /* les unes à la suite des autres */
                       /* sans intervention du DSP */
```

```
tx_tcb.c = 3; /* taille du buffer d'émission tx_buf */
tx_tcb.im = 1; /* bloc à transférer du DSP vers le CNA */
               /* incrément pour pointer sur */
               /* l'élément suivant de tx_buf */
tx_tcb.ii = tx_buf; /* pointe sur buffer d'émission tx_buf */
tx_tcb.cp = &tx_tcb.ii; /* pointe sur lui-même (autoréférencé) */
                       /* donc paramétrage identique pour */
                       /* toutes les DMA qui vont s'enchaîner */
                       /* les unes à la suite des autres */
                       /* sans intervention du DSP */
```

Le CODEC AD1847 utilisé en TP

Présentation générale

Jusque là, nous sommes restés à l'intérieur du **DSP** qui représente le cœur du traitement **numérique** du signal.

Passons maintenant à l'interface avec le monde **analogique**, assurée ici par un **CODEC**.

Un **CODEC** correspond à l'association dans un même composant :

↳ d'un **CAN** : Convertisseur Analogique Numérique (en **entrée**),

↳ d'un **CNA** : Convertisseur Numérique Analogique (en **sortie**).

Voici le synoptique complet **constructeur** du codec **AD1847**.

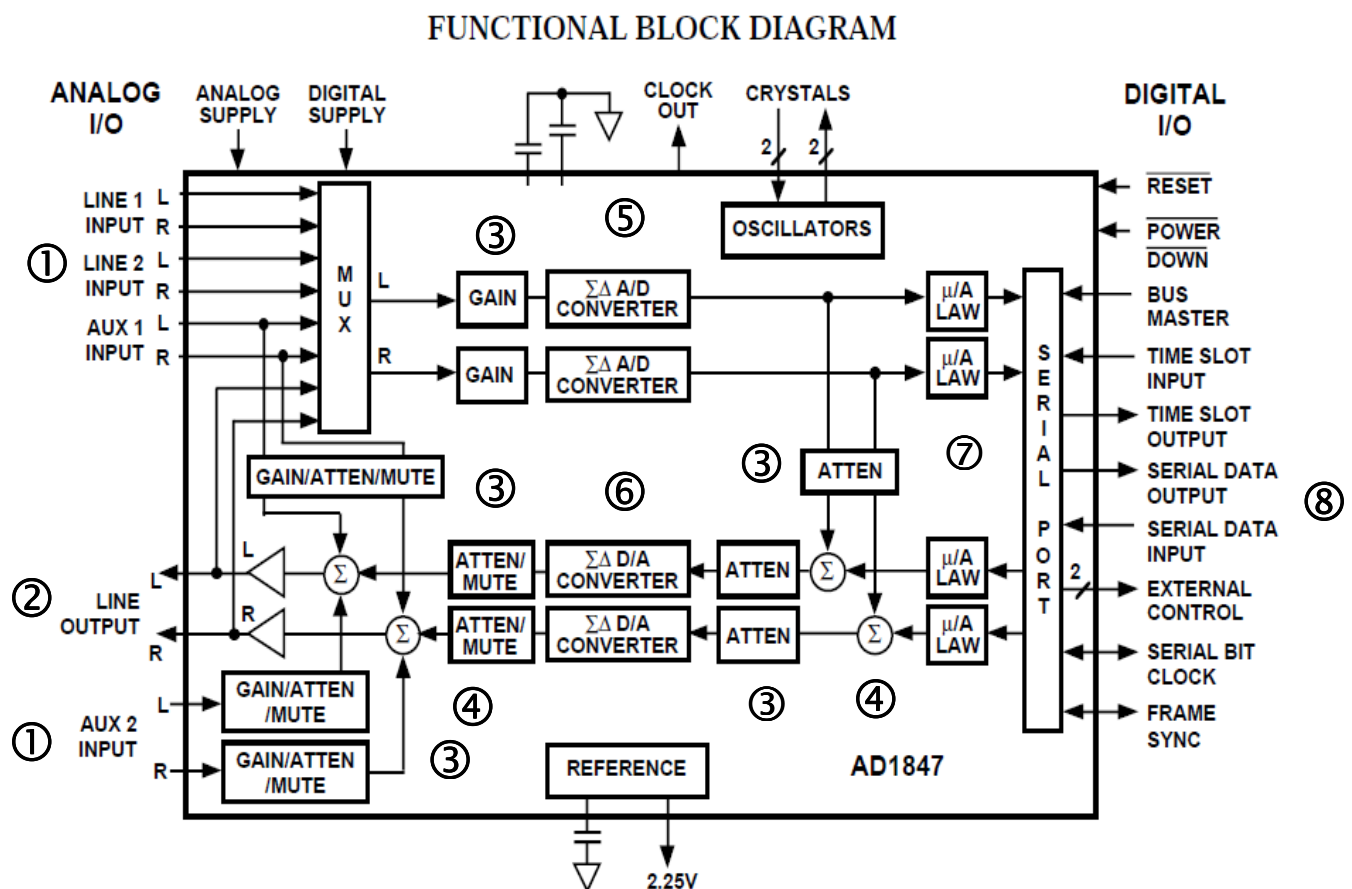


Figure V.1. (page 51)

Le CODEC AD1847 utilisé en TP

Présentation générale (suite)

On y distingue entre autre :

① les **entrées analogiques** (broches de type **INPUT**) :

↳ 4 entrées : **LINE1**, **LINE2**, **AUX1** et **AUX2** dont les voies sont **dédoublées** : une voie **gauche** (Left) et une voie **droite** (Right) car ce codec fonctionne en **stéréo** (2 prises de son).

② les **sorties analogiques** (broches de type **OUTPUT**) :

↳ 1 sortie : **LINE OUTPUT** aussi en **stéréo** (pour 2 haut-parleurs).

③ un ensemble **d'atténuateurs** (blocs **ATTEN**) ou **d'amplificateurs** (blocs **GAIN**) à gain programmable via des registres internes du Codec :

↳ pour gérer les **niveaux** des différents signaux.

④ un ensemble de **sommateurs** (blocs Σ) :

↳ pour gérer le **mixage** et/ou le **cheminement** des signaux analogiques ou numériques dans le composant en fonction de l'application à réaliser.

⑤ et ⑥ deux **CAN** (blocs **A/D CONVERTER**) et deux **CNA** (blocs **D/A CONVERTER**)

↳ 1 **CAN** et 1 **CNA** par voie à cause du fonctionnement en stéréo.

⑦ un ensemble de blocs de type μ/A **LAW** :

↳ pour gérer la **compression/décompression** des données.

⑧ un ensemble de **broches d'E/S** :

↳ pour le **couplage** et le **dialogue** avec le **DSP**.

Le CODEC AD1847 utilisé en TP

Conversion de type $\Sigma\Delta$

Différents procédés existent pour réaliser le passage du monde analogique vers le monde numérique, et vice-versa :

- convertisseurs **simple ou double rampe** :

⇒ principe **assez ancien** et **pas très précis**,

- convertisseurs à **approximations successives** :

⇒ **bon marché** mais **assez lents**,

- convertisseurs « **flash** » :

⇒ parmi les **plus rapides** mais aussi **les plus chers**, réservés plutôt aux applications vidéo où on n'a pas le choix...

En pratique, le choix final dépend, comme souvent, du meilleur compromis entre la **rapidité**, la **précision** et le **coût**.

La technique de conversion mise en œuvre dans le codec **AD1847** dite $\Sigma\Delta$ est ici un bon compromis entre ces trois alternatives.

Son principal avantage est de n'utiliser que des circuits issus du domaine numérique.

Le CODEC AD1847 utilisé en TP

Conversion de type $\Sigma\Delta$ (suite)

Sans être exhaustif, voici le principe de fonctionnement d'un tel CAN, basé sur la remarque suivante :

dans le cas du son (ou de l'image également), dans la très grande majorité des cas, chaque échantillon prélevé à l'instant $t_n = n.T_e$ «ressemble» à celui prélevé à l'instant précédent $t_{n-1} = (n-1).T_e$

En d'autres termes, la partie commune de deux échantillons **successifs** n'est pas pertinente, seul compte la différence entre les valeurs de ces deux échantillons consécutifs.

L'idée sera donc de ne coder que cette **différence** entre ces deux échantillons, seule porteuse d'information.

Cela permet d'**éviter les redondances** :

⇒ partie commune des deux échantillons

et de ne prendre en compte que les **informations pertinentes** :

⇒ la différence entre 2 échantillons.

Voilà pour expliquer le second terme Δ du nom de ce convertisseur.

Voici le schéma de principe pour faire une description **qualitative**.

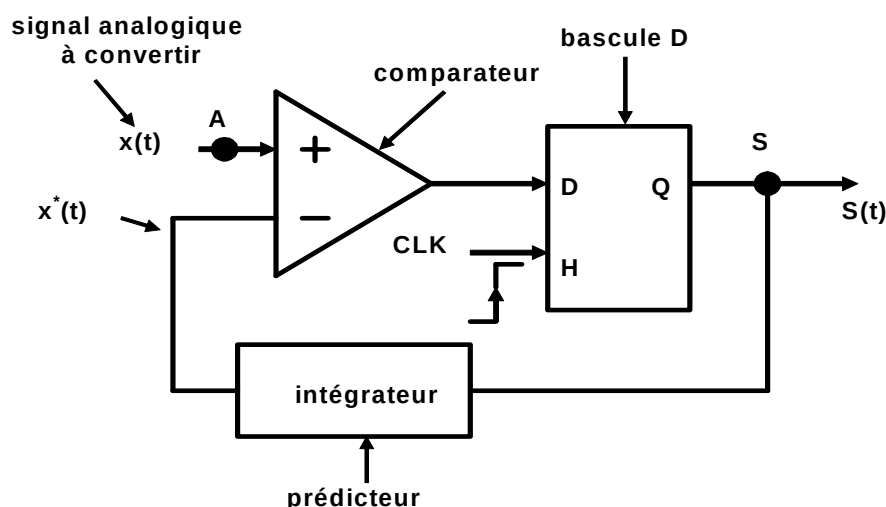


Figure V.2. (page 52)

Le CODEC AD1847 utilisé en TP

Conversion de type $\Sigma\Delta$ (suite)

On compare :

- la valeur du signal analogique à convertir $x(t)$ à l'instant t_n et injecté sur l'entrée $+$ du comparateur,
- avec une image $x^*(t)$ de la valeur de ce signal à l'instant t_{n-1} et injecté sur l'entrée $-$ du comparateur : on verra un peu plus loin comment est déterminé cette valeur $x^*(t)$.

Le comparateur est prévu pour fournir :

- soit un niveau **haut** (**1** logique) si $x(t_n) \geq x^*(t_{n-1})$,
- soit un niveau **bas** (**0** logique) si $x(t_n) < x^*(t_{n-1})$.

En sortie du comparateur, on aura un signal qui vaut soit **1**, soit **0**.

Au rythme d'une horloge **CLK** (en réalité sur ses fronts montants ou descendants), la sortie du comparateur est mémorisée sur la sortie **Q** d'une **bascule D**.

On aura donc au point **S** un signal **S(t)** formé d'une séquence de **1** et de **0**, **synchrone** avec le signal **CLK**, codant la différence entre deux échantillons successifs :

↪ un **1** va coder une **augmentation** du signal $x(t)$,

↪ un **0** codera une **diminution** de ce signal $x(t)$.

Cette horloge **CLK** sert aussi à fixer les différents instants t_n .

Le CODEC AD1847 utilisé en TP

Conversion de type $\Sigma\Delta$ (suite)

Au final, quand le signal $S(t)$:

- « passera plus de temps » à **1** qu'à **0** :
↳ cela voudra dire que le signal $x(t)$ a tendance à **augmenter**.
- « passera plus de temps » à **0** qu'à **1** :
↳ cela voudra dire que le signal $x(t)$ a tendance à **diminuer**.
- « passera autant de temps » à **1** qu'à **0** (signal carré) :
↳ cela voudra dire que le signal $x(t)$ **varie très peu**.

Ainsi en intégrant $S(t)$, on obtiendra l'image recherchée $x^*(t_{n-1})$ de l'échantillon **précédent**, prédiction de la valeur du signal analogique $x(t_{n-1})$. D'où la présence de **l'intégrateur** sur la figure précédente, nommé ici un **prédicteur**, car il permet de « prédire » $x^*(t_{n-1})$.

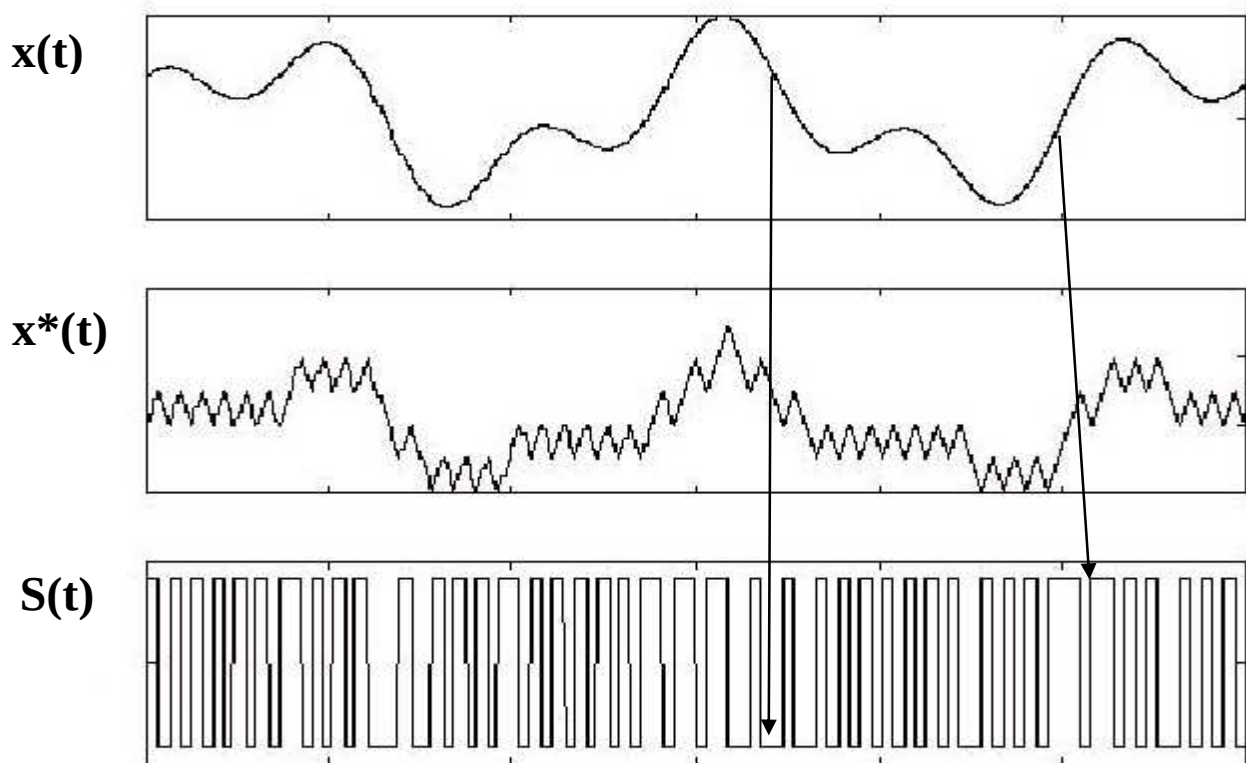


Figure V.3. (page 53)

Le CODEC AD1847 utilisé en TP

Conversion de type $\Sigma\Delta$ (suite)

Un tel convertisseur fonctionne finalement sur 1 bit pour coder soit l'augmentation, soit la diminution du signal analogique $x(t)$.

Il permet de travailler à des fréquences (fréquence du signal **CLK**) **très élevées** :

d'après la documentation technique de l'**AD 1847**, la fréquence de **CLK** vaut **64 fois la fréquence d'échantillonnage** programmée.

Ce qui simplifie le filtre **anti-aliasing** analogique (ou filtre **d'anti-repliement**) nécessaire avant la conversion et dont le rôle est de supprimer, dans le spectre de $x(t)$, toute fréquence supérieure à $\frac{F_e}{2}$.

Ici, un simple RC analogique du premier ordre est suffisant, un **filtre passe-bas numérique** interne au convertisseur venant s'ajouter à la suite pour limiter la bande de fréquences du signal d'entrée à 0,4 fois la fréquence d'échantillonnage programmée et supprimer ainsi tout repliement spectral dans la bande de base.

On peut cependant montrer qu'un tel convertisseur présente des défauts si la tension d'entrée $x(t)$ varie trop vite : la séquence de 1 et de 0 en sortie du comparateur n'arrive plus à suivre cette trop rapide variation de $x(t)$ (un peu comme pour le slew-rate d'un A.O.P.).

Pour atténuer ce problème et éviter les trop fortes variations de $x(t)$, on commence donc par intégrer $x(t)$ en rajoutant un intégrateur au point **A** (cf. figure V.2. page 52) avant de l'injecter sur l'entrée **+** du comparateur d'entrée. Il faudra bien sûr rajouter un dérivateur plus en aval dans la chaîne pour être bien conforme avec $x(t)$.

Voilà pour expliquer le premier terme Σ du nom de ce convertisseur.

Le CODEC AD1847 utilisé en TP

Format des données

Voici les **trois formats de données** gérés par ce Codec.

① 16 bits signés et **linéaire**

Ce format est largement utilisé en **audio**.

Voici la **courbe de transfert** entre :

- la valeur **numérique** (en abscisses),
- la valeur **analogique** (en ordonnées).

Cette courbe est bien **linéaire**, bien qu'en fait elle soit constituée d'un très grand nombre ($2^{16} = 65536$) de marches d'escaliers comme le montre le zoom sur cette figure.

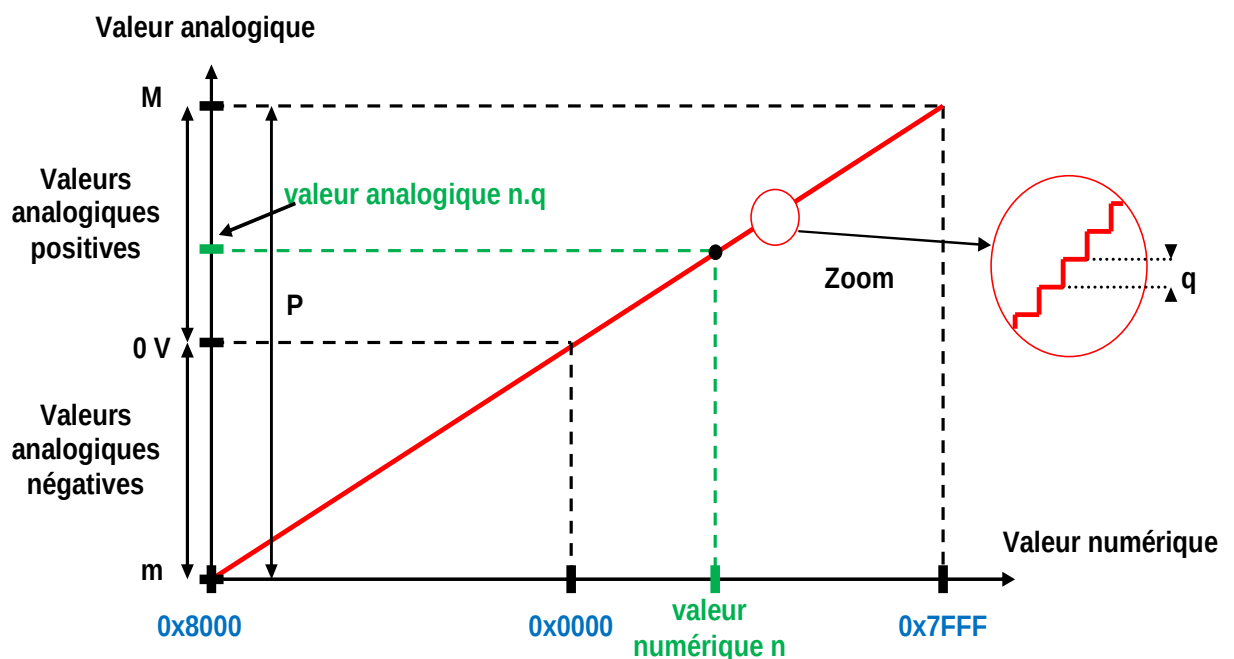


Figure V.4. (page 55)

Le CODEC AD1847 utilisé en TP

① 16 bits signés et linéaire (suite)

Côté analogique : axe des ordonnées de la courbe de transfert

⇒ la différence $P = M - m$ est appelée **la plage de conversion** :
Full Scale Input Voltage

⚡ ⚡ : **risque de saturation du convertisseur !!!**

Les variations du signal analogique d'entrée doivent rester entre ces deux limites sous peine de **saturer (voire endommager)** le convertisseur :

$$m \leq x(t) \leq M$$

On a d'ailleurs ici $m = -M$ (valeurs symétriques).

Extrait de la documentation technique de l'**AD1847** :

ANALOG INPUT	Min	Typ	Max	Units
Full-Scale Input Voltage (RMS Values Assume Sine Wave Input)		1		V rms
	2.54	2.8	3.10	V p-p

Et donc : $M \cong +1,4 \text{ V } (+\sqrt{2} \text{ V})$ et $m \cong -1,4 \text{ V } (-\sqrt{2} \text{ V})$.

⇒ la quantité $q = \frac{M - m}{2^{16}}$ est appelée **le pas de quantification**.

Elle donne une information sur la précision de la conversion :
c'est la hauteur de chaque marche d'escalier (cf. zoom précédent)

Chaque fois qu'**on rajoute un bit** sur la valeur numérique de la conversion, **on multiplie la précision par deux**.

⇒ le nombre $20 \cdot \log(2^{16}) = 320 \cdot \log(2) = \mathbf{96 \text{ dB}}$ est **la dynamique** du codage, image de la **précision**.

Le CODEC AD1847 utilisé en TP

① 16 bits signés et linéaire (suite)

Côté numérique : axe des abscisses de la courbe de transfert

Les valeurs numériques du résultat de la conversion sont sur **16 bits**.

0b 0000 0000 0000 0000 = 0x0000 = (0)₁₀ :

↪ correspond à la valeur analogique **nulle (0V)**,

0b 0111 1111 1111 1111 = 0x7FFF = (32 767)₁₀ :

↪ correspond à la valeur analogique **maximum (M)**,

0b 1000 0000 0000 0000 = 0x8000 = (-32 768)₁₀ :

↪ correspond à la valeur analogique **minimum (m)**.

Le codage est **bien signé** (c'est-à-dire en complément à 2) :

- les valeurs **analogiques positives** donneront une valeur numérique avec le **MSB** à **0** : entre **0x0000** et **0x7FFF**,
- les valeurs **analogiques strictement négatives** donneront une valeur numérique avec le **MSB** à **1** : entre **0x8000** et **0xFFFF**.

Le CODEC AD1847 utilisé en TP

② 8 bits non signés et **linéaire**

La courbe de transfert de ce format est également **linéaire** avec la même réserve sur sa forme en marche d'escaliers de :

$2^8 = 256$ marches ici, donc individuellement « plus hautes ».

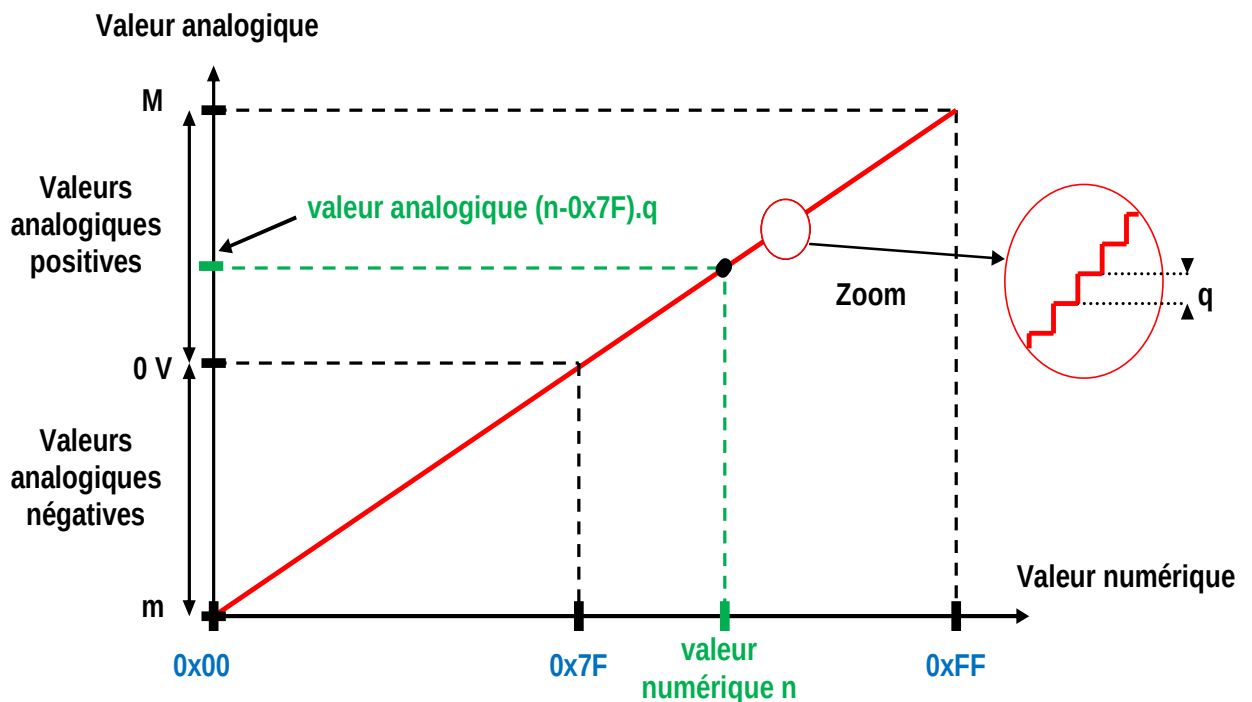


Figure V.5. (page 56)

⇒ les valeurs extrêmes **M** et **m** de la tension analogique d'entrée sont bien sûr les mêmes que dans le cas précédent.

Le CODEC AD1847 utilisé en TP

② 8 bits non signés et linéaire (suite)

⇒ le pas de quantification $q = \frac{M - m}{2^8}$ est ici 256 fois plus grand que précédemment : la précision en sera directement affectée. (chaque valeur numérique code un intervalle de valeurs analogiques plus grand).

⇒ la dynamique du codage : $20 \cdot \log(2^8) = 160 \cdot \log(2) = 48 \text{ dB}$ est ici 2 fois plus faible.

⇒ les valeurs numériques du résultat sont sur 8 bits.

0b 0000 0000 = 0x00 = (0)₁₀ :

↪ correspond à la valeur analogique minimale (m),

0b 0111 1111 = 0x7F = (127)₁₀ :

↪ correspond à la valeur analogique nulle (0V),

0b 1111 1111 = 0xFF = (255)₁₀ :

↪ correspond à la valeur analogique maximale (M).

Le codage est bien non signé (au sens classique du terme) puisque :

- les valeurs analogiques strictement positives donneront une valeur numérique avec le MSB à 1 : entre 0x80 et 0xFF,
- les valeurs analogiques négatives donneront une valeur numérique avec le MSB à 0 : entre 0x00 et 0x7F.

C'est ce qu'on appelle classiquement le « binaire naturel ».

Le CODEC AD1847 utilisé en TP

③ 8 bits comprimés

La courbe de transfert entre le domaine analogique et le domaine numérique n'a plus globalement la forme d'une droite.

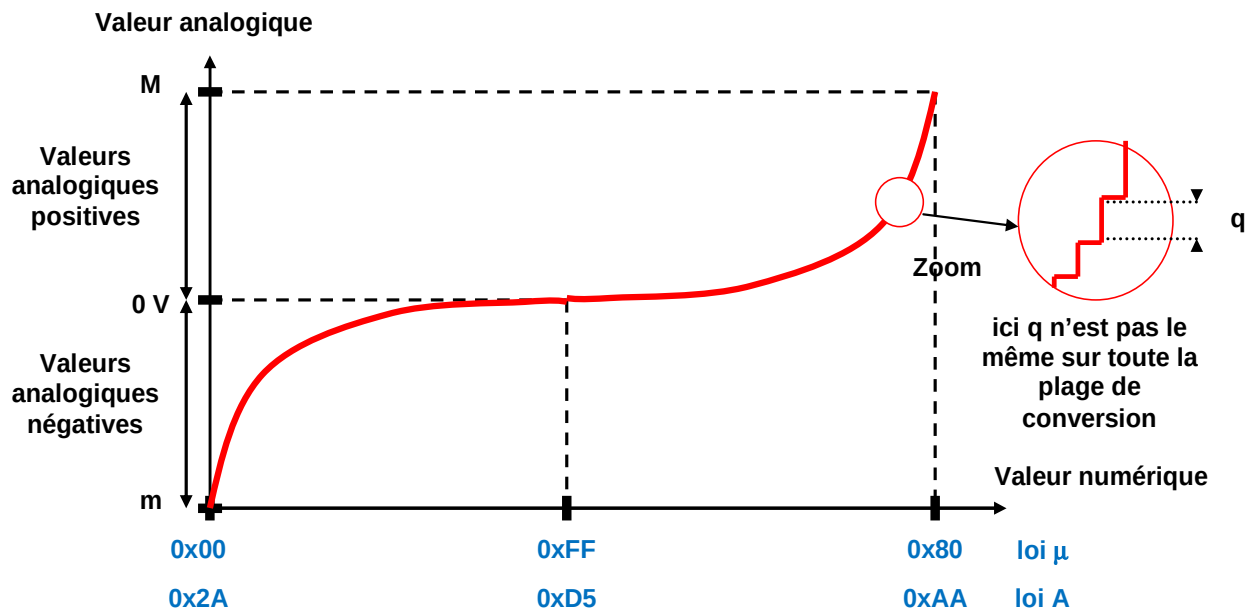


Figure V.6. (page 57)

Cette courbe correspond à deux types de lois :

- **loi A** : normes Européennes,
- **loi μ** : normes Américaines et Japonaises.

D'où vient la forme un peu « bizarre » de cette courbe ?

Dans un codage linéaire, toutes les « marches » de l'escalier ont la même hauteur, qu'on soit :

- au milieu de l'escalier : valeurs analogiques proches du 0V et donc plutôt de faibles valeurs absolues,
- ou à ses extrémités : valeurs analogiques éloignées du 0V et donc de valeurs absolues plutôt importantes.

Ce qui revient à dire que **le pas de quantification q est le même sur toute la plage de conversion du convertisseur.**

Le CODEC AD1847 utilisé en TP

③ 8 bits comprimés (suite)

Mais on peut raisonnablement se dire qu'il serait plus intéressant d'avoir une précision plus importante pour les faibles valeurs analogiques que pour les grandes.

En effet, faire une erreur de 1 mV sur une valeur analogique de 1 V est moins grave que faire la même erreur de 1 mV sur une grandeur analogique de 1 mV !!!

Et donc d'avoir un escalier :

↳ plutôt « **plat** » aux **alentours du 0V (plus précis)**,

↳ et de plus en plus « **raide** » quand on **s'éloigne de part et d'autre du 0V (moins précis)**.

Au final, avec 8 bits comprimés :

- la loi **A** donne une **dynamique équivalente** de **72 dB**,
- la loi **μ** donne une **dynamique équivalente** de **64 dB**.

Soit une valeur **intermédiaire** entre le format 8 bits linéaire (**48 dB**) et le format 16 bits linéaire (**96 dB**).

Cette technique sera surtout utilisée en **télécommunications audio numériques** car elle permet d'avoir une **dynamique tout à fait acceptable** uniquement en codant le son sur 8 bits (et non sur 16 bits), ce qui permet de **gagner un facteur 2** dans les transmissions, toute chose étant égale par ailleurs.

Au final, c'est un bon compromis entre la **précision** de la conversion et la **rapidité** de transmission.

Le CODEC AD1847 utilisé en TP

③ 8 bits comprimés (suite)

A la base, le Codec travaille toujours en 16 bits linéaire. Ce n'est :

- qu'après : en **sortie** du **CAN**,
- ou avant, en **entrée** du **CNA**,

qu'on peut interposer (ou pas) via un registre interne du codec, les blocs **μA LAW** ⑦ (cf. figure V.1. page 51) pour diminuer le nombre de bits échangés entre le Codec et le DSP, tout en gardant, comme on vient de le voir une précision acceptable.

Comment tout cela fonctionne t'il ?

- dans le sens **CAN** vers **DSP** (donné pour la loi A) :

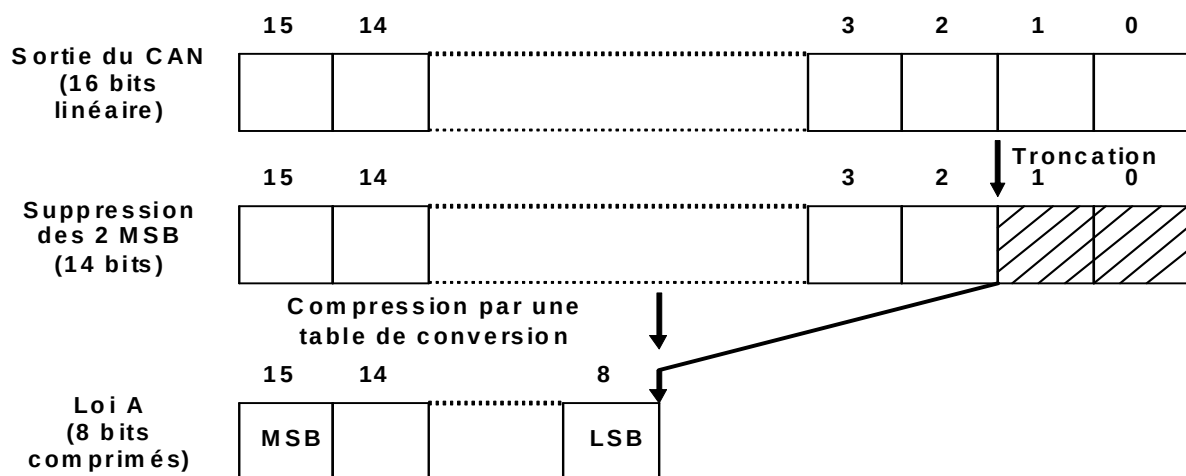


Figure V.7. (page 58) : loi A

- **troncation** de la valeur 16 bits pour ne garder que **14 bits** (loi A) (ou **13 bits** : loi μ) en **supprimant** les 2 (ou 3) bits de poids faibles,
- compression** en utilisant une **table de conversion** pour respecter la forme de la courbe de la figure V.6. (page 57) et passer sur 8 bits. C'est pendant cette phase de compression que sera adaptée la valeur du pas de quantification q en fonction de la valeur absolue du signal.

Le CODEC AD1847 utilisé en TP

③ 8 bits comprimés (suite)

- dans le sens **DSP** vers **CNA** (donné pour la loi A) :

C'est un peu le mécanisme inverse qui se produit.

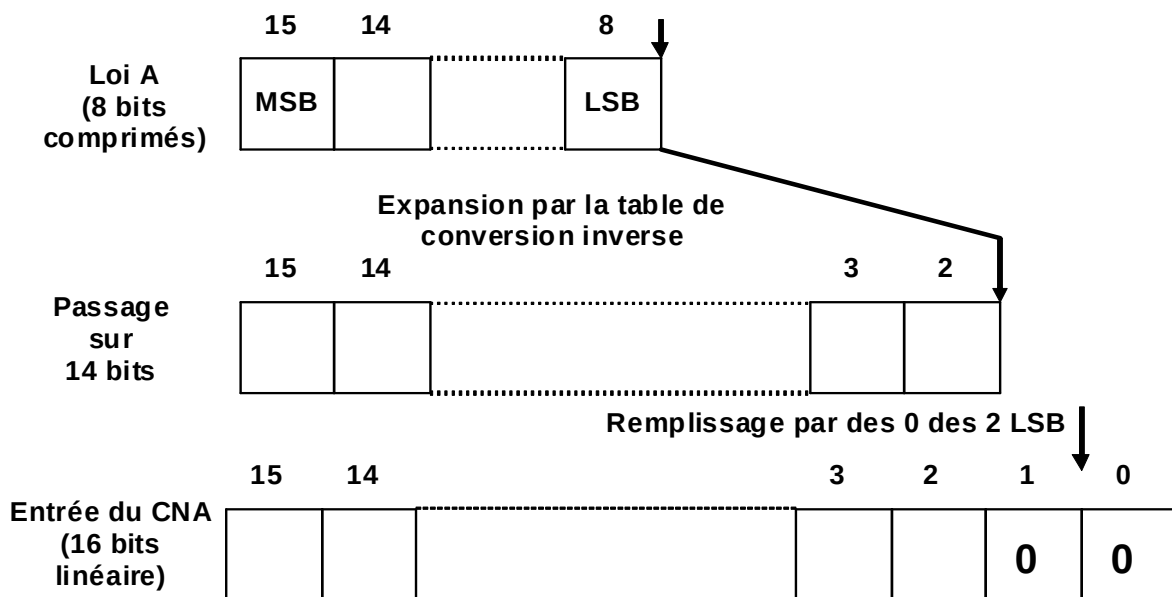


Figure V.8. (page 59) : loi A

- **expansion** de la valeur numérique reçue de 8 bits vers 14 bits pour la loi A (ou vers 13 bits pour la loi μ) en utilisant une **table de conversion inverse** de la précédente.

- **remplissage par des 0** des 2 (ou 3) bits de poids faibles pour obtenir une valeur numérique sur 16 bits qui sera utilisée comme entrée du **CNA**.

Notons enfin que les liaisons séries **SPORT0** et **SPORT1** côté DSP peuvent aussi gérer **automatiquement** tous ces mécanismes de compression/expansion, ce qui évite d'avoir à écrire sur le DSP la partie de code qui l'effectuerait si ce n'était pas le cas.

Le CODEC AD1847 utilisé en TP

Liaison DSP ↔ CODEC : protocole TDM

Il est temps maintenant de coupler le **DSP** et le **Codec** via la liaison série synchrone **SPORT0**.

L'échange des valeurs numériques des échantillons entre le Codec et le DSP utilise le protocole **TDM** (**T**ime **D**ivision **M**ultiplexed).

A première vue, ce protocole semble compliqué et on peut se demander ce qui a motivé ses concepteurs.

L'idée de base est de pouvoir gérer des liaisons **multipoints**, pour avoir plusieurs émetteurs/récepteurs de données sur la même liaison.

Pour cela, des tranches de temps (appelées des **Slots**) sont allouées à chacun des éléments pouvant dialoguer sur la liaison.

↳ d'où le nom de **Time Division Multiplexed** :

multiplexage temporel par division du temps en slots.

Pour bien calibrer les différents slots alloués, il est nécessaire de connaître le type et la quantité d'informations qui vont être échangés pendant ce couplage.

Ces informations seront repérées et numérotées dans la suite par les chiffres ① à ⑤.

Le CODEC AD1847 utilisé en TP

Liaison DSP $\rightleftharpoons$ CODEC : protocole TDM (suite)

Bilan des informations à échanger entre DSP et Codec

Cas simple d'un seul DSP et d'un seul Codec.

- du DSP vers le Codec :

- ① **initialisation des registres internes du Codec (Control Word)**

- choix des différents gains,
- cheminement interne par mixage des différents signaux,
- choix de la fréquence d'échantillonnage F_e ,
- mise en route de l'auto-calibrage ...

- ① **envoi d'une valeur numérique vers le CNA de la voie gauche**

- ② **envoi d'une valeur numérique vers le CNA de la voie droite**

- du Codec vers le DSP :

- ③ **envoi de l'état interne du Codec pour sa surveillance :**

- saturations éventuelles sur les voies d'entrées (**Status Word**),
- contenu des registres de configuration (**Index Readback**) ...

- ④ **envoi de la sortie numérique du CAN de la voie gauche :**

- venant de **Line 1 Left Input**,
- ou de **Line 2 Left Input**,
- ou de **Aux 1 Left Input** ...

tout ceci suivant la configuration effectuée pendant la phase ④.

- ⑤ **envoi de la sortie numérique du CAN de la voie droite :**

- venant de **Line 1 Right Input**,
- ou de **Line 2 Right Input**,
- ou de **Aux 1 Right Input** ...

tout ceci suivant la configuration effectuée pendant la phase ④.

Le CODEC AD1847 utilisé en TP

Liaison DSP $\leftrightarrow$ CODEC : protocole TDM (suite)

Broches concernées par le couplage DSP $\leftrightarrow$ CODEC

Tant au niveau du Codec que du DSP, il existe 2 **broches** notées :

- **SDO** comme **S**erial **D**ata **O**utput :

↪ **broche de sortie** par laquelle les valeurs numériques sortiront **bit à bit** du composant.

Côté **Codec**, c'est la **sortie** du **CAN** qui donnera **x(n)**

Côté **DSP**, c'est la **sortie** correspondant par exemple à **y(n)** (sortie du filtre numérique).

- **SDI** comme **S**erial **D**ata **I**ntput :

↪ **broche d'entrée** par laquelle les valeurs numériques rentreront **bit à bit** dans le composant.

Côté **Codec**, c'est l'**entrée** du **CNA** (donc **y(n)**)

Côté **DSP** c'est l'**entrée** correspondant par exemple à **x(n)** (entrée du filtre numérique).

Pour réaliser cet échange, **deux solutions hardware** sont possibles.

Le CODEC AD1847 utilisé en TP

Liaison DSP $\rightleftharpoons$ CODEC : protocole TDM (suite)

Solutions hardware

- Liaison à un seul fil

La sortie série **SDO** (côté DSP et Codec) et l'entrée série **SDI** (côté DSP et Codec) sont reliées entre elles par **un seul** fil.

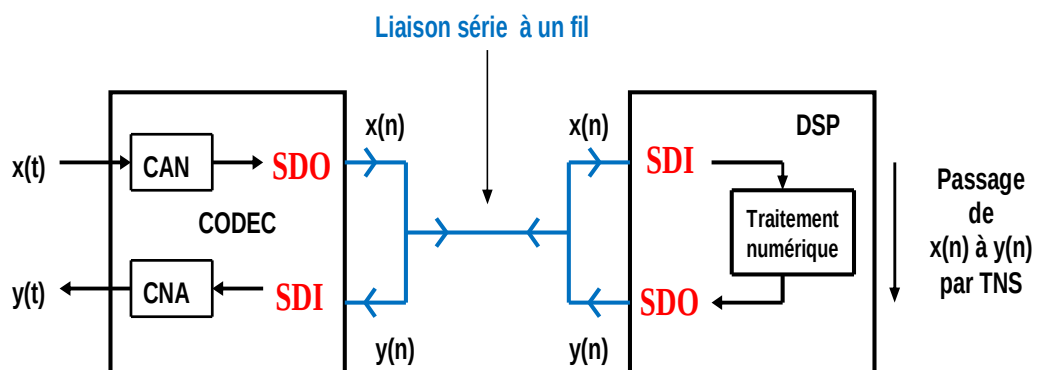


Figure V.9. (page 62)

L'intérêt est de n'avoir qu'un seul fil de dialogue **bidirectionnel** entre le DSP et le Codec.

Par contre, on ne peut faire du **vrai full duplex** sur un seul fil.

On ne peut faire que du **half duplex** (ou semi-duplex).

En effet si le Codec et le DSP envoient en même temps les valeurs numériques $x(n)$ et $y(n)$, il y aura **collision** entre ces valeurs.

Le CODEC AD1847 utilisé en TP

Liaison DSP ↔ CODEC : protocole TDM (suite)

• Liaison à un seul fil (suite)

On peut quand même réaliser **du full duplex** mais à l'alternat en utilisant le **multiplexage temporel** :

⇒ 3 tranches de temps consécutives (**SLOT 0**, **SLOT 1** et **SLOT 2**) et dans cet ordre sont allouées pour les blocs ①, ② et ③ précédents,

⇒ suivies immédiatement de 3 autres tranches de temps consécutives (**SLOT 3**, **SLOT 4** et **SLOT 5**) et dans cet ordre pour les blocs ④, ⑤ et ⑥.

Ce qui assure les échanges **dans les deux sens** et **sans collision**.

Mais avec une perte de temps, car il faut consommer **6 tranches** de temps au total pour un échange complet dans les deux sens.

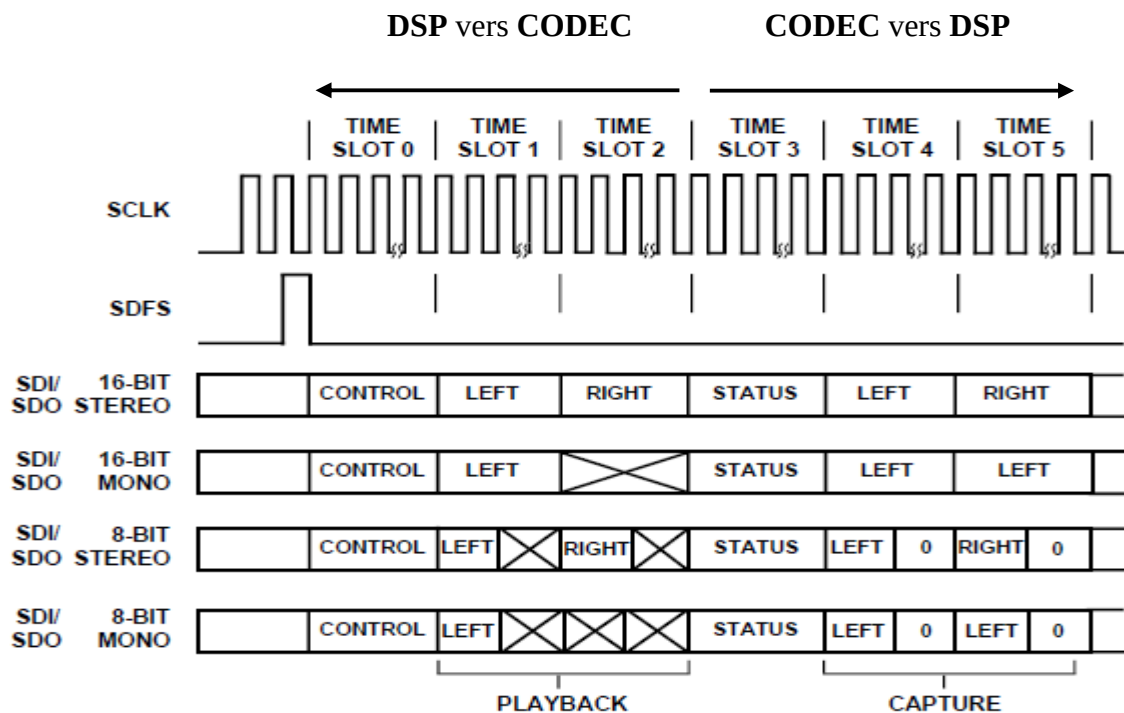


Figure V.10. (page 62)

Le CODEC AD1847 utilisé en TP

Liaison DSP $\rightleftharpoons$ CODEC : protocole TDM (suite)

Solutions hardware (suite)

- Liaison à deux fils

Chaque sortie série **SDO** (côté DSP et Codec)
est reliée
à chaque entrée série **SDI** (côté DSP et Codec)
et donc par **deux** fils indépendants.

Il y a maintenant deux fils **monodirectionnels** et **différents** pour **SDI** et **SDO**, donc plus aucun risque de collisions.

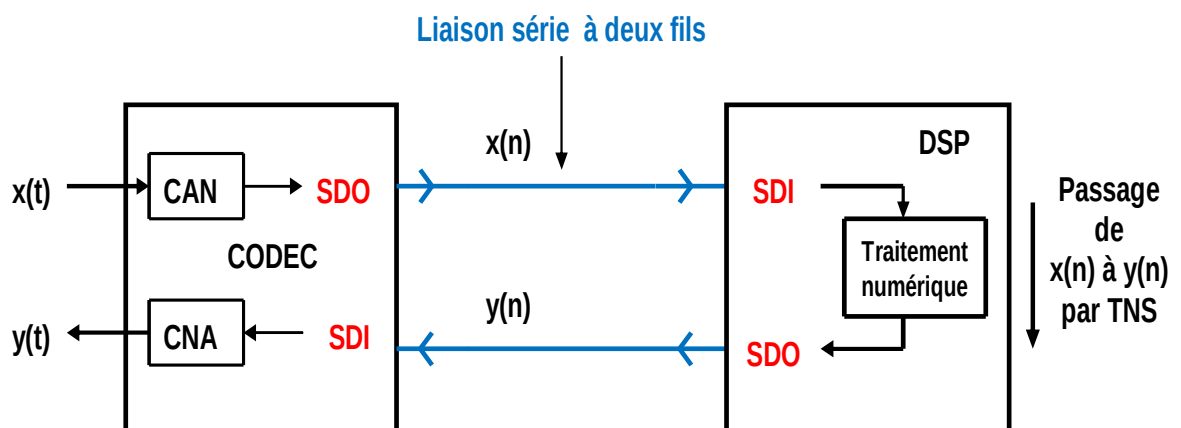


Figure V.11. (page 63)

C'est cette solution qui est choisie sur la carte de développement utilisée en TP.

Le CODEC AD1847 utilisé en TP

Liaison DSP ↔ CODEC : protocole TDM (suite)

• Liaison à deux fils (suite)

On peut donc envisager d'affecter 3 tranches de temps **identiques** et **synchrones** (SLOT 0, SLOT 1 et SLOT 2) pour les blocs ①, ② et ③ et pour les blocs ④, ⑤ et ⑥ (c'est-à-dire bloc ① en même temps que bloc ④ etc...).

D'où un gain en temps d'un facteur 2 par rapport au cas précédent, puisque seulement **3 tranches** de temps sont consommées pour un échange complet dans les deux sens.

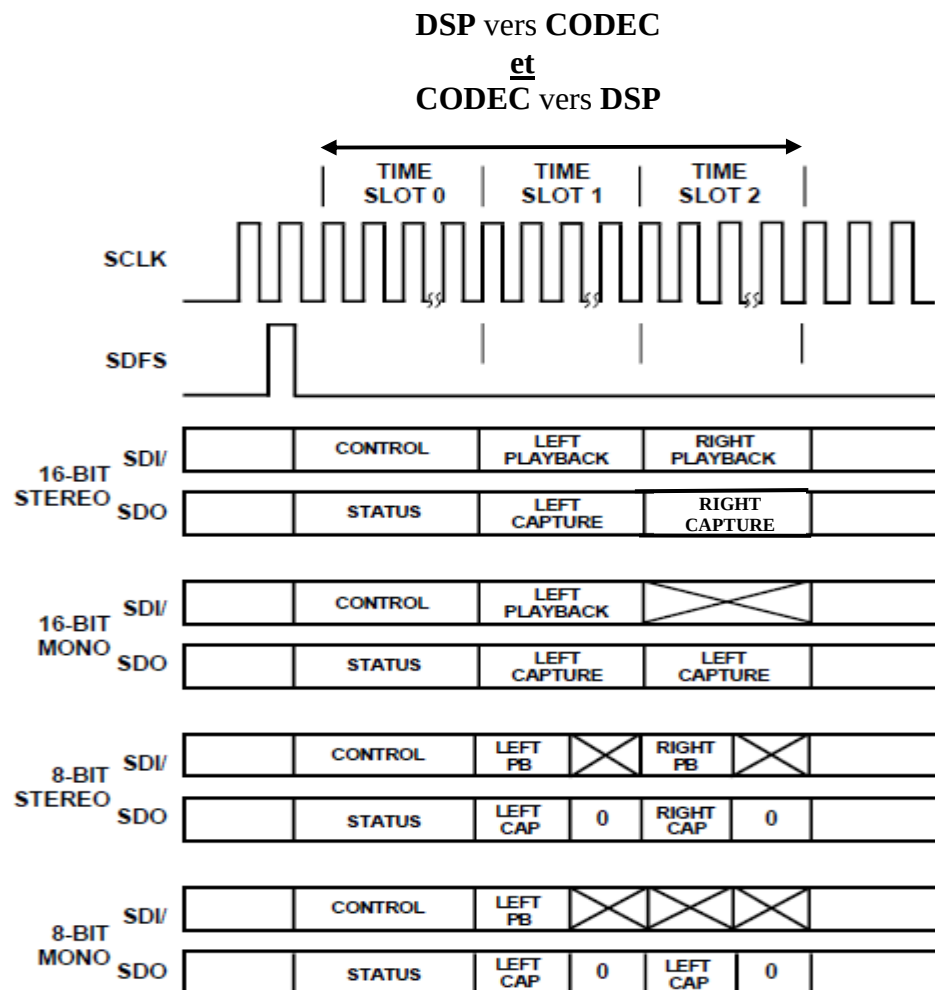


Figure V.12. (page 63)

Le système d'évaluation EZ-KIT utilisé en TP

Présentation générale

Nous allons maintenant donner quelques éléments sur la **carte d'évaluation** que vous utiliserez pendant les séances de TP.

Même **méthodologie de travail**, très standard, que celle utilisée lors des développements faits sur microcontrôleurs :

- ⇒ **écriture, compilation, linkage** des programmes sur le PC,
- ⇒ carte d'évaluation à **l'extérieur** et reliée au PC,
- ⇒ **téléchargement** des exécutables du PC vers la carte d'évaluation,
- ⇒ **test, mise au point et validation** de ces programmes en fonction du cahier des charges.

Architecture matérielle et synoptique

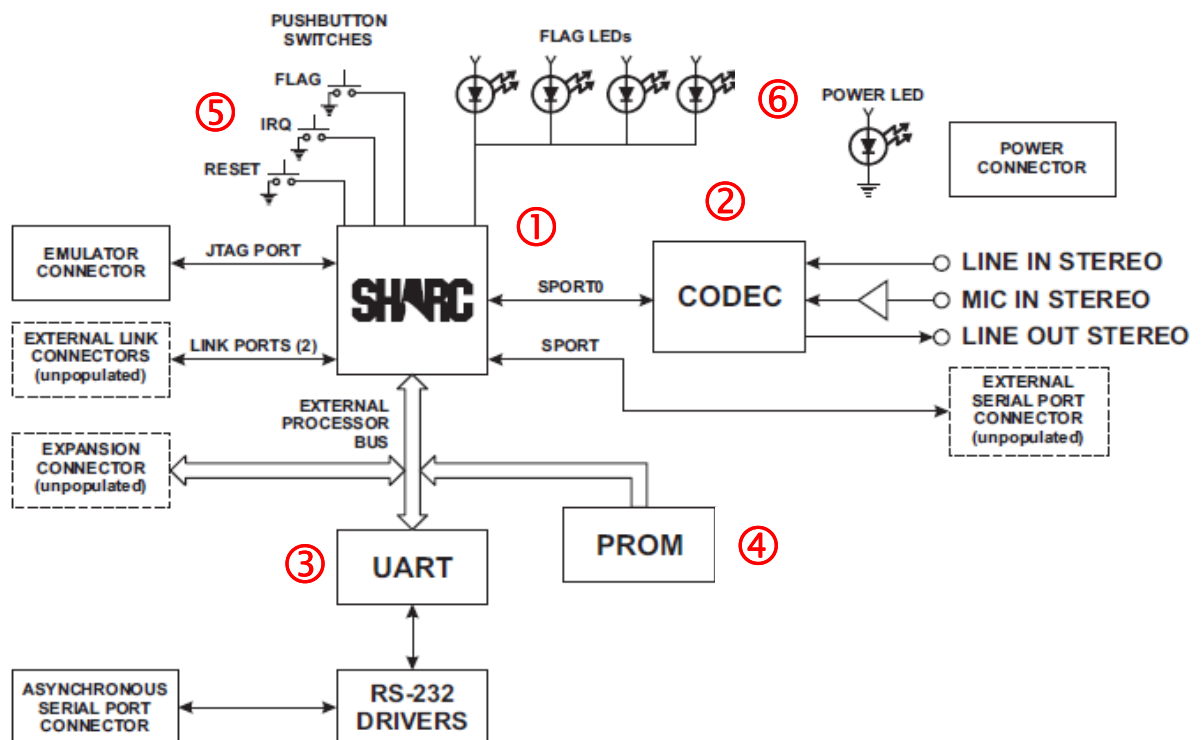


Figure VI.1. (page 65)

Le système d'évaluation EZ-KIT utilisé en TP

Architecture matérielle et synoptique (suite)

Les principaux éléments vous sont maintenant familiers.

① : le SHARC :

↪ cœur du système : l'ADSP 21061 cadencé à 40 Mhz.

② : le CODEC :

↪ l'AD 1847 avec ses entrées et sorties analogiques et une liaison 2 fils via SPORT0 du DSP.

③ : un UART :

↪ le PC16550 avec l'interface RS232 pour la liaison avec le PC hôte de développement.

Même UART que celui qui équipe la majorité des PC sur leurs COMs (descendant du maintenant déjà ancien 8250). Il fonctionne sous IT (IRQ2 du DSP).

Il est situé par défaut à l'adresse de base \$0040 2000.

④ : une PROM :

↪ pour l'initialisation des RAM du DSP à la mise sous tension.

Notez l'absence de mémoire externe (autre que cette PROM) sur le KIT, les 1 Mégabits internes au DSP 21061 devant suffire pour la plupart des applications qui seront demandées dans le TP.

⑤ un ensemble de 3 switches (PUSHBUTTON SWITCHES) :

↪ reset, simulation d'IRQ1, Flag sur l'input Flag du DSP.

⑥ un ensemble de 5 leds (FLAG LEDs) :

↪ sur les outputs Flags du DSP ou sur l'alimentation.

Le système d'évaluation EZ-KIT utilisé en TP

Implantation physique des composants

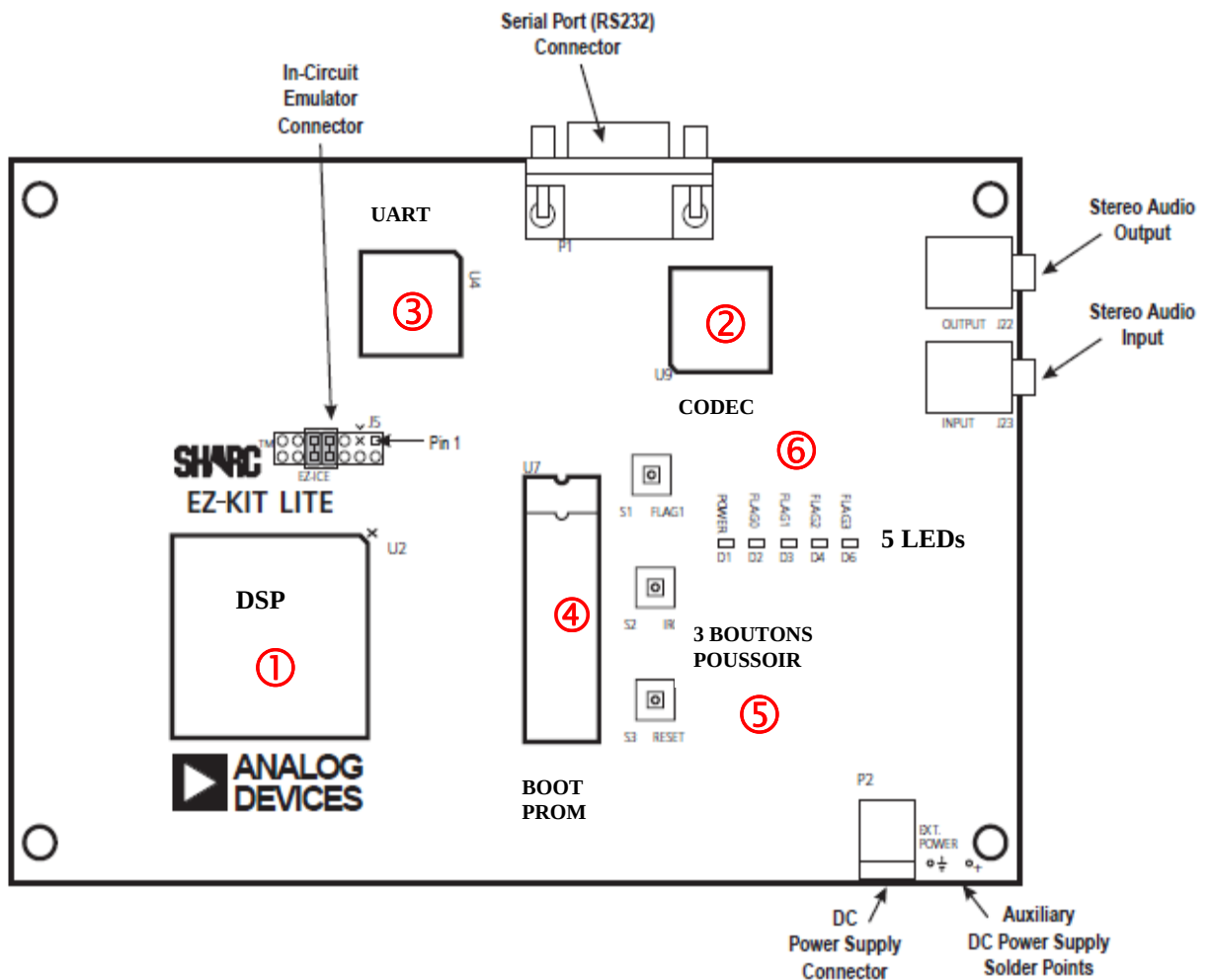


Figure VI.2. (page 66)

Le système d'évaluation EZ-KIT utilisé en TP

Le fichier d'architecture matérielle EZKIT.ACH

Pour fournir aux différents outils de développement :

↳ principalement au **linker** pour placer correctement le code et/ou les données en mémoire,

↳ mais aussi au **simulateur**,

↳ et éventuellement à **l'émulateur**,

des informations sur **l'architecture matérielle** du système :

- type de DSP dans la famille,
- emplacement (adresse de début et adresse de fin) des différents segments de code et de données système et utilisateur,
- taille des cases mémoire pour assurer les échanges via les différents bus...

Ainsi, si on programme en C, le linker connaîtra grâce à ce fichier d'architecture les affectations de ces différentes zones, ce qui évite les problèmes en permettant un **linkage optimal** des fichiers .OBJ.

En assembleur, il faudrait être **plus vigilant** ...

Le système d'évaluation EZ-KIT utilisé en TP

Le fichier d'architecture matérielle EZKIT.ACH (suite)

```

|-----
| .SYSTEM          SHARC_EZKIT_Lite;
|
| This architecture file is required for use with the SHARC EZ-KIT
| Lite development software. It is structured for use with the C
| compiler but also can be used with assembly code.
|
| This architecture file allocates:
|   Internal 133 words of 48-bit run-time header in memory block 0
|       16 words of 48-bit initialization code in memory block 0
|       619 words of 48-bit kernel code in memory block 0
|       7424 words of 48-bit C code space in memory block 0
|       4K words of 32-bit PM C data space in memory block 0
|
|       8K words of 32-bit C DM data space in memory block 1
|       4K words of 32-bit C heap space in memory block 1
|       3712 words of 32-bit C stack space in memory block 1
|       384 words of 32-bit kernel data in memory block 1
|
| .PROCESSOR = ADSP21061;
|
|   -----
|   Internal memory Block 0
|   -----
| .SEGMENT/RAM/BEGIN=0x00020000 /END=0x00020084 /PM/WIDTH=48      seg_rth;
| .SEGMENT/RAM/BEGIN=0x00020085 /END=0x00020094 /PM/WIDTH=48      seg_init;
| .SEGMENT/RAM/BEGIN=0x00020095 /END=0x000202ff /PM/WIDTH=48      seg_knlc;
| .SEGMENT/RAM/BEGIN=0x00020300 /END=0x00021fff /PM/WIDTH=48      seg_pmco;
| .SEGMENT/RAM/BEGIN=0x00023000 /END=0x00023fff /PM/WIDTH=32      seg_pmda;
|
|   -----
|   Internal memory Block 1
|   -----
| .SEGMENT/RAM/BEGIN=0x00024000 /END=0x00025fff /DM/WIDTH=32      seg_dmda;
| .SEGMENT/RAM/BEGIN=0x00026000 /END=0x00026fff /DM/WIDTH=32 /cheap seg_heap;
| .SEGMENT/RAM/BEGIN=0x00027000 /END=0x00027e7f /DM/WIDTH=32      seg_stak;
| .SEGMENT/RAM/BEGIN=0x00027e80 /END=0x00027fff /DM/WIDTH=32      seg_knld;
|
|   -----
|   External Memory Select 1 is reserved for the UART.
|   -----
|
| .ENDSYS;

```

Figure VI.3. (page 67)

On peut rappeler que :

- le **Block 0** de la mémoire correspond à la mémoire de code **PM**,
- le **Block 1** correspond à la mémoire de donnée **DM**.

Le système d'évaluation EZ-KIT utilisé en TP

Le fichier d'architecture matérielle EZKIT.ACH (suite)

Au reset de la carte (à froid ou à chaud) après le chargement de l'EPROM ④ dans les RAM internes du DSP, voici le découpage mémoire et les affectations qui sont faites.

Segment Name	Description	Memory Block	Length	Width	Start Address	End Address
seg_rth	Run-time header	0	133	48	0x00020000	0x00020084
seg_init	Initialization data	0	16	48	0x00020085	0x00020094
seg_knlc	Kernel Code	0	619	48	0x00020095	0x000202FF
seg_pmco	User Code	0	7424	48	0x00020300	0x00021FFF
seg_pmدا	User PM Data	0	4096	32	0x00023000	0x00023FFF
seg_dmdا	User DM Data	1	8192	32	0x00024000	0x00025FFF
seg_heap	User C Heap	1	4096	32	0x00026000	0x00026FFF
seg_stak	User C Stack	1	3712	32	0x00027000	0x00027E7F
seg_knld	Kernel Data	1	384	32	0x00027E80	0x00027FFF

Figure VI.4. (page 68)

Ces affectations appellent les précisions suivantes :

- le segment **seg_rth** (Run-time-header) :

➤ pour la gestion des **vecteurs d'interruptions** et du **reset** et pour le **code d'initialisation du système**. Il est utilisé par exemple par les fonctions du C comme **interrupt**, **signal** ou **raise** (voir plus loin).

- le segment **seg_init** (Initialisation data) :

➤ pour les **données et les paramètres d'initialisation** du système.

- le segment **seg_knlc** (Kernel Code) :

➤ contient le **code du noyau** ou **KERNEL** : petit logiciel de communication fonctionnant sous l'IRQ2 de l'UART pour échanger des paquets de données entre le PC et l'EZ_KIT sur le principe de question / réponse (lecture/écriture mémoire PM ou DM, lecture/écriture registre DSP, lancement de programme...).

Le système d'évaluation EZ-KIT utilisé en TP

Le fichier d'architecture matérielle EZKIT.ACH (suite)

- le segment **seg_knld** (Kernel Data) :

↪ contient les **données** de ce noyau.

- le segment **seg_stak** (User C Stack) :

↪ pour la **pile** du C et le stockage entre autre de l'adresse de retour par la fonction appelante et du pointeur du cadre de pile.

- le segment **seg_heap** (User C Heap) :

↪ pour le « **tas** » du C et les allocations/désallocations dynamiques de mémoire (fonctions **malloc**, **calloc** **realloc** ou **free** du C).

- le segment **seg_pmco** (User Code) :

↪ pour les **programmes** de l'utilisateur.

-les segments **seg_pmda** et **seg_dmda** (User PM Data, DM Data):

↪ pour les **données globales ou statiques** de l'utilisateur, respectivement en **PM** ou en **DM**.

Quand on programme en C, toutes ces affectations sont gérées automatiquement par le linker et sont donc transparentes pour vous.

Remarque

Dans le cours complet d'autres éléments sur les aspects software de l'environnement de l'outil EZ-KIT sont présentés mais non décrits ici, car on aura l'occasion de les mettre en œuvre en TP.

Mise en œuvre en langage C

Mise en pratique de tous les éléments précédents pour écrire un premier programme en C exploitant **au mieux** les principales caractéristiques de la carte d'évaluation.

Un exemple commenté et détaillé en C : « canevas.c »

Notez bien que ce programme ne fait aucun traitement numérique du signal (ou si peu), mais il représente la **structure minimale** nécessaire pour exploiter au mieux les caractéristiques de la carte d'évaluation.

C'est une espèce de « socle » minimal que vous utiliserez pour faire les différents traitements numériques demandés en TP. D'où son nom...

Vous y rajouterez donc les lignes de code nécessaire pour réaliser les exercices et applications demandés en TP.

 **mais attention !!!**

**Compte tenue de la relative complexité de ce programme,
il est conseillé de toujours garder une version
initiale intègre de « canevas.c » dans son répertoire de travail
en changeant son nom
à chaque modification que vous effectuerez.**

Conseil

Relire ce source et le comprendre est un bon moyen de faire les liens entre tous les aspects théoriques et pratiques vus jusqu'à maintenant.

Pour cela, voici quelques remarques sur ce programme.

Mise en œuvre en langage C (suite)

Différentes structures de données et fonctions ont été écrites ou appelées depuis la librairie du C comme par exemple :

- **void init_dsp(void)** :

↳ pour **initialiser** le **DSP** et son **timer interne** pour qu'il génère des **interruptions** périodiques,

- **void init_sport0(void)** :

↳ pour **initialiser** et **configurer** la liaison **SPORT0** entre le DSP et le Codec sous **DMA chaînée** :

- en réception (via la structure tcb **rx_tcb** et le buffer **rx_buf[3]**) et
- en transmission (via la structure tcb **tx_tcb** et le buffer **tx_buf[3]**), avec **traitement des ITs** et gestion du **protocole TDM**,

- **void init_codec(void)** :

↳ pour « **resetter** » correctement le Codec (par sa broche **RESET**) en lançant une **auto-calibration** avant de l'initialiser,

- **void it_timer()** :

↳ routine d'IT pour gérer les ITs du timer interne du DSP,

- **void it_dma_receive()** :

↳ routine d'IT pour traiter les ITs en réception sous DMA des échantillons du signal en entrée,

- **void it_dma_transmit()** :

↳ routine d'IT pour traiter les ITs en émission sous DMA des échantillons du signal en sortie,

- **set_flag()**:

↳ fonction de Analog Devices pour piloter les **FLAGS LEDs**,

- **asm(« »)** :

↳ pour inclure en ligne du code assembleur dans un source en C.

```

/*****
/*      PROGRAMME CANEVAS.C
*****/

/* fichier de définition des registres internes et de certains de leurs champs de bits */
#include <def21060.h>
#include <21060.h>
#include <sport.h>          /* les registres et champs de bits des SPORT0 et 1 */
#include <macros.h>

/* d'autres include pour le traitement du signal ....*/
#include <signal.h>
#include <filters.h>
#include <math.h>

/* définition d'un tableau de 16 Words, chacun correspondant à : */
/* -pour l'octet de poids fort :      CLOR  MCE   RREQ  Res    numéro d'index du registre */
/*                                1      1      0      0      de $00 à $FF */
/* CLOR = 1 : remise à jour des bits de surcharge des CAN d'entrée dans le Status Word entre */
/* chaque acquisition : permet éventuellement si on le gère d'augmenter la précision des CAN */
/* MCE est mis à 1 pour les 15 premiers Words et à 0 au dernier pour faire une autocalibration du */
/* Codec après son initialisation */
/* - pour l'octet de poids faible : valeur à écrire dans ce registre */
/* Ce tableau regs_1847 est en fait la liste des 16 mots de contrôle qu'il faut envoyer pour initialiser le Codec */

int    regs_1847[16]= {

    0xC000,      /* registre d'index 0 : CAN sur entrée gauche de Line1 , 0 dB d'atténuation */
    0xC100,      /* registre d'index 1 : CAN sur entrée droite de Line1 , 0 dB d'atténuation */
    0xC280,      /* registre d'index 2 : coupe voie gauche de l'entrée Aux1, +12 dB si présente */
    0xC380,      /* registre d'index 3 : coupe voie droite de l'entrée Aux1, +12 dB si présente */
    0xC480,      /* registre d'index 4 : coupe voie gauche de l'entrée Aux2, +12 dB si présente */
    0xC580,      /* registre d'index 5: coupe voie droite de l'entrée Aux2, +12 dB si présente */
    0xC600,      /* registre d'index 6 : CNA validé sur sortie gauche de Line , 0 dB d'atténuation */
    0xC700,      /* registre d'index 7 : CNA validé sur sortie droite de Line , 0 dB d'atténuation */
    0xC850,      /* registre d'index 8 : format des données échangées avec le DSP */
                    /* C/L = 0 : format linéaire ( pas de compression ) */
                    /* PCM=1 : 16 bits  signés */
                    /* S/M=1 : en stéréo */
                    /* CSF2:0 = 000 : échantillonnage à 8 kHz ( voir doc AD1847 pour autres fréquences ) */
                    /* CSL = 0 :choix de XTAL1 à 24,576 MHz */
    0xC909, /* registre d'index 9 : Interface Configuration */
                    /* ACAL = 1 : autorise le lancement d'une autocalibration du Codec après son */
                    /* initialisation lorsque MCE repassera à 0 ( à faire après chaque mise sous tension ) */
                    /* PEN = 1 : sortie des données sur le CNA */
    0xCA00,      /* registre d'index 10 : pour la gestion de l'horloge commune de la liaison */
                    /* série en cas de plusieurs Codecs : ici un seul Codec en Bus Master, donc : */
                    /* CLKTS = 0 et XCTL1:0 = 00 toujours */
    0xCB00,      /* registre d'index 11 : pas de registre : usage futur : on envoie 00 dans rien !! */
    0xCC40,      /* registre d'index 12 : pour la gestion du protocole TDM */
                    /* FRS = 0 : 32 slots par trame */
                    /* TSSEL = 1 : système à 2 fils (voir pages 62 et 63 de ce document) */
                    /* rappelons que la clock de la liaison DSP-Codec est à 12,288 MHz */
    0xCD00,      /* registre d'index 13 : pour le mixage numérique entre sortie CAN et entrée CNA */
                    /* DME = 0 : pas de mixage ici */
                    /* atténuation de -1,5 dB si ce mixage était autorisé */
    0xCE00, /* registre d'index 14 : pas de registre : usage futur : on envoie 00 dans rien !! */
    0x8F00      /* registre d'index 15 : pas de registre : usage futur : on envoie 00 dans rien !! */
                    /* mais force le retour de MCE à 0, donc lance l'autocalibration !! */
    };

```

```

/* buffer de réception des données venant du Codec : taille de 3 Words : status, voie gauche, voie droite */
int    rx_buf[3];
/* buffer de transmission des données vers le Codec : taille de 3 Words : contrôle, voie gauche et voie droite */
/* ici initialisés pour configurer le TDM */
int    tx_buf[3]={0xCC40,    0,    0};

/* définition d'une structure de TCB pour le chaînage des DMA de la liaison DSP-Codec */
/* voir pages 46 à 50 du polycopié de cours complet */
/* un TCB complet comprend en fait 9 éléments et non 4 comme dans la présentation simplifiée page 26 */
typedef struct    {
    unsigned    ei;    /* non utilisé ici : multiprocessing */
    unsigned    em;    /* non utilisé ici : multiprocessing */
    unsigned    ec;    /* non utilisé ici : multiprocessing */
    unsigned    gp1;    /* registre d'usage général n°1 */
    unsigned    gp2;    /* registre d'usage général n°2 */
    unsigned**   cp;    /* pointe sur le TCB suivant dans la chaîne */
    unsigned    c;    /* taille du bloc à transférer à chaque DMA */
    int    im;    /* incrément pour calculer l'adresse du transfert suivant dans le DMA en cours */
    unsigned*    ii;    /* pointeur sur le buffer : adresse de base du buffer */
}    tcb;

/* le premier TCB de la chaîne de réception initialisé avec 3 mots par DMA à des adresses successives (c = 3 et im = 1) */
tcb    rx_tcb = {0,    0,    0,    0,    0,    0,    3,    1,    0};
/* le premier TCB de la chaîne d'émission initialisé avec 3 mots par DMA à des adresses successives (c = 3 et im = 1) */
tcb    tx_tcb = {0,    0,    0,    0,    0,    0,    3,    1,    0};
int*    tx_ptr;
int    tx_compteur;

/*****
/*
    interruption timer exécutée ici 4 fois par seconde
*****/
void    it_timer ( int sig_num )
{
    /* inverse l'état de la Led câblée sur la sortie FLAG2 */
    set_flag ( SET_FLAG2, TGL_FLAG );    /* extension du C : */
                                         /* mettre SET_FLAG pour l'éteindre et CLR_FLAG pour l'allumer */
}

/*****
/*
    interruption DMA en transmission sur tx_buf vide
*****/
void    it_dma_transmit ( int sig_num )
{
    /* vérifie si les 16 données d'initialisation ont été envoyées au Codec */
    if ( tx_compteur )
    {
        /* mettre le word dans tx_buf[0], pointer sur le suivant et décrémenter le comptage */
        tx_buf[0] = *tx_ptr++;
        tx_compteur--;
        /* n'oubliez pas que (ici en stéréo) tx_buf[1] (voie gauche) et tx_buf[2] (voie droite) peuvent contenir */
        /* des valeurs numériques que l'on peut envoyer */
        /* vers les CNA du Codec, tx_buf[0] contenant quant à lui le mot de contrôle (revoir le protocole TDM) */
    }
}

/*****
/*
    interruption DMA en réception sur rx_buf plein
*****/
void    it_dma_receive ( int sig_num )
{
    /* on mettra ici le soft de traitement du signal qui reçoit les échantillons (en stéréo) dans rx_buf[1] (voie gauche) et */
    /* rx_buf[2] (voie droite) et qui peut générer une sortie numérique à envoyer dans tx_buf[1] ou tx_buf[2] */
    /* ici on peut par exemple simplement renvoyer la valeur lue sur la voie Line1 gauche vers la sortie Line gauche */
    tx_buf[1] = rx_buf[1];
}

```

```

/*****
/*   initialisation du DSP
/*   et exemples de routine de commande du timer
/*   et d'insertion de code assembleur dans du C : asm ( " mnémorique assembleur " )
*****/
void    init_dsp ( void )
{
    timer_off ( );                /* extension du C ANSI inhibant le timer */
    timer_set( 10000000,    10000000 ); /* initialise TPERIOD et TCOUNT du timer */
                                     /* génération d'une IT lorsque TCOUNT=0 */
                                     /* ici à 40 MHz : entre chaque IT :  $10^7/40.10^6 = 0,25$  s */
                                     /* donc fréquence des IT du timer = 4 Hz */

    asm( " #include <def21060.h> " );
    asm ( " bit set mode1 NESTM; " ); /* imbrication d'interruption autorisée : ici timer, DMA */
                                     /* par mise à 1 du bit 11 NESTM du registre interne mode1 */
                                     /* exemple d'insertion de code assembleur dans du C */

    tx_ptr      = regs_1847;        /* pointeur vers données d'initialisation du Codec */
    tx_compteur  = 0;                /* init du comptage de mots à envoyer */

    interrupt ( SIG_TMZ,    it_timer ); /* installation de l'interruption timer sur TCOUNT = 0 */
                                     /* placée en bas niveau de priorité */
                                     /* pour le haut niveau utiliser SIG_TMZ0 au lieu de SIG_TMZ */
                                     /* toutes les 0,25 s, la routine it_timer sera exécutée */
}

/*****
/*   initialisation et configuration de SPORT0 du DSP
/*   autorisation et installation des vecteurs de DMA
/*   initialisation des TCB et lancement des DMA en émission et en réception
*****/
void    init_sport0 ( void )
{
    /* affectation et configuration des slots */
    sport0_iop.mtcs =    0x00070007; /* transmission sur slots 0,1,2,16,17,18 */
    sport0_iop.mrcs =    0x00070007; /* réception sur slots 0,1,2,16,17,18 */
    sport0_iop.mtccs =    0x00000000; /* pas de compression en émission sur tous les slots */
    sport0_iop.mrccs =    0x00000000; /* pas de compression en réception sur tous les slots */

    /* écriture dans le STCTL0 : le Transmit Control Register */
    /* les champs de bits sont définis dans sport.h */
    sport0_iop.txc.mdf      = 1; /* délai entre chaque trame du TDM en multi-channel */
    sport0_iop.txc.schen    = 1; /* autorisation du chaînage de DMA sur émission */
    sport0_iop.txc.sden     = 1; /* autorisation de la DMA sur émission */
    sport0_iop.txc.lafs     = 0; /* à 0 en multi-channel */
    sport0_iop.txc.ltfs     = 0; /* synchro trame active au niveau haut */
    sport0_iop.txc.ditfs    = 0; /* synchro trame à chaque chargement du transmit register */
    sport0_iop.txc.itfs     = 0; /* à 0 en multi-channel */
    sport0_iop.txc.tfsr     = 0; /* à 0 en multi-channel */
    sport0_iop.txc.ckre     = 0; /* échantillonnage des bits sur front descendant de la clock */
    sport0_iop.txc.gclk     = 0; /* clock générée en transmission seulement */
    sport0_iop.txc.iclk     = 0; /* clock générée en externe utilisée */
    sport0_iop.txc.pack     = 0; /* transmission de 32 bits en 2 fois 16 bits */
    sport0_iop.txc.slen     = 15; /* longueur d'un mot ( ici 16 bits ) -1 */
    sport0_iop.txc.sendn    = 0; /* format Endian ( = 0 : MSB en premier, = 1 : LSB en premier */
    sport0_iop.txc.dtype    = 1; /* justification à droite avec extension de signe aux MSB */
    sport0_iop.txc.spen     = 0; /* à 0 en multi-channel */

```

```

/* écriture dans le SRCTL0 : le Receive Control Register */
/* les champs de bits sont définis dans sport.h */
    sport0_iop.rxc.nch      = 31; /* en TDM, nombre de slots -1 */
    sport0_iop.rxc.mce      = 1; /* autorisation du TDM ou multi-channel */
    sport0_iop.rxc.spl      = 0; /* à 0 en multi-channel */
    sport0_iop.rxc.d2dma    = 0; /* non validation de la DMA en D2 : à 0 en multi-channel */
    sport0_iop.rxc.schen    = 1; /* autorisation du chaînage de DMA en réception */
    sport0_iop.rxc.sden     = 1; /* autorisation de la DMA en réception */
    sport0_iop.rxc.lafs     = 0; /* à 0 en multi-channel */
    sport0_iop.rxc.ltfs     = 0; /* synchro trame active au niveau haut */
    sport0_iop.rxc.irfs     = 0; /* synchro trame générée en externe */
    sport0_iop.rxc.rfsr     = 0; /* à 0 en multi-channel */
    sport0_iop.rxc.ckre     = 0; /* échantillonnage des bits sur front descendant de la clock */
    sport0_iop.rxc.gclk     = 0; /* clock générée en transmission seulement */
    sport0_iop.rxc.iclk     = 0; /* clock générée en externe utilisée */
    sport0_iop.rxc.pack     = 0; /* transmission de 32 bits en 2 fois 16 bits */
    sport0_iop.rxc.slen     = 15; /* longueur d'un mot ( ici 16 bits ) -1 */
    sport0_iop.rxc.sendn    = 0; /* format Endian ( = 0 : MSB en premier, = 1 : LSB en premier */
    sport0_iop.rxc.dtype    = 1; /* justification à droite avec extension de signe aux MSB */
    sport0_iop.rxc.spen     = 0; /* à 0 en multi-channel */

/* autorisation des interruptions DMA sur SPORT0 en réception et en émission */
    interrupt ( SIG_SPR0I,  it_dma_receive ); /* installation de l'interruption DMA en réception */
                                           /* cette it sera exécutée chaque fois que le buffer de */
                                           /* réception de 3 mots rx_buf sera plein */
    interrupt ( SIG_SPT0I,  it_dma_transmit ); /* installation de l'interruption DMA en émission */
                                           /* cette it sera exécutée chaque fois que le buffer de */
                                           /* transmission de 3 mots tx_buf sera vide */

/* initialisation du TCB de transmission pour le chaînage des DMA */

    tx_tcb.ii = tx_buf; /* pointe sur le buffer de transmission */
    tx_tcb.cp = &tx_tcb.ii; /* pointe sur lui-même donc paramétrage identique pour toutes les DMA */
                           /* qui vont s'enchaîner */
    *(int*)CP2 = ( (int)&tx_tcb.ii | 0x30000 ); /* initialise le registre CP2 du DSP */
                                           /* ( canal 2 de DMA en transmission ) */
                                           /* force le bit 17 ( bit PCI autorisant l'IT DMA ) */
                                           /* et le bit 16 ( MSB de l'adresse du premier TCB ) à 1 */
                                           /* initialise le premier paramétrage et lance la DMA en transmission */
                                           /* les 3 words de tx_buf vont être émis puis une IT DMA émission aura lieu */
                                           /* entraînant l'exécution de la routine it_dma_transmit et un paramétrage */
                                           /* automatique de la prochaine DMA avec les mêmes tx_tcb et tx_buf */
                                           /* ( ce n'est pas à l'utilisateur de lancer la prochaine DMA ) */

/* initialisation du TCB de réception pour le chaînage des DMA */

    rx_tcb.ii = rx_buf; /* pointe sur le buffer de réception */
    rx_tcb.cp = &rx_tcb.ii; /* pointe sur lui-même donc paramétrage identique pour toutes les DMA */
                           /* qui vont s'enchaîner */
    *(int*)CP0 = ( (int)&rx_tcb.ii | 0x30000 ); /* initialise le registre CP0 du DSP */
                                           /* ( canal 0 de DMA en réception ) */
                                           /* force le bit 17 ( bit PCI autorisant l'IT DMA ) */
                                           /* et le bit 16 ( MSB de l'adresse du premier TCB ) à 1 */
                                           /* initialise le premier paramétrage et lance la DMA en réception */
                                           /* les 3 premier words reçus seront stockés dans rx_buf */
                                           /* puis une IT DMA réception aura lieu */
                                           /* entraînant l'exécution de la routine it_dma_receive et un paramétrage */
                                           /* automatique de la prochaine DMA avec les mêmes rx_tcb et rx_buf */
                                           /* ( ce n'est pas à l'utilisateur de lancer la prochaine DMA ) */
}

```

```

/*****
/*      envoi des initialisations vers le Codec
*****/

void      init_codec ( void )
{
    tx_ptr          = regs_1847;          /* pointeur vers données d'initialisation du Codec */
    tx_compteur     = 16;                 /* init du comptage de mots à envoyer */

    /* puis on attend l'émission complète des 16 mots sous IT DMA par la routine it_dma_transmit */
    while ( tx_compteur )
        idle ();                         /* attente jusqu'à tx_compteur = 0 */

    /* on a vu qu'après le dernier mot de contrôle envoyé par le DSP, le Codec se met en autocalibration */
    /* on vient lire le Status Word émis par le Codec et reçu sous IT DMA réception dans rx_buf[ 0 ] */
    /* le bit 1 (bit ACI) du Status Word est à 1 pendant l'autocalibration ( 384 périodes d'échantillonnage ) et à 0 sinon */
    /* il faut d'abord attendre que le bit ACI soit à 1 : démarrage de l'auto-calibration */
    /* sinon on risque de trouver 0 tout de suite et de penser que l'auto-calibration est finie alors qu'elle n'a pas commencé */
    /* on fait pour cela une scrutation de l'état haut de ACI */
    while ( !( rx_buf[ 0 ] & 0x0002 ) )
        idle ();                         /* attente jusqu'à bit ACI = 1 : début auto-calibration */
    /* puis on attend la fin de l'autocalibration en scrutant le passage à 0 du bit ACI */
    while ( rx_buf[ 0 ] & 0x0002 )
        idle ();                         /* attente jusqu'à bit ACI = 0 : fin d' auto-calibration */
}

/*****
/*      Le main
*****/

void      main      ( void )
{
    int i ;
    /* initialisation du DSP */
    init_dsp ();

    /* reset du Codec : mise à 0 de la sortie FLAG0 relié à la broche /Reset du Codec */
    set_flag ( SET_FLAG0,CLR_FLAG);
    for ( i=0 ; i<0xFFFF ; i++ );        /* et attendre suffisamment longtemps */
    set_flag ( SET_FLAG0,SET_FLAG);      /* avant de remonter le reset */

    /* configuration du SPORT0 du DSP */
    init_sport0 ( );

    /* envoi des commandes d'initialisation vers le Codec : ces commandes sont envoyées sous DMA */

    init_codec ( );

    /* valide le timer et ses interruptions périodiques */
    timer_on ( );

    /* attente infinie, dérangée par des IT du timer et des DMA */
    while ( 1 )
        idle;

}

*****/

```